

Foundations of Machine Learning (CS5590)

Instructor: Saketh

November 24, 2025

Contents

Contents	i
1 Mimicing human Learning is ML	3
2 Some ML Examples	7
3 Formal setup: Supervised Inductive Batch Learning	9
4 Supervised Learning = Risk Minimization	11
5 Bayes Optimal: Classification	13
6 Bayes Optimal: Regression	17
7 Empirical Risk Minimization	21
8 Linear Regression	23
9 Linear Regression: Model Error	27
10 Linear Regression: Estimation Error	31
11 Estimation vs Model error	35
12 Binary Classification Using Linear Models	39
13 Logistic Regression and SVM Classification	41

14 Linear Classification: Estimation Error	45
15 Norm-Regularized Linear Models: Estimation Error	47
16 ERM Solvers: Gradient Descent	51
17 A Closer Look at Gradient Descent	53
18 SGD : An alternative to ERM	55
19 Convergence of SGD	59
20 SGD vs SGD+ERM vs GD+ERM	63
21 Probabilistic Modeling: Likelihood Estimation	67
22 Maximum Likelihood Estimation MLE	69
23 Family of Exponential Models	73
24 Generative models for Regression and classification	77
24.1 Generative Linear Regression	77
24.2 Bayes Classifier	78
24.2.1 Naïve Bayes Classifier	80
25 Discriminative Modelling	83
26 Discriminative models for regression and classification	85
26.1 Discriminative Linear Regression	85
26.2 Discriminative Logistic Regression	86
27 Model Selection	89
28 Linear Models in Infinite Dimensional Spaces	91
29 Kernel Methods	95

<i>CONTENTS</i>	iii
29.1 Kernel Trick	96
29.2 Kernels	97
30 Modularity in Modelling	101
31 Deep Learning Models: Neural Networks	103
32 Back propagation	107
33 Overparametrized Models	109
34 Non-parametric Models: k-nearest neighbour	115
34.1 Nearest Neighbour Classification	116
34.2 k -NN Classification	117
34.3 k -NN Regression	117
35 Non-parametric Models: KDE	119
35.1 KDE based generative models	120
36 Generative AI	123
36.1 Energy based Models	123
36.2 Langevin Sampling	124
36.3 Conditional Sampling	125
37 Online Learning	127
37.1 Inductive Bias $\implies$ Online Learning	129
38 Online Convex Optimization	133
38.1 Regret of Online Gradient Descent (OGD)	135
38.2 Online to Batch conversion (OGD $\rightarrow$ SGD)	135
39 Averaging (Ensembling)	137
40 Boosting	143

40.1	Square loss: Matching Pursuit	144
40.2	Exponential loss: AdaBoost	145
41	Structured Prediction: Graphical Models	147
42	Models for Sequences: RNN encoder & decoder	149

Lecture 1

Mimicing human Learning is ML

We began by observing some aspects of human learning. In simple words, humans typically interact with world and “learn”. Learning is perhaps the process of improving their cognitive abilities¹. And, when confronted with a task, they “think” based on their learnings and accomplish the task. Thinking is perhaps the process of using their existing cognitive abilities in a mathematically/logically sound way to accomplish the task. We also noted that learning also involves “thinking”, with the key factor being that in learning there is interaction with external knowledge (world).

The grand goal of Machine Learning (ML) is: “To develop computer programs that can mimic the process of human learning for accomplishing complex tasks (e.g., solving problems in Astrophysics, Biology, Environment etc.)”². The important keyword here is mimic (rather than copy/replicate). This is because today’s science is very far from understanding how humans learn.

Though understanding the human brain and the learning process is complicated and difficult to replicate, there are a few aspects that are relatively easier to replicate in modern computer systems. For example, human interaction with world to-a-large-extent can be modelled via data flow through I/O devices (and it’s storage). The process of thinking can be modelled very well via mathematical/logical deductions/reasoning/inference. And ofcourse, real-world tasks are readily tangible and correspond to scientific problems in various domains, most of the time with well-studied frameworks for evaluating the performance etc.

We noted that, without loss of generality, we can model tasks as (mathematical) functions. For e.g., consider the task: given any image, describe it in

¹Please refer pg 5–6 in <https://www.overleaf.com/read/rksrtrtsbvnr#81f68b> for a short description of cognitive abilities.

²Also read section 1.2 in Shalev-Shwartz and Ben-David [2014].

words. Here, image is the input to the function and the caption is the output. As another example, consider the task: given observable parameters for weather on any particular day, what is the likelihood for rainfall being $10\text{mm} \pm 2\text{mm}$. Here, the interval of amount of rainfall is the input to the function and the output is corresponding likelihood. So the function is a likelihood function. This function that models the task is also known as the **predictive function** for that task.

Based on these observations, classical AI (1980-1990) focused on manually building knowledge databases (expert systems) and using powerful inference machines that can use these databases to accomplish complex tasks. In medieval age (1990-2010), research focussed on using sophisticated mathematical models for automatically building such expert systems from cheaply procurable/available data, leading to what is known as the field of ML, Machine Learning. The cheaply available data is known as **training data** and the sophisticated mathematical models are known as **ML models**. In other words, ML models play the role of a human-brain and adapting them to a particular task is dubbed as “learning” in machines. Accordingly, a **machine learning algorithm** is nothing but a function/procedure that takes input as a ML model, and data, and gives output as the task-relevant predictive function. $A : \mathcal{M} \times \mathcal{D} \rightarrow \mathcal{F}$, where $\mathcal{M}$ is a set of ML models (e.g., neural networks models or kernel machines etc.), $\mathcal{D}$ is a set of plausible training data relevant to the task at hand, and $\mathcal{F}$ is some syntactically correct set of potential predictive functions. For example, for the task of predicting current flow given voltage potential, $\mathcal{F}$ can be set of (non-negative) real-valued linear functions over reals.

Another important aspect in human learning is that: no learning is assumption-free i.e., having a bias is fundamental to learning. For e.g., consider the simple task where the baby needs to recognize various animals. Perhaps the mother (teacher/supervisor) picks a picture-book to show images of animals and she will recite the corresponding names. After showing few pics/books, the baby becomes efficient in recognizing animals by their names. Even in this simple and preliminary task, the baby (sub-consciously perhaps) makes an important assumption “similarly looking animals are named similarly”. If the baby does not make this assumption (i.e., does not have this inherent bias), then we are not sure what learning even means in this context. Imagine a weird world, where two particular tigers are named differently: one a tiger and another a bat; while a particular rat is also called a tiger. It is clear that we humans, with our current biases, will fail miserably in such a world, even in such basic tasks. In simpler words, consider the task of learning given names of humans. This seems impossible for us (for our bias)! So we resort to explicitly memorizing names. While **section 1.1 in Shalev-Shwartz and Ben-David [2014]** shows how learning in pigeons may not be useful, <https://pubmed.ncbi.nlm.nih.gov/21965161/> argues that pigeons can

solve complicated optimization problems. So what is important is an alignment of bias and the task at hand. In summary, to mimic human learning process, we need to have mechanisms to enforce biases/assumptions in ML models. And we expect that choosing ML models that are “ideal” for the task may be crucial. In-fact, there are theorems which formally prove that assumptions are fundamental to machine learning too³.

In figure 1 a pictorial representation of a typical ML pipeline is shown, as per the above discussion.

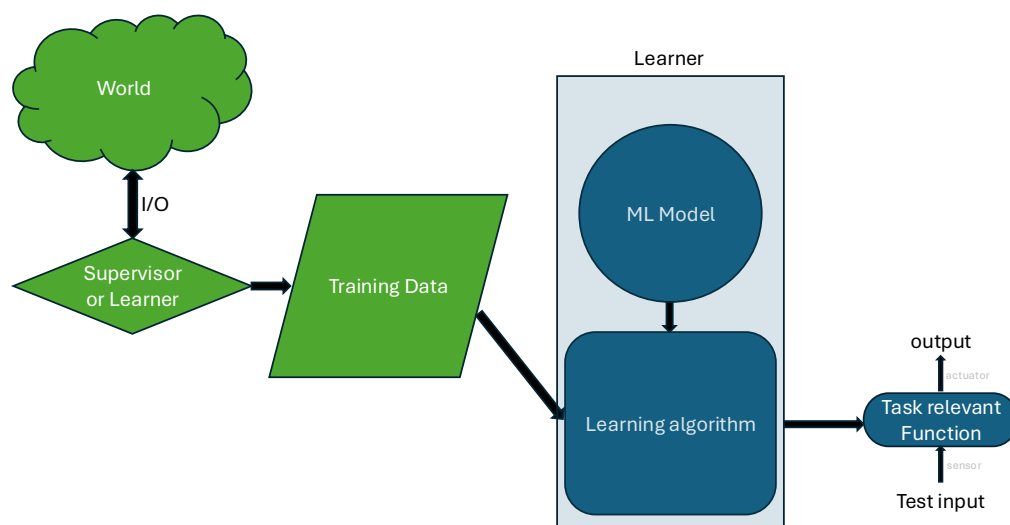


Figure 1.1: Typical ML pipeline

From the above example of mother teaching baby, another important aspect of human learning can be highlighted: to test/evaluate the baby’s learning, one will ask it to recognize names of animals in new pics/books. If one of the pics already used by the mother are shown, then it is not clear if the baby recognizes by mere memorization or through meaningful generalization. Hence, even in machine learning, evaluation of the final predictive function must be done via novel inputs (and corresponding outputs), which did not appear in training data. Such novel data is called as “test data”.

Refer [section 1.1 in Shalev-Shwartz and Ben-David \[2014\]](#). Interested students may read [section 1.4 in Shalev-Shwartz and Ben-David \[2014\]](#) to know the

³<https://arxiv.org/pdf/2304.05366>

relationship between ML and other subjects. See also sections 1.1, 1.2 in Mohri et al. [2018].

Lecture 2

Some ML Examples

In order familiarize ourselves with the ML pipeline, we began with a known problem and understood the same as a ML problem via this pipeline.

Given an uncalibrated resistor (located in a certain unknown, yet fixed, environment), consider the task of predicting the current flow through it for any given voltage potential. This task was accomplished by us during our school days. We can in fact understand this entire process using the previously stated ML pipeline. Observe that the prediction function is a function of the form $f : \mathbb{R}^+ \mapsto \mathbb{R}^+$ in this case (takes voltage as input and predicts current as output). So $\mathcal{F}$ in our ML pipeline is all such (syntactically correct) functions. In fact, the relevant model that was taught to us in school goes with the name ‘Ohm’s Law’, who believed the existence of an inherent property called resistance, which relates voltage to current by the popular law: $V = IR$. So, in school, we assumed the prediction function is further a linear function. Accordingly, the ML model in our pipeline can simply be a set of linear functions parametrized by their slope, $1/R$. The least-squares-fit algorithm we used in school days can now be dubbed as the learning algorithm. Once this algorithm is run, we have our estimate of $1/R$, which can be used in evaluation/deployment via simple inference/deduction: $I = V/R$.

While the above example makes us feel at home, an important and perhaps the most crucial aspect of human learning is that human’s performance at the given task improves with experience i.e., with more interaction with the environment, they tend to become better skilled. Analogously, we expect that the quality of the final prediction function learnt (via the ML pipeline) improves with the training dataset size. Or at least, asymptotically, as $m \rightarrow \infty$, where m is training dataset size, we wish to know if the quality of the prediction function reaches the maximum possible?

So a critical question is whether something like this can be proved/guaranteed

for the linear least-squares-fit learning algorithm? The answer is “yes” under mild assumptions. Answers to such questions are in-depthly studied in the subject called [statistical learning theory](https://people.iith.ac.in/saketha/teaching/cs5470.html) (for e.g., see <https://people.iith.ac.in/saketha/teaching/cs5470.html> course). We will give a rough sketch of the usual justification in a subsequent lecture.

For a deeper understanding: consider an algorithm that only finds the unique line passing through first two training datapoints. Given that our training points are obtained from lab experiments, where readings are made through devices of finite precision and the fact that the lab environment is stochastic (i.e., environmental conditions like humidity/temperature change, but on an average they are preserved), this unique line may not even pass through the origin, and hence cannot be the “correct” one. And, ofcourse the quality of the prediction function with such an algorithm cannot improve with a larger training set, as it only looks at the first two datapoints and ignores the rest.

In this specific example of current/amperage prediction and other similar applications, observe that the fundamental ML problem is simplified as that of [function induction](#). In the sense that, given function values at m points, induce it's value at others. Mathematically, this problem makes sense only if further assumptions are made. Such assumptions are dubbed again as ML modelling assumptions, more specifically as [inductive bias](#).

In order to give a completely different example, to which CS students might relate, we considered this problem: Suppose you have a fixed, yet unknown boolean function $f^* : \{0,1\}^n \mapsto \{0,1\}$. While we are not given f^* , we are given a set of boolean functions, F , that contains f^* . Ofcourse, F is subset of all possible 2^{2^n} boolean functions. Similiar to the baby & mother interative learning example from lectures, to help you figure out f^* from candidates in F , the following is done:

1. x_1 (a n-boolean input) is shown - then you need to guess $f^*(x_1)$ - then you either make a mistake or not - in either case $f^*(x_1)$ is then revealed.
2. x_2 (another n-boolean input) is shown - then you need to guess $f^*(x_2)$ - then you either make a mistake or not - in either case $f^*(x_2)$ is then revealed.
3. so on continues forever.

In a subsequent lecture we will discuss this example and intriguingly see that non-trivial function induction is possible without making any further assumptions!

Lecture 3

Formal setup: **Supervised** Inductive Batch **Learning**

We will now try to formally define and study learning problems, which closely resemble the Ohm's law problem from school. Intuitively, we understand such problems, but our objective is to write down the formal definition of the problem. Once this is done, we can study possible solutions to this mathematical problem.

Recall that the intuitive goal in such problems is that of function induction, i.e., given the function values at few domain points, induce them ever after. We began by denoting the training set by $\{(x_1, y_1), \dots, (x_m, y_m)\}$. These are the observations curated/collected by the supervisor. Note that observing the function value always in the training set is a characteristic of "Supervised Learning" and the entire set is available before learning and evaluation only starts after learning in "Batch learning". Accordingly, we denote the other plausible observations, which the supervisor may curate/collect "ever after" by $\{(x_j, y_j) \mid j = m + 1, \dots, \infty\}$, for evaluating/deploying the learner.

We then formally wanted to say: the problem of Supervised Inductive Batch Learning, **Supervised Learning**, in short, is to find a function, f^* , such that $f^*(x_i) = y_i \forall i = 1, \dots, m, \dots$. But this is an ill-posed problem, because, in general, there could be many many functions which match with f^* for $i = 1, \dots, m$ and differ arbitrarily ever after.

In the context of supervised learning, one assumption that could alleviate this issue and is perhaps not a restrictive assumption is (reasonable to assume in Ohm's law expt too): "the kinds of examples (input-output pairs) in training set and ever after are the same". More mathematically, we assume, there exists

an unknown, yet fixed, likelihood¹, $p(x,y)$, and all the datapoints in the training set and those coming ever after are merely samples from this likelihood. In other words, the first sample, (X_1, Y_1) is a random variable distributed as $p(x, y)$, second sample, (X_2, Y_2) is another random variable also distributed as $p(x, y)$, ... so on. In other words, (X_i, Y_i) $i = 1, \dots, m, \dots$ are all id (identically distributed). Note that we are not assuming $p(x, y)$ is known. We are only assuming it generates the data, both during training and ever after.

Intuitively, we can understand $p(x, y)$ as a mathematical model for the supervisor/evaluator in human learning, who typically collects observational data for training/evaluation/deployment. The key assumption is that the nature/environment remains the same during training data collection and actual deployment.

The methodology of using such stochastic assumptions to enable ML is known as [stochastic learning framework](#). This lecture is an expansion of [section 2.1 in Shalev-Shwartz and Ben-David \[2014\]](#).

¹Instead of saying mass function for discrete variables, density for continuous, and measure for general cases, ML folks use the term likelihood. Depending on situation it must be interpreted accordingly. Also, sometimes $p(x, y)$ is used to denote the likelihood function and sometimes its evaluation at x, y . This also needs to be understood as per the context.

Lecture 4

Supervised Learning = Risk Minimization

Now that we have the necessary assumptions in place, we would like to formally define the supervised learning problem under this stochastic framework. Going back to the intuition of function induction, now the goal seems to be find f^* , such that $f^*(X_i) = Y_i, \forall i = 1, \dots, m, \dots$. But unlike previously, where X_i, Y_i were replaced by x_i, y_i , now we have a different issue! In general, there may be no function satisfying the conditions. For example, consider the case where $X \sim q(x)$, and $Y = g(X) + N$, where q, N are some fixed likelihoods, and g is a fixed function. This is the case when there is parallax-error/ampere-meter-least-count in Ohm's law experiment. Further, if X is independent of N , then, by definition, $g(X) + N$ cannot be written as some function of X only. Thus there can exist no function f^* satisfying our conditions¹.

The root cause of this issue, is $p(y|x)$ being potentially non-degenerate i.e., given x the value of y need not be deterministic. Whereas, in function induction, y is supposedly fixed given x . So there is bound to be error of representing a random variable via a fixed quantity. Such an error can be quantified only if it is known how to compute error/mismatch between two different outputs. So, we assume a so-called **loss function**, $l : \mathcal{Y} \times \mathcal{Y} \mapsto \mathbb{R}$, which quantifies how two different outputs differ, is known. Here, $\mathcal{Y}$ is the output-space.

For example, in the case of Ohm's law, outputs are real, so a possible loss function is $l(y, y') \equiv (y - y')^2$. This quantifies how y, y' differ from each other. This is called **squared-loss or l_2 loss**. Similarly, one can choose the loss as $l(y, y') \equiv |y - y'|$. Ofcourse there are many such options for loss functions. Which loss

¹Please note the subtle difference. $f(x_i) = y_i \forall i$ may have too many solutions; whereas, $f(X_i) = Y_i \forall i$ may not have any solution!

function is employed for defining the learning is an ML modelling choice. Usually, apart from domain-knowledge, considerations like computational feasibility etc. determine this choice².

With this, intuitively, the goal is to perhaps find a function that incurs the least loss. But the issue is that each example/datapoint induces a loss-value i.e., f incurs loss $l(f(X_i), Y_i)$ on the i^{th} example (X_i, Y_i) . Then, which datapoint's loss needs to be minimized ? Since the task is function induction, we wish to find the function that achieves the least loss on all points. For CS algorithms students, this is immediately clear that if something needs to work for all inputs, then it must work for the worst-case input too. Accordingly, our objective of supervised learning must be³:

$$(4.1) \quad \arg \min_f R[f] \quad \text{where,}$$

$$(4.2) \quad \mathcal{R}[f] \equiv \max_{i=1, \dots, m, \dots} l(f(X_i), Y_i) \quad (\text{Worst case loss})$$

There are a couple of issues with this definition:

1. Firstly, if loss is unbounded, then for any positive likelihood, $p(x, y) > 0 \forall x \in \mathcal{X}, y \in \mathcal{Y}$, for example Gaussian, $\mathcal{R}[f] = \infty$ for any f . (Think why?)
2. If loss is bounded⁴, the objective treats very unlikely input-output pairs at par with very likely input-output pairs. This renders the solution less “desirable” or useless. Infact, the minimizer would be a constant function, with this constant as the “midpoint” of plausible outputs in terms of the loss. More specifically, $f^*(x) = \arg \min_{y \in \mathcal{Y}} \max_{y' \in \mathcal{Y}} l(y, y')$. So it's like ignoring inputs!

Hence, we do not proceed with this “CS friendly” objective and weight the loss of a pair with it's likelihood, so that the most likely points have a better say. Formally, this can be written as⁵:

$$(4.3) \quad \arg \min_f R[f] \quad \text{where,}$$

$$(4.4) \quad \mathcal{R}[f] \equiv \mathbb{E} [l(f(X), Y)] \quad (\text{Expected loss})$$

Expected loss is called as **Risk**. In other words, the objective of supervised learning is **Risk minimization**. This lecture is an expansion of **definition 2.1 in Bach [2024]**, **definition 3.3 in Shalev-Shwartz and Ben-David [2014]**, **definition 2.15 in Mohri et al. [2018]**.

²For mathematical correctness, we assume it is measurable too.

³Minimization can be over all possible measurable functions.

⁴This case was elaborated in lecture with a phase-change example in Ohm's law expt.

⁵Other meaningful alternatives are minimize median of loss or mode of loss or loss at some percentile etc. But minimizing average loss is most popularly studied in textbooks for its elegance

Lecture 5

Bayes Optimal: Classification

Any solution to the risk minimization problem is called as a [Bayes Optimal](#) predictor. In other words:

Definition 5.0.1. *Estimating the Bayes optimal given samples from the underlying likelihood is the problem of Supervised Learning.*

The risk of Bayes optimal is known as [Bayes Optimal Risk](#). In order to become more familiar with this problem, we wished to closely study it in some special cases.

We begin with the special case where the output space is a finite set. This is known, in general, as a [classification problem](#) and the finite possible outputs are called as [classes](#). The specific output for a given input, which is one of the classes is called the [label](#) of the input. Further, if there are only two classes, it is known as a [Binary Classification](#) task. E.g., ML applications like email spam filtering, product sentiment analysis, Windows-Hello/Apple-Face-ID typically involve a Binary classification task. If there are more than two classes, it is often qualified as a [multi-class classification](#) task. E.g., ML applications like online credit-card/loan approval, Google News involve a multi-class classification problem.

If in a classification problem, there is no known structure among the classes, i.e., each class is unique in it's own way bearing no defined similarity with the other classes, then, the most appropriate loss perhaps is the so-called [0 – 1-loss](#):

$$(5.1) \quad l(y, y') \equiv \begin{cases} 1 & \text{if } y \neq y' \\ 0 & \text{otherwise.} \end{cases}$$

In other words, it is zero if labels match and unity otherwise (mismatch loss). If there are other known structures among the classes, for e.g., if classes are

“yes”, “maybe”, “no”, then perhaps there is a total order among them. In such cases, loss can be appropriately chosen: for e.g., a symmetric loss such that $l(\text{“yes”}, \text{“maybe”}) \equiv 1, l(\text{“no”}, \text{“maybe”}) = 1, l(\text{“yes”}, \text{“no”}) = 2$. Such classification problems where classes have a total ordering are known as [ordinal regression](#) problems.

With l_{01} we derived¹ that the Bayes optimal is nothing but the mode of the posterior, $p(y|x)$, as a function of x . This is from [section 5.1.2 in Murphy \[2022\]](#), [pg 29 in Bach \[2024\]](#).

¹handwritten in fig 5.1 below.

Theorem 1 If loss is 0-1 loss, then the Bayes optimal is mode of posterior.

Proof Bay lemma 1,

$$f^*(x) = \underset{y \in \mathcal{Y}}{\operatorname{argmin}} E \left[\frac{L(y, Y)}{x} \right]$$

$$= \underset{y \in \mathcal{Y}}{\operatorname{argmin}} \sum_{\substack{y' \in \mathcal{Y}, \\ y' \neq y}} p(y'/x)$$

$$= \underset{y \in \mathcal{Y}}{\operatorname{argmin}} 1 - p(y/x)$$

$$= \underset{y \in \mathcal{Y}}{\operatorname{argmax}} p(y/x) \quad (\text{mode of posterior})$$

Theorem 2 If loss is squared loss, Bayes optimal is posterior mean.

$$\text{Proof: } f^*(x) = \underset{y \in \mathbb{R}}{\operatorname{argmin}} E[L(y, Y)/x] = \underset{y \in \mathbb{R}}{\operatorname{argmin}} E[(y - Y)^2/x]$$

$$= \underset{y \in \mathbb{R}}{\operatorname{argmin}} y^2 - 2y E[Y/x] + E[Y^2/x]$$

$$= \underset{y \in \mathbb{R}}{\operatorname{argmin}} (y - E[Y/x])^2 \quad (\because \text{Completing squares})$$

$$= E[Y/x] \quad (\text{posterior mean})$$

Lecture 6

Bayes Optimal: Regression

We derived an analytical expression for Bayes optimal with squared loss¹. It turned out to be the mean of the posterior, $p(y|x)$, as a function of x . This is from [section 5.1.5 in Murphy \[2022\]](#), [pg 30 in Bach \[2024\]](#). An important side result² of these derivations is that Bayes optimal has an equivalent alternative definition:

$$(6.1) \quad f^*(x) \equiv \arg \min_{y \in \mathcal{Y}} \mathbb{E}_{Y/x \sim p(y|x)} [l(y, Y) / x].$$

This is from [proposition 2.1 in Bach \[2024\]](#).

Now that we are familiar with the supervised learning objective and its avatars in special cases, we recalled the definition 5.0.1 of the supervised learning problem: solve risk minimization given training set. Mathematicians call such optimization problems where the objective is an expectation under a fixed likelihood, which is known through samples alone, as [Stochastic optimization](#) problems. So one strategy is to directly use algorithms for solving stochastic optimization problems.

The other strategy is an indirect strategy: estimate the underlying joint likelihood using the training set, and then solve the corresponding risk minimization (which is then a usual optimization problem). This is known as [generative modelling](#).

Another indirect strategy is: estimate the posterior likelihood using the training set, and then using it solve (6.1). This is known as [discriminative modelling](#).

We will begin with the direct approach in next lecture.

¹handwritten in fig 5.1 above.

²handwritten in fig 6.1, 6.2 below.

Derivations CS5590

Lemma 1: Bayes optimal, defined by, $f^* \equiv \underset{f}{\operatorname{argmin}} R\{f\}$,
is also given by:

$$f^*(x) = \underset{y \in Y}{\operatorname{argmin}} E[\ell(y, Y)/x] \quad \text{--- (I)}$$

Both these are equivalent definitions.

Proof: $f^* \equiv \underset{f}{\operatorname{argmin}} E[\ell(f(x), Y)]$

$$\begin{aligned} \min_f R\{f\} &= \min_f E[E[\ell(f(x), Y)|x]] \\ &\stackrel{\text{Total Expectation rule}}{=} \min_{f(x) \in X} \int E[\ell(f(x), Y)|x=x] dP(x) \quad \equiv g(x; f) \\ &= \int \min_{f(x)} E[\ell(f(x), Y)|x=x] dP(x) \end{aligned}$$

See claim (1) in APPENDIX

Same problem as in (I)

Exchange is justified

Let g . Note that $g(x; f)$ is only a function of $f(x)$ and is independent of $f(x')$ for $x' \neq x$. And the variables $f(x), f(x')$ have no constraints coupling them.

$$\min_{x, y} f(x) + g(y) = \min_x f(x) + \min_y g(y)$$

Area of g is minimized iff $g(y)$ is minimized. (1)

Figure 6.1: Equivalent Bayes optimal expression

APPENDIX

Claim 1: $\min_f \int g(f(u), u) du \stackrel{?}{=} \int \min_{a \in R} g(a, u) dx$

Overall fixed valued function.

~~(And, $g(a, u)$ is concave of LHS)~~
 In fact, if $a^*(u) = \arg\min_{a \in R} g(a, u)$, then a^* is a reduction of LHS.

Proof:

We have:

$$g(f(u), u) \geq \min_a g(a, u) \quad \forall u, \forall f \quad (\because \text{defn. of min})$$

$$\Rightarrow \int g(f(u), u) dx \geq \int \min_a g(a, u) dx \quad \forall f$$

$$\Rightarrow \min_f \int g(f(u), u) du \geq \int \min_a g(a, u) du.$$

But if $f = a^*$, then LHS = RHS.

Hence claim is proved.

Claim 2: Let $f(x) = \frac{1}{2} x^T A x + b^T x + c$. Then,

$$\nabla f(x_0) = \frac{1}{2} (A + A^T) x_0 + b$$

$$\nabla^2 f(x_0) = \frac{1}{2} (A + A^T)$$

(A)

Figure 6.2: Exchange integral-min.

Lecture 7

Empirical Risk Minimization

Since the key difficulty in solving (4.3) seems to be in computing the objective (true risk), one obvious idea is to perhaps approximate the (true) risk with average loss computed over training set: $\hat{R}_m[f] \equiv \frac{1}{m} \sum_{i=1}^m l(f(X_i), Y_i)$, which we call as the **Empirical Risk** (with f). The motivation for this is the law of large numbers that guarantees that the average (loss with f) over the training set will be arbitrarily close to the expectation (of loss with f), with arbitrarily high probability, provided m is big enough!

This motivates the idea of **Empirical Risk Minimization**, also known as Stochastic Average Approximation:

$$(7.1) \quad \hat{f}^m \equiv \arg \min_f \hat{R}_m[f]$$

as a proxy for true risk minimization, (4.3). However, this does not work: pick any function such that $f(x_i) \equiv y_i \forall i = 1, \dots, m$ and is defined as an extreme value elsewhere. Such a function would have extremely high expected loss (true risk) but zero empirical risk. Infact, we can design too many such bad functions, given any generating likelihood. So $R[\hat{f}_m^{\text{ERM}}]$ could be arbitrarily higher than Bayes optimal risk. Interested students may refer corollary 5.2 in Shalev-Shwartz and Ben-David [2014] for a formal statement of the same.

One strategy to alleviate this issue is to ofcourse restrict the search space of functions by avoiding arbitrary functions, so that information about function at some points can restrict possible function values at others. This is the idea in modelling functions or introducing inductive bias. For example, the Ohm's law setup avoids this issue by modelling linear functions alone. With such inductive biases in place, the risk cannot be arbitrarily different in the training set and ever after, just because biased functions do not behave arbitrarily! A formal treatment of such issues is postponed to a statistical learning theory course.

So ERM is usually:

$$(7.2) \quad \hat{f}_{|\mathcal{F}}^m \equiv \arg \min_{f \in \mathcal{F}} \hat{R}_m[f]$$

We expect $R[\hat{f}_{|\mathcal{F}}^m]$ is close to $R[f_{|\mathcal{F}}^*]$, where $f_{|\mathcal{F}}^*$ is the (true) risk minimizer over $\mathcal{F}$ (Bayes optimal restricted to $\mathcal{F}$). The difference between risks of $\hat{f}_{|\mathcal{F}}^m$ and $f_{|\mathcal{F}}^*$ is known as [estimation error](#), which for reasonable $\mathcal{F}$, decays with m .

However, because $f^* \notin \mathcal{F}$, further error is induced: the difference in risks of Bayes optimal and it's restriction is called as [approximation/model error](#) (which ofcourse does not decay with m). Sum of these two errors is known as the [generalization error](#).

In order to understand ERM better, we consider a special case: regression problem with square loss and inductive bias as linear functions. If $\mathcal{X} = \mathbb{R}^n, \mathcal{Y} = \mathbb{R}$, then, linear functions are functions of the form $f(x) = w^\top x \ \forall \ x \in \mathbb{R}^n$. This is popularly known as [Linear Regression](#). The set of all functions that satisfy the inductive bias of being linear is itself sometimes referred to as the [linear model](#), $\mathcal{L}$. And, w , which identifies/indexes various functions in the model is known as the [parameter](#).

This lecture is based on [section 2.2 and beginning of section 2.3 in Shalev-Shwartz and Ben-David \[2014\]](#).

Lecture 8

Linear Regression

It is easy to see that the restricted Bayes optimal for linear regression is given by:

$$(8.1) \quad w^* \equiv \arg \min_{w \in \mathbb{R}^n} \mathbb{E} \left[\left(w^\top X - Y \right)^2 \right]$$

$$(8.2) \quad = \arg \min_{w \in \mathbb{R}^n} w^\top \mathbb{E}[X X^\top] w - 2w^\top \mathbb{E}[XY]$$

The corresponding ERM problem is given by:

$$(8.3) \quad \hat{w}^m \equiv \arg \min_{w \in \mathbb{R}^n} \frac{1}{m} \sum_{i=1}^m \left(w^\top x_i - y_i \right)^2$$

$$(8.4) \quad = \arg \min_{w \in \mathbb{R}^n} w^\top \left(\frac{1}{m} \sum_{i=1}^m x_i x_i^\top \right) w - 2w^\top \left(\frac{1}{m} \sum_{i=1}^m y_i x_i \right)$$

We then observed that the objectives in either problem are convex quadratic because the Hessian is a correlation matrix, which is always positive semi-definite for any likelihood. $v^\top \mathbb{E}[X X^\top] v = \mathbb{E}[v^\top X X^\top v] = \mathbb{E} \left[\left(X^\top v \right)^2 \right] \geq 0 \ \forall \ v$. Hence the optimal solution is where gradient of the objective is zero¹. In other words, they must satisfy these linear equations:

$$(8.5) \quad \mathbb{E}[X X^\top] w^* = \mathbb{E}[XY]$$

$$(8.6) \quad \left(\frac{1}{m} \sum_{i=1}^m x_i x_i^\top \right) \hat{w}^m = \left(\frac{1}{m} \sum_{i=1}^m y_i x_i \right)$$

In linear algebra terminology, (8.8) is known as the Normal equation corresponding to the system of equations: $w^\top x_i = y_i \ \forall \ i = 1, \dots, m$. Even if the latter does not have a solution, the former always has!

¹handwritten in fig 8.1 below.

As long as $p(x)$ is such that the correlation matrix, $\Sigma \equiv \mathbb{E}[XX^\top]$, is invertible (positive definite), the sample correlation, $\hat{\Sigma} \equiv \frac{1}{m} \sum_{i=1}^m x_i x_i^\top$, is also invertible, with probability one. So both these systems have a unique solution:

$$(8.7) \quad w^* = \mathbb{E}[XX^\top]^{-1} \mathbb{E}[XY]$$

$$(8.8) \quad \hat{w}^m = \left(\frac{1}{m} \sum_{i=1}^m x_i x_i^\top \right)^{-1} \left(\frac{1}{m} \sum_{i=1}^m y_i x_i \right)$$

Now, by applying results like law of large numbers (multiple times), we know $\hat{w}^m$ converges (in some appropriate sense) to w^* as $m \rightarrow \infty$. This is because the individual terms in the RHS of these equations correspond to the true mean and the sample mean respectively. In other words, ERM is a meaningful learning algorithm, atleast for linear models.

Encouraged by this intuition, we wanted to understand this learning process more formally.

Please refer [sections 3.1-3.4 in Bach \[2024\]](#), [section 9.2.1 in Shalev-Shwartz and Ben-David \[2014\]](#), [section 11.3.1 in Mohri et al. \[2018\]](#).

$$\text{Proof: } f(x) = \frac{1}{2} x^T A x + b^T x + c$$

$$= \frac{1}{2} x^T \left(\sum_i A_i x_i \right) + \sum_i b_i x_i + c$$

$$= \frac{1}{2} \sum_j x_j \sum_i a_{ij} x_i + \sum_i b_i x_i + c$$

$$\Rightarrow \frac{\partial f(x)}{\partial x_k} = \frac{1}{2} \left(\frac{\partial}{\partial x_k} (a_{kk} x_k^2) + \frac{\partial}{\partial x_k} \left(\sum_{i \neq k} a_{ik} x_i x_k \right) + \frac{\partial}{\partial x_k} \left(\sum_{j \neq k} a_{kj} x_k x_j \right) + \frac{\partial}{\partial x_k} (b_k x_k) \right)$$

$$= \frac{1}{2} \left(2 a_{kk} x_k + \sum_{i \neq k} (a_{ik} + a_{ki}) x_i \right) + b_k$$

$$= \frac{1}{2} (A^k x + A_k^T x) + b_k$$

$$\Rightarrow \nabla f(x_0) = \frac{1}{2} \begin{bmatrix} \frac{\partial f(x_0)}{\partial x_1} \\ \vdots \\ \frac{\partial f(x_0)}{\partial x_n} \end{bmatrix} = \frac{1}{2} \begin{bmatrix} A^1 + A_1^T \\ A^2 + A_2^T \\ \vdots \\ A^n + A_n^T \end{bmatrix} x + \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix}$$

$$= \frac{1}{2} (A + A^T) x + b$$

$$\Rightarrow \frac{\partial^2 f(x)}{\partial x_p \partial x_k} = \frac{1}{2} \frac{\partial}{\partial x_k} (a_{kp} x_p + a_{pk} x_k)$$

$$= \frac{1}{2} (a_{kp} + a_{pk})$$

$$\Rightarrow \nabla^2 f(x_0) = \frac{1}{2} (A + A^T)$$

Figure 8.1: Gradient and Hessian of quadratic function.

Lecture 9

Linear Regression: Model Error

In this lecture we wish to deep dive into the first error term in generalization, the model error. Ofcourse, if the underlying likelihood, $p(x, y)$, is arbitrary then model error could be arbitrarily high. One way to understand this term is to ask, for which non-trivial class/kind of likelihoods, the model error is zero/low?

We begin by realizing that the objective (8.1) is infact the distance between the (vector embeddings) of Y and $w^\top X$ in the canonical Hilbert (Euclidean) space of random variables, $\mathcal{H}$. This space is nothing but the set of all random variables (defined over the same probability space), closed under linear combinations endowed with the inner-product $\langle A, B \rangle_{\mathcal{H}} = \mathbb{E}[AB]$. In such a space, the distance is given by $\|A - B\|_{\mathcal{H}}^2 \equiv \langle A - B, A - B \rangle_{\mathcal{H}} = \mathbb{E}[(A - B)^2]$. Hence the restricted Bayes optimal, defined by (8.1), is same as:

$$(9.1) \quad w^* = \arg \min_{w \in \mathbb{R}^n} \|w^\top X - Y\|_{\mathcal{H}}^2$$

This is essentially same as projecting the vector embedding of Y onto the linear span of those of $X_1, \dots, X_n$. From standard (Euclidean) projection theorem, we know that the error vector, $N \equiv w^{*\top} X - Y$, with the optimal weights, w^* , will be such that $N \perp X_i \forall i = 1, \dots, n$ i.e., $\mathbb{E}[X_i N] = 0 \forall i = 1, \dots, n$, which is $\mathbb{E}[X N] = 0$ in short.

Now, if the embeddings of X_i are linearly independent, then we infact know that w^* is unique. This is true whenever correlation $\mathbb{E}[X X^\top]$ is invertible. This is because, $v^\top X = 0 \iff \|v^\top X\|_{\mathcal{H}}^2 = 0 \iff \mathbb{E}[(v^\top X)^2] = 0 \iff \mathbb{E}[v^\top X X^\top v] = 0 \iff v^\top \mathbb{E}[X X^\top] v = 0 \iff v = 0$, because $\mathbb{E}[X X^\top]$ is positive definite (by invertible assumption).

Uniqueness of w^* simplifies the situation: consider the likelihoods, $p(x, y)$, where $p(x)$ is arbitrary with only restriction that correlation is invertible. Then,

generate $X \sim p(x)$, then, $p(y|x)$ is given by the generating formula:

$$(9.2) \quad Y = \sum_{i=1}^n \alpha_i X_i + N, \quad \mathbb{E}[NX] = 0$$

where N is any rv such that $\mathbb{E}[NX] = 0$. Then we definitely know that w^* must be the same as α (vector has a unique representation under a given basis). So the risk of restricted Bayes optimal is $\|N\|_{\mathcal{H}}^2 = \mathbb{E}[N^2]$. However, the Bayes optimal for such likelihood is $\mathbb{E}[Y|x] = \sum_{i=1}^m \alpha_i x_i + \mathbb{E}[N|x]$. Therefore, the Bayes optimal risk is $\mathbb{E}[N - (\mathbb{E}[N|x])^2] = \mathbb{E}[\text{var}(N|x)]^1$. Hence, the model error is given by $\mathbb{E}[N^2] - \mathbb{E}[\text{var}(N|x)]$, which is equal to $(\mathbb{E}[N])^2 + \text{var}(\mathbb{E}[N|x])$ by total variance rule. If additionally, we assume zero-mean noise, $\mathbb{E}[N] = 0$, the model error reduces to $\text{var}(\mathbb{E}[N|x])$ or if we assume $N \perp X$, i.e., X is independent of N , then model error reduces to $(\mathbb{E}[N])^2$. If we assume both, like most books do², then model error is exactly zero!

In summary, the more assumptions we make on the generating likelihood, the better we can bound the model error. In the lecture, we also re-derived³ (8.7) starting from (8.1) using only projection theorem (avoiding any use of Hessian/gradient).

Infact, the condition of invertible correlation can also be relaxed. In such a case, w^* is not unique, so we may not be able to recover α . But the projection error (risk) with w^* will be exactly same as that with α . So the model error expressions given above will remain the same even for a general $p(x)$.

There is an analogous projection based interpretation for the ERM (8.3), which is more straight-forward as the space is usual space of Euclidean vectors. See section 11.2.2.2 in Murphy [2022] or section 3.3.2 in Bach [2024] for the same.

¹ $\text{var}(N|x)$ is the conditional variance.

²See eqn. (3.4) in Bach [2024], or eqn. (11.1) in Murphy [2022]

³Theorem 3 in handwritten notes in fig 9.1, 9.2 below.

Theorem 3 Derive (8.7) from (8.1) using Projection theorem. Assume $E\{xx^T\}$ is invertible.

Proof: From (8.1),

$$\begin{aligned}
 \omega^* &= \underset{\omega \in \mathbb{R}^n}{\operatorname{argmin}} E[(\omega^T x - y)^2] \\
 &= \underset{\omega \in \mathbb{R}^n}{\operatorname{argmin}} \|\omega^T x - y\|_{\mathcal{H}}^2 \quad \left[\begin{array}{l} \text{Vector Space of RVs with} \\ \langle A, B \rangle_{\mathcal{H}} \equiv E[AB] \\ \Downarrow \\ \|A-B\|_{\mathcal{H}}^2 = \langle A-B, A-B \rangle_{\mathcal{H}} \\ = E[(A-B)^2] \end{array} \right] \\
 &\left[\begin{array}{l} \text{If } x, \dots, x_n \text{ are orthogonal, then and} \\ \text{unit norm, then} \end{array} \right] \quad \omega_i^* = \langle y, x_i \rangle \\
 &= \underset{\omega \in \mathbb{R}^n}{\operatorname{argmin}} \|\omega^T (E[xx^T]^{-1/2} \tilde{x}) - y\|_{\mathcal{H}} \quad \text{① where } \tilde{x} \equiv E[xx^T]^{-1/2} x \\
 &= E[xx^T]^{-1/2} \underset{\bar{\omega} \in \mathbb{R}^n}{\operatorname{argmin}} \|\bar{\omega}^T \tilde{x} - y\|_{\mathcal{H}} \quad \begin{array}{l} \text{change of variables} \\ \bar{\omega} \equiv E[xx^T]^{1/2} \omega \end{array} \\
 &= E[xx^T]^{-1/2} \bar{\omega}^*, \text{ where, } \bar{\omega}_i^* = \langle \tilde{x}_i, y \rangle_{\mathcal{H}} \\
 &= E[xx^T]^{-1/2} \xrightarrow{\text{Projection theorem.}} E[\tilde{x}_i, y] \xrightarrow{\text{defn. of } \langle \cdot, \cdot \rangle_{\mathcal{H}}} \text{①} \\
 &= E[xx^T]^{-1/2} E[E[xx^T]^{-1/2} x] \rightarrow \text{①} \\
 &= E[xx^T]^{-1} E[xy], \text{ which is (8.7)}
 \end{aligned}$$

③

Figure 9.1: Linear regression as a projection.

Theorem 3 (Alternate derivation)

From (8.1)

$$w^* = \underset{w \in \mathbb{R}^n}{\operatorname{argmin}} E[(w^T X - Y)^2]$$

$$= \underset{w \in \mathbb{R}^n}{\operatorname{argmin}} \|w^T X - Y\|_{\ell_2} \quad \left(\text{Same space as in above derivation} \right)$$

By projection theorem, we know that $N \equiv w^{*T} X - Y$ is orthogonal to $X_1, \dots, X_n$. i.e. $\langle N, X_i \rangle_H = E[N X_i] = 0 \quad \forall i=1, \dots, n$

This leads to the central equation in linear regression:

$$Y = w^{*T} X + N, \quad E[NX] = 0$$

Now, w^* can be obtained by simple algebra:

$$\begin{aligned} E[YX] &= E[(w^{*T} X + N) X] \\ &= E[(X^T w^* + N) X] \\ &= E[X(X^T w^* + N)] \\ &= E[X X^T] w^* + E[XN] \\ &= E[X X^T] w^*, \text{ which is same as (8.5)} \end{aligned}$$

(3a)

Lecture 10

Linear Regression: Estimation Error

One important insight from the model error discussion is that even though model error is not zero, the linear coefficients, w , (which may represent physically meaningful quantities like the resistance in case of Ohm's law expt), are exactly recovered by the restricted Bayes optimal. So with respect to these coefficients, there is absolutely no difference between the restricted Bayes optimal and the (full) Bayes optimal. However, the Bayes optimal also additionally captures the information in the $\mathbb{E}[N|X]$ term, which is absent (set to zero) in case of the restricted Bayes optimal, leading to non-zero model error.

We then focused our attention on the estimation error term: $R[\hat{w}^m] - R[w^*]$. From (8.7)-(8.8) it is clear that the corresponding terms in RHS of both expressions are the true/population mean and empirical/sample mean respectively. So, from law of large numbers, it seems $\hat{w} \rightarrow w$ in some sense. In order to understand the convergence behavior more closely, we set out to compute it exactly in atleast one special (yet non-trivial) case.

Note that $R[\hat{w}]$ is itself random because $\hat{w}$ depends on the training set, which we assumed is a set of random samples from $p(x, y)$. Hence, estimation error can be either computed either in expectation or with high-probability, but not exactly. For simplicity of derivation, we restricted ourself to compute expected estimation error i.e., $\mathbb{E}[R[\hat{w}^m]] - R[w^*]$. Without any further assumptions, only using (9.2), we first derived¹ that $\mathbb{E}[R[\hat{w}^m]] - R[w^*] = \mathbb{E}[(\hat{w}^m - w^*)^\top \mathbb{E}[XX^\top](\hat{w}^m - w^*)]$.

As per (9.2), let n be the random vector with i^{th} entry as $n_i \sim N$, where $y_i = \sum_{j=1}^n w_j^* x_{ij} + n_i$, where x_{ij} is the j^{th} input (feature) of the i^{th} training

¹handwritten in fig 10.1 below. Also see prop 3.9 in Bach [2024].

sample. Using this notation, and eliminating y_i in (8.8) leads to $\hat{w}^m - w^* = \left(\frac{1}{m} \sum_{i=1}^m x_i x_i^\top\right)^{-1} \left(\frac{1}{m} \sum_{i=1}^m n_i x_i\right)$. Substituting this into the above expression for expected estimation error and simplifying by using an additional assumption: N is independent of X and the samples n_i are mutually independent, leads to²:

$$(10.1) \quad \mathbb{E}[R[\hat{w}^m]] - R[w^*] = \frac{\mathbb{E}[N^2]}{m} \mathbb{E}_{x_i \sim p(x)} \left[\text{trace} \left(\left(\frac{1}{m} \sum_{i=1}^m x_i x_i^\top \right)^{-1} \mathbb{E}[X X^\top] \right) \right]$$

In summary, the key term to be upper bounded is the Frobenius inner product³ (similarity) between expected inverse sample correlation and (true) correlation. Since sample correlation is pd (positive definite) and $h(X) = \text{trace}(X^{-1}M)$, $X, M \succ 0$ is a convex function, applying Jensen's inequality, we get lower bounds on estimation error: RHS of (10.1) $\geq \frac{\mathbb{E}[N^2]}{m} \left\langle \left(\frac{1}{m} \sum_{i=1}^m \mathbb{E}[x_i x_i^\top] \right)^{-1}, \mathbb{E}[X X^\top] \right\rangle = \frac{\mathbb{E}[N^2]}{m} \left\langle \left(\mathbb{E}[X X^\top] \right)^{-1}, \mathbb{E}[X X^\top] \right\rangle = \frac{n}{m} \mathbb{E}[N^2]$. Hence a lower bound on expected estimation error is:

$$(10.2) \quad \mathbb{E}[R[\hat{w}^m]] - R[w^*] \geq \Omega\left(\frac{n}{m}\right).$$

Interested students may read section 3.8.1 in Bach [2024] for a specific $p(x)$ (Gaussian), where this lower bound is (almost) achieved (assuming $m \gg n + 1$). You may also refer equation⁴ (3.31), (3.41) in Mohri et al. [2018].

²Proposition 3.10 in Bach [2024].

³Frobenius inner product is the default “dot product” in the vector space of matrices. $\langle A, B \rangle = \text{trace}(A^\top B) = \text{vec}(A)^\top \text{vec}(B)$.

⁴read VC-dimension as dimension, n . This is the same for linear models.

$$E\{R[\hat{\omega}] - R[\omega^*]\} = ? \quad (\text{looking for simplified expression}).$$

$$\begin{aligned} E\{R[\hat{\omega}]\} &= E\{E\{(\hat{\omega}^T X - Y)^2\}\} \quad \rightarrow (9.2) \\ &= E\{E\{(\hat{\omega}^T X - (\omega^{*T} X + N))^2\}\} \\ &= E\{E\{(\hat{\omega} - \omega^*)^T X X^T (\hat{\omega} - \omega^*) - 2(\hat{\omega} - \omega^*)^T X N + N^2\}\} \\ &= E\{(\hat{\omega} - \omega^*)^T E\{X X^T\} (\hat{\omega} - \omega^*) - 2(\hat{\omega} - \omega^*)^T E\{X N\} + E\{N^2\}\} \quad \rightarrow (9.2) \end{aligned}$$

$$R[\omega^*] = E\{(\omega^{*T} X - Y)^2\} \stackrel{(9.2)}{=} E\{N^2\}$$

$$\therefore E\{R[\hat{\omega}] - R[\omega^*]\} = E\{(\hat{\omega} - \omega^*)^T E\{X X^T\} (\hat{\omega} - \omega^*)\}$$

$$\begin{aligned} |\hat{\omega}^T X - \omega^{*T} X| &= |\hat{\omega}^T x - \omega^{*T} x| = |\hat{\omega}^T x_i - \omega^{*T} x_i + \omega^{*T} x_i - \omega^{*T} x| \quad \text{last term} \\ &\leq \underbrace{|\hat{\omega}^T (x - x_i)|}_{\text{ir term}} + \underbrace{|\omega^{*T} x_i - \omega^{*T} x|}_{\text{zero}} + |\omega^{*T} x_i - \omega^{*T} x| \quad \text{large output} \\ &\leq \|\hat{\omega}\| \|x - x_i\| \quad \rightarrow \end{aligned}$$

$\therefore$ ^{Norm} Regularized ERM norms are expected to have less estimation error than Unregularized ERM norms.

Figure 10.1: First: Simplified expression for estimation error in linear regression. Second: Intuition for parameter-norm controlling estimation error.

Lecture 11

Estimation **vs** Model error

In fact, complementary to the lower bounds for the (expected) estimation error (10.2), statistical learning theory helps one upper bound the estimation error¹:

$$(11.1) \quad R[\hat{w}^m] - R[w^*] \leq \mathcal{O} \left(\sqrt{\frac{n}{m}} \right) \text{ with high probability.}$$

Interested students may refer last para on pg 90 along with (4.10) in Bach [2024] and/or theorems 6.8, 9.3 in Shalev-Shwartz and Ben-David [2014] and/or (3.30) in Mohri et al. [2018] for more formal expressions².

The important take-home from the lower and upper bounds is the dependence of estimation error on $\frac{n}{m}$:

1. As $m \rightarrow \infty$, estimation error decays to zero. This is good news because, now, formally, we can call ERM as a valid learning algorithm (for linear models). Since it mimics human learning, where skills/efficiency improve with experience.
2. The speed of convergence, in terms of m , is in worst case $\sqrt{1/m}$, which might be very slow. i.e., tremendous amounts of training data may be required for small improvements!
3. There are some special cases, where better convergence rates of $1/m$ are possible. But, these seem to be best rates, in the sense that they cannot be improved in general. This is again not a very good news.

¹Holds for any (bounded) loss!

²read VC-dimension as dimension, n . This is the case for linear models.

4. As the following discussion suggests, the dependence on factors like n suggests an inherent trade-off between model error and estimation error. This trade-off is sometimes called as bias-variance tradeoff.

While it is clear the more the number of parameters, n , the worse it is for estimation. It is not clear how n controls the model error. To see this, consider general quadratic functions (over R): $ax^2 + bx + c$. Note that these can also be

written as $w^\top \phi(x)$, where $\phi(x) = \begin{bmatrix} x^2 \\ x \\ 1 \end{bmatrix}$. Similarly, cubics can be written as

linear functions over $\phi(x) = \begin{bmatrix} x^3 \\ x^2 \\ x \\ 1 \end{bmatrix}$. Even if there are two variables, i.e., $n = 2$,

such linear parametrization is possible with $\phi(x) = \begin{bmatrix} x_1^3 \\ x_2^3 \\ x_1^2 x_2 \\ x_1 x_2^2 \\ x_1^2 \\ x_2^2 \\ x_1 x_2 \\ x_1 \\ x_2 \\ 1 \end{bmatrix}$. In general, if there

are n dimensional inputs and d degree polynomials we need to consider, then it is same as linear functions over $\phi(x) \in \mathbb{R}^{\binom{n+d-1}{d}}$. So linear models can be used to model any Bayes optimal just by choosing the appropriate d . In other words, linear models are [universal](#) approximators. It is clear that higher the d , lower the model error (as we are missing less and less functions), however, estimation error grows atleast as $\approx O(\frac{d^n}{m})$, by Sterling's approximation. Hence on one hand we wish the number of parameters was higher to reduce model error, whereas, we wish the number of parameters was lower to control the estimation error.

In practice, this inherent trade-off always results in a mad race to find the Bayes optimal. This is because we want both errors to be low for their sum, the generalization error to be low. Imagine companies investing in building tech to handle large models (compute cost etc) and handling large data (collection/storage/retrieval costs). To begin with any company would start with a modest guess for the number of parameters and training set size. This is because if they overestimate these, then they may end up spending too much if the Bayes optimal is "easy" (Bayes optimal is linear over inputs say). Once companies find this is not enough (application not near the easy case), there will be a race

to pump up no. parameters and m . If m is high, there will be competition for compute sophistication, if no. parameters are high, there will be investment to procure more data. This mad race continues till someone finds an approximation of Bayes optimal that reaches human accuracy level (imagine LLMs in late 2022). Once it is known that there is still scope for finding the Bayes optimal (very complex function), even more companies will invest leading to a mad-rat-race with billions of dollars pouring in to find the Bayes optimal (the other LLMs 2023-now)! Meanwhile, many interesting applications around this Bayes optimal function may be built (e.g., code generation using OpenAI GPT-4.5/Anthropic Claude 3 Opus/Google Gemini 2.5 Pro etc.).

It is truly satisfying for an ML enthusiast to clearly see how the lower and upper bounds and the theoretical bias-variance trade-off discovered in early 1990s shaped up the reality in 2020s.

Here is another popular way of understanding this tradeoff, which is often covered in books. Let us assume m is fixed and recall that estimation error grows wrt $\frac{n}{m}$, where n is number of parameters. Then if n is too low (model is too simple), i.e., model error is dominating among the two errors (estimation error is low), then the training set error with ERM solution will be very close to the true risk with ERM solution because estimation error is low; but this will itself be (unnecessarily) high. This is because we potentially miss any look-alikes of the Bayes optimal. Such a regime is popularly known as [underfitting](#). An evidence of being in the underfitting regime is that the training set error will be significantly larger than a guess for the Bayes optimal error (say, error made by a human expert). Then if n is too high (model is too complex), i.e., estimation error is dominating among the two errors (model error is low), then the training set error with ERM solution will be very low because the number of functions covered are too many for the m . But this may be far from the true risk with ERM solution because estimation error is expected to be high. Such a regime is popularly known as [overfitting](#)³. An evidence of being in the overfitting regime is that the training set error will be significantly lower (say, zero) than a guess for the Bayes optimal error (error made by a human expert). If $n \gg m$, then the regime is known as the [overparametrization](#) regime. Mostly, overfitting and over-parametrization go together.

Please note that underfitting and overfitting are the operating regimes. In general, the sweet spot (min of generalization error) lies somewhere in between these two regimes. But good generalization may occur in the overfitting regime too, which will be discussed in detail at appropriate stage in the course. In short,

³This is the regime described formally in corollary (5.2) in Shalev-Shwartz and Ben-David [2014].

for example, if the number of parameters is too high, but the underlying likelihood is not the worst case for the lower bound (10.2), then the estimation error could be still low. Or when the ERM solution employed⁴ is not one of the “nasty” functions⁵ with high risk, then the restricted Bayes optimal risk may still be not far from the risk of the ERM solution employed. In such cases, one may observe good generalization in the overfitting regime too Bartlett et al. [2020]. The reader is encouraged to think: when can it happen that good generalization is achieved in the underfitting regime ? For more classical discussions along these lines, please refer pg 33 in Bach [2024], section 1.2.3 in Murphy [2022], sections 2.2.1, 2.3 (intro), fig 2.1 Shalev-Shwartz and Ben-David [2014].

However, this discussion raises an important question. Is there a different modelling strategy than linear modelling (where the inductive bias is on number of coefficients in basis expansion⁶ of the Bayes optimal) such that the bias-variance trade-off is better ? Say $\log(n)/m$ or independent of n ? The answer to this will be given while discussing kernel methods and their follow-up. Interested students may read up chapter 5 in Shalev-Shwartz and Ben-David [2014] for more details.

⁴Recall that there will be many ERM solutions in such cases as number of unknowns/parameters is too few compared to the number of samples.

⁵formally described in corollary (5.2) in Shalev-Shwartz and Ben-David [2014].

⁶Note that monomials are just one set of basis function for the space of functions. There are other possibilities, such as Fourier expansion, etc.

Lecture 12

Binary Classification Using Linear Models

We then began to study binary classification problems. Without loss of generality let us assume $\mathcal{Y} = \{-1, 1\}$, $\mathcal{X} = \mathbb{R}^n$. In case of regression problems, we directly modelled the Bayes optimal. This was because the Bayes optimal is the mean of the posterior, which could be any function from $\mathcal{X} \mapsto \mathcal{Y}$. However, in binary classification, Bayes optimal takes specific forms: $f^*(x) = \begin{cases} 1 & p(1|x) \geq p(-1|x) \\ -1 & \text{otherwise} \end{cases}$.

This is same as saying $f^*(x) = \text{sign}(2p(1|x) - 1)$. Hence, it seems more appropriate to model $g^*(x) \equiv 2p(1|x) - 1$ using linear models than to model $f^*(x)$ directly. Before studying modelling details, we tried to visualize the Bayes optimal classifier using the locus of all points where $g(x) = 0$. This is the “decision boundary/surface”. Points belonging to either side are labeled with each class respectively by the classifier. We took various simple examples and visualized this.

However, we know that $|g^*(x)| \leq 1$. Hence, we need to use models for functions whose range is $[-1, 1]$ and not entire $\mathbb{R}$. One general way to do this is to use the cosine definition: $g(x) \equiv \frac{w^\top \phi(x)}{\|w\| \|\phi(x)\|}$ i.e., normalized linear models.

Infact, careful normalization between $[-1, 1]$ is also not needed. As long as $\max_{x \in \mathcal{X}} |w^\top \phi(x)| < \infty$, it is fine because the sign does not change with positive scaling. Hence, linear models with an additional constraint $\|w\| \leq W$, for some $W > 0$, are also appropriate assuming $\max_{x \in \mathcal{X}} \|\phi(x)\| = R < \infty$ (data is bounded). Indeed, $\underbrace{|w^\top \phi(x)|}_{\text{Cauchy Schwartz}} \leq \|w\| \|\phi(x)\| \leq WR < \infty$. Linear models with constrained

parameters are known as [regularized/bounded linear models](#).

In simple words, we are avoiding functions like $\tan(x)$, $|x| < \frac{\pi}{2}$. This is because $\max_{|x| < \frac{\pi}{2}} |\tan(x)| = \infty$ and $\tan(x)$ can be arbitrarily closely approximated by linear models because of the (monomial based) series expansion. However, unless $W \rightarrow \infty$, in which case linear models are same as bounded linear models, functions like $\tan(x)$ cannot be arbitrarily closely approximated by bounded linear models.

Now that we have a model¹, we focused our attention on the loss function. While the default 0-1 loss is great, the major drawback is that the corresponding (true/emprical) risk minimization problems turn out to be computationally challenging². Thus one usually employs a surrogate loss that is a convex upper bound (convex majorant). Convexity make the corresponding risk minmization problems computationally tractable. Upper bound makes the risk bounds useful: if $l_{01} \leq l_s$, then $\mathbb{E}[l_{01}(f(X), Y)] \leq \mathbb{E}[l_s(f(X), Y)]$, where l_s is a surrogate. Hence, the generalization/estimation error bounds on surrogate loss directly imply bounds on 0-1 loss.

Since we wish to employ (regularized) linear models, the surrogate losses are usually defined in terms of candidate functions g rather than $f = \text{sign}(g)$. The simplest surrogate is the so-called **hinge loss**: $l_h(f(x), y) \equiv \max(0, 1 - yg(x))$. This is piece-wise linear. A more smoother but non-linear surrogate is obtained via the LSE (log sum exponential) approximation of the non-differentiable $\max$: $\max(0, -yg(x)) \approx \log(e^0 + e^{-yg(x)})$. With appropriate normalization (divide by $\log(2)$), this is also an upper bound. This normalization is often ignored as it does not matter for the corresponding risk minimization problems. This later loss: $l_r(f(x), y) \equiv \log(1 + e^{-yg(x)})$ is called the **logistic loss**. An even more aggressive surrogate is exponential loss $l_e(f(x), y) \equiv e^{-yg(x)}$.

Please read [section 4.1.1 in Bach \[2024\]](#) and/or [section 12.3 in Shalev-Shwartz and Ben-David \[2014\]](#) for more on loss surrogates.

¹But what if we model $h(x) \equiv \log(p(1|x)) - \log(p(0|x))$? Then also, the classifier is $f(x) = \text{sign}(h(x))$. In such a case though, bounded norm is not at all necessary as it seems $h(x) \in (-\infty, \infty)$. So where is the catch? Can you resolve this paradox? Is regularization then not necessary for modelling binary classifiers?

²Infact a more comprehensive statement can be made: refer Feldman et al. [2009] for details.

Lecture 13

Logistic Regression and SVM Classification

As seen in previous lecture, sometimes parameters in standard models may further be constrained to satisfy the requirements of the learning problem. In case of binary classification, since bounded functions are to be modelled, the appropriate constraint was of the form $\|w\| \leq W$. Such functions of parameters that are restricted as per the problem requirements (prior/domain knowledge) are known as **regularizers**. W is referred to as a **hyperparameter**. Given a W , the model is fixed.

If the loss was 0-1 loss, then W does not matter. This is because the loss is a function of $\text{sign}(w^\top \phi(x))$, which is independent of $\|w\|$. However, in case of the surrogate losses, W matters as these losses are functions of $w^\top \phi(x)$ and not $\text{sign}(w^\top \phi(x))$. In general, it is clear that higher the W , lesser the model error and vice-versa.

If hingeloss is used along with regularized linear models, the modelling is referred to as **SVM classification**. The risk minimization problem in this case is:

$$(13.1) \quad \min_{w \in \mathbb{R}^n} \mathbb{E} \left[\max \left(0, 1 - Y w^\top \phi(X) \right) \right]$$

$$(13.2) \quad \text{s.t.} \quad \|w\| \leq W.$$

In fact, from Lagrange duality theory, it is known that the solutions of the above problem (i.e., restricted Bayes optimal) are exactly the same as those of:

$$(13.3) \quad \min_{w \in \mathbb{R}^n} \frac{1}{2} \|w\|^2 + C \mathbb{E} \left[\max \left(0, 1 - Y w^\top \phi(X) \right) \right],$$

for some appropriate $C > 0$. In other words, we can employ (13.3) and hyperparameter as C rather than (13.1) and hyperparameter as W . The advantage of

(13.3) is that the objective is strongly convex and hence far easier to optimize (compared to the merely convex objective in (13.1)). Interested students may refer section 4.5.5 in Bach [2024] for further details. We will hence use this as default form unless to make a point.

If logistic loss is used along with regularized linear models, the modelling is referred to as [logistic regression](#). It's risk minimization problem reads as:

$$(13.4) \quad \min_{w \in \mathbb{R}^n} \frac{1}{2} \|w\|^2 + C \mathbb{E} \left[\log \left(1 + e^{-Y w^\top \phi(X)} \right) \right],$$

The ERM problems of SVM and logistic regression hence read as:

$$(13.5) \quad \min_{w \in \mathbb{R}^n} \frac{1}{2} \|w\|^2 + C \frac{1}{m} \sum_{i=1}^m \left[\max \left(0, 1 - y_i w^\top \phi(x_i) \right) \right],$$

$$(13.6) \quad \min_{w \in \mathbb{R}^n} \frac{1}{2} \|w\|^2 + C \frac{1}{m} \sum_{i=1}^m \left[\log \left(1 + e^{-y_i w^\top \phi(x_i)} \right) \right],$$

Since any $W > 0$ might lead to model error¹, one hacky-idea is to avoid using any regularizer and hope that the $\|w^*\| \not\rightarrow \infty$. This leads to the following unregularized versions of SVM and logistic regression:

$$(13.7) \quad \min_{w \in \mathbb{R}^n} \mathbb{E} \left[\max \left(0, 1 - Y w^\top \phi(X) \right) \right],$$

$$(13.8) \quad \min_{w \in \mathbb{R}^n} \mathbb{E} \left[\log \left(1 + e^{-Y w^\top \phi(X)} \right) \right],$$

Infact, we can formally prove that $\|w^*\| \not\rightarrow \infty$ with surrogate losses whenever the underlying likelihood is such that the (restricted) Bayes optimal error with 0-1 loss is non-zero. This is because in such cases the risk, which is probability of misclassification, $P \left[Y w^\top \phi(X) < 0 \right]$, is non-zero (with any w) and if w direction is kept same while blowing it's norm (magnitude), i.e., the classifier is kept the same, then the surrogate loss value also blowsup. This is because the loss increase for misclassified cases blowsup unbounded, whereas the loss decrease for correctly classified cases has a bounded decrease (max decrease of 1). Note that the classification itself don't change as the classifier is the same. So in such cases, solely from a model error perspective, it is completely safe to employ unregularized SVM/logistic regression (13.7),(13.8).

¹What if $\|w^*\| > W$?

However, if the restricted optimal Bayes error is zero with 0-1 loss, then $P[Yw^\top \phi(X) < 0] = 0$ for the restricted Bayes optimal w . In other words, with probability 1, $Yw^\top \phi(X) > 0$. It is clear that if now w direction is kept same and the magnitude is blownup, then, in case of logistic regression, the loss value keeps decreasing and infact loss tends to zero only as $\|w\| \rightarrow \infty$. Hence norm regularization is essential even in terms of model error.

The situation in case of hinge loss is a bit better. As magnitude of w increase loss decreases, but beyond a certain stage, once $P[Yw^\top \phi(X) \geq 1] = 1$, any further increase in $\|w\|$ does not improve the hinge loss as it is already zero. However, still, any $\alpha w, \alpha \geq 1$ provides a risk minimizer. So one has to be careful to not employ the minimizer where $\alpha \rightarrow \infty$. Note that the risk minimizer with $\alpha = 1$ is the risk minimizer with the minimum norm! We will come back to such min. norm. solutions at a later stage (happens to be a life saving criterion in overfitting regime Bartlett et al. [2020]).

In summary, unregularized logistic regression and SVM are fine from the perspective of model error whenever restricted Bayes optimal risk with 0-1 loss is non-zero i.e., in most real-world applications. In the other cases, regularization is definitely needed for logistic regression, and some caution is needed in case of SVM (like employing near min-norm ERM solutions, which is again equivalent to regularization).

The corresponding ERM problems for unregularized SVM and logistic regression are:

$$(13.9) \quad \min_{w \in \mathbb{R}^n} \frac{1}{m} \sum_{i=1}^m \left[\max(0, 1 - y_i w^\top \phi(x_i)) \right],$$

$$(13.10) \quad \min_{w \in \mathbb{R}^n} \frac{1}{m} \sum_{i=1}^m \left[\log(1 + e^{-y_i w^\top \phi(x_i)}) \right],$$

Lecture 14

Linear Classification: Estimation Error

An important concept from the previous lecture is the regularizer, $\Omega(w)$, which, as discussed, is a way to enforce additional constraints on parameters in a linear model. The constraints come from domain/prior knowledge. For e.g., in binary classification, bounded functions need to be modelled and hence $\Omega(w) = \|w\|$ seemed appropriate. In case of Ohm's Law kind of problems, where the parameters are non-negative, one might want to use the regularizer: $\Omega(w) \equiv \max_i -w_i$ and hyperparameter $W = 0$ etc. This would enforce $w \geq 0$. For vector-valued regression (problem from tutorial), one is typically interested in latent factor models: $W_{n \times d} = U_{n \times r} V_{r \times d}$. For example, if images are outputs and captions are inputs, the outputs (images) are believed to be generated from concise abstract concepts ($r \ll n, d$), which are verbally expanded as inputs (captions). This is same as the regularizer $\Omega(W) = \text{rank}(W)$ and hyperparameter as r .

Regularizers, infact, can be independently motivated purely from the perspective of estimation/learning. For linear models, whether it is classification or regression or any problem with (bounded) loss, it turns out that the norm-regularizer is very *efficient*. We will begin this discussion with informal intuition. From Cauchy Schwartz again, now for a fixed input x , it is clear that the range of outputs across the functions in the model is controlled by $\|w\|$. Intuitively¹, it is clear that the uniform deviation between the functions induced by any valid parameter and that induced by w^* (Bayes optimal parameter within restricted to W) decreases with W . So, in terms of estimation error, lower W is preferable. We then wished to present a bit more formal and geometrically appealing argument for this, in case of binary classification. Before this, we first talked about the

¹Intuition in handwritten note in fig 10.1 above.

unregularized case with 0-1 loss (will it give analogous n/m dependency as in case of linear regression?).

The challenge is unlike in the case of linear regression, linear classification does not have an analytical solution (except perhaps in a few special cases). Hence, we need to take a more general route. Towards this end, we note the following important result from statistics that holds for any finite collection of functions, $\mathcal{F}$: with high probability²,

$$(14.1) \quad R[\hat{w}^m] - R[w^*] \leq O \left(\sqrt{\frac{\log(|\mathcal{F}|)}{m}} \right).$$

The key idea is to upper bound the estimation error given any m random inputs from the underlying distribution (need not those from the training set). Then the overall estimation error is simply the (true) mean of this bound wrt $p(x)$. Now, given any set of m inputs, the set of all functions $\text{sign}(w^\top \phi(\cdot))$ can be categorized into finitely many equivalence classes. Clearly, the number of equivalence classes is $\leq 2^m$ (all possible labelings). Infact, we realized this is tight only if $m \leq n + 1$. Else, an upper bound on this number is obtained by choosing $n + 1$ from m in $\binom{m}{n+1}$ ways and then assigning all possible labelings to these $n + 1$. This leads to the upper bound: $\binom{m}{n+1} 2^{n+1}$. Substituting this in above bound leads to:

$$(14.2) \quad R[\hat{w}^m] - R[w^*] \leq O \left(\sqrt{\frac{n}{m}} \right),$$

with high probability. So if one does unregularized linear classification, one is doomed with the exactly same bias-variance tradeoff as in case of linear regression! Ofcourse, we know it from (11.1), but this gives a closer look. The idea of counting equivalence classes is described more formally in section 6.5.1 in Shalev-Shwartz and Ben-David [2014]. The same idea helps us to get an intuition of how norm-regularization controls estimation error. This is discussed in the next lecture.

²Formally states as corollary 4.6 in Shalev-Shwartz and Ben-David [2014].

Lecture 15

Norm-Regularized Linear Models: Estimation Error

In the previous lecture we gave an intuition that norm-regularization controls the variability in the function values and hence controls the estimation error, we wished to have a slightly more deeper understanding of this phenomenon. So we took the example SVM classification (hingeloss + norm-regularized linear models) and continued the argument of counting the number of equivalence classes. Recall that here a modelling assumption is that the data is bounded i.e., $\max_{x \in \mathcal{X}} \|\phi(x)\| = R < \infty$.

Hinge-loss is zero iff there exists a parameter, w , $Yw^\top \phi(X) \geq 1$. That means positively labeled points must lie above the hyperplane $w^\top \phi(x) = 1$ and negatively labeled points must lie below the hyperplane $w^\top \phi(x) = -1$. But the distance between these two hyperplanes is $\frac{2}{\|w\|} \geq \frac{2}{W}$. i.e., As W decreases, zero hinge-loss classification can be achieved only by greater gap/margin between the hyperplanes. So now counting the equivalence classes can be done by looking at how many labellings can be given to m (random) inputs while respecting this margin.

It was easy to see that, regardless of how high is the dimension (number of parameters), if $0 < W \leq \frac{1}{R}$, then we are insisting on a margin of atleast, $> 2R$. So at the maximum one point can be given all possible labelings. So only two equivalence classes exist, i.e., the effective dimension is 0. If $W = \frac{1}{R}$, the margin is atleast $= 2R$. So by considering two diametrically oppositely located inputs on the sphere of radius R , we can still give all labellings to two points. i.e., effective dimension is 1. Same is the situation for $\frac{1}{R} \leq W < \frac{4}{3R}$. But then $W = \frac{4}{3R}$, then we can consider three points on an equilateral triangle (2-d simplex) and still give all possible labellings as the altitude of this triangle is $\frac{3R}{2}$ i.e., effective dimension

is 2. Same is the case if $\frac{4}{3R} \leq W < \frac{3}{2R}$.. and so on¹. In summary, W controls the margin and in turn controls the effective dimensionality and hence controls the estimation error.

Analogous is the case in linear regression. Let the SVD (singular value decomposition) of the data matrix, $[x_1 x_2 \dots x_m]_{n \times m}$, be $U \Sigma V^\top$. Then, the normal equation ((8.8)) can be written as $U \Sigma \Sigma^\top U^\top \hat{w} = U \Sigma V^\top y \iff \Sigma \Sigma^\top \bar{w} = \Sigma \bar{y}$, after appropriate change of bases (rotations), $\bar{w} = U^\top \hat{w}$ and $\bar{y} = V^\top y$.

Now, suppose $p(x)$ is such that data spans an $r \ll n$ dimensional subspace, then maximum rank of Σ is r . Without loss of generality, if we assume first r singular values are non-zero and the rest are zero, then, the normal equation reduces to: $\bar{w}_i = \frac{\bar{y}_i}{\sigma_i}, i = 1, \dots, r$ and the rest of the parameters $\bar{w}_j, j = r+1, \dots, n$ are free parameters that can take any value. In other words, there are many ERM solutions, each of which have exactly same first r entries (in the rotated space).

The crucial observation is that among all these ERM solutions, the solution $\bar{w}_{min} \equiv \begin{bmatrix} \frac{\bar{y}_1}{\sigma_1} \\ \vdots \\ \frac{\bar{y}_r}{\sigma_r} \\ 0 \\ \vdots \\ 0 \end{bmatrix}$ has the minimum norm. Since rotation does not change

norm, $U \bar{w}_{min}$ is nothing but the min-norm least squares solution. Thus norm-regularization in regression controls the inherent dimensionality (rank) and hence the estimation error even in linear regression. Motivated by this, we define regularized linear regression, which is popularly known as [Ridge regression](#)²:

$$(15.1) \quad \min_{w \in \mathbb{R}^n} \frac{1}{2} \|w\|^2 + C \mathbb{E} \left[\left(Y - w^\top \phi(X) \right)^2 \right],$$

The corresponding ERM problem read as:

$$(15.2) \quad \min_{w \in \mathbb{R}^n} \frac{1}{2} \|w\|^2 + C \frac{1}{m} \sum_{i=1}^m \left[\left(y_i - w^\top \phi(x_i) \right)^2 \right],$$

In general, risk minimization with regularized linear models and general loss reads as:

$$(15.3) \quad \min_{w \in \mathbb{R}^n} \frac{1}{2} \|w\|^2 + C \mathbb{E} \left[l \left(Y, w^\top \phi(X) \right) \right],$$

¹Each simplex of various dimensions, gives a critical point.

²You can show that $C \rightarrow \infty$ leads to the min-norm least squares solution above. Interested students may revise Moore-Penrose inverse and its relation with the discussion.

This is called as [Regularized Risk Minimization](#). The corresponding ERM problem read as:

$$(15.4) \quad \min_{w \in \mathbb{R}^n} \frac{1}{2} \|w\|^2 + C \frac{1}{m} \sum_{i=1}^m \left[l(y_i, w^\top \phi(x_i)) \right],$$

Finally, without proof (proposition 4.5 in Bach [2024] or corollary 13.9 in Shalev-Shwartz and Ben-David [2014]), we note the following result on how norm-regularization, W , controls the estimation error, for general (Lipschitz) losses:

$$(15.5) \quad R[\hat{w}^m] - R[w^*] \leq O\left(\frac{WR}{\sqrt{m}}\right).$$

The amazing aspect of this bound is that it is completely dimension free. We will later on study kernel methods, which put this to the best use.

Lecture 16

ERM Solvers: Gradient Descent

To find an ERM solution one needs to solve an optimization problem, whose general form is (15.4). Such problems, which are of the form: $\min_w h(w)$ usually do not have straightforward analytical solutions. One needs to solve them using numerical procedures. An appropriate class of numerical methods is that that use gradient information iteratively. For modern day problem sizes, computing the (full) gradient may itself be prohibitive, so any fancier classes like Newton methods etc. are completely avoidable.

Using the intuition of steepest instantaneous direction of descent, we motivated the popular update rule of gradient descent. This motivation highlights the greedy nature of gradient descent: it takes best possible instantaneous choice. More formally, at the k^{th} iteration, we wish to choose the direction, u , such that after taking a (infinitesimal) step, s_k , in that direction from the iterate, $w^{(k)}$, the value of the function, $h(w^{(k)} + su)$, is the minimum: $\arg \min_{\|u\|=1}$. The update rule is: Using the intuition of steepest instantaneous direction of descent, we motivated the popular update rule of gradient descent. This motivation highlights the greedy nature of gradient descent: it takes best possible instantaneous choice. More formally, at the k^{th} iteration, we wish to choose the direction, u , such that after taking a (infinitesimal) step, s_k , in that direction from the iterate, $w^{(k)}$, the value of the function, $h(w^{(k)} + s_k u)$, is the minimum: $\arg \min_{\|u\|=1} h(w^{(k)} + s_k u) \approx \arg \min_{\|u\|=1} h(w^{(k)}) + \nabla h(w^{(k)})^\top (w^{(k)} + s_k u - w^{(k)}) = \arg \min_{\|u\|=1} \nabla h(w^{(k)})^\top u = -\nabla h(w^{(k)})$. Hence, the update rule is:

$$(16.1) \quad w_{k+1} \equiv w_k - s_k \nabla h(w_k),$$

h may be is the objective of (15.4). We gave some intuition that if repeated for enough iterations, with appropriate step size, we may reach minimizers of h .

Since step size dictates the rate of parameter update, it is popularly known as the [learning rate](#).

We then derived the specific update rules for ERM problems corresponding to (regularized/unregularized) linear regression, SVM classification, logistic regression, etc., and also for a general loss case, (15.4):

$$(16.2) \quad w^{(k+1)} = w^{(k)} - s_k \left(w^{(k)} + C \frac{1}{m} \sum_{i=1}^m \left[l' \left(y_i, (w^{(k)})^\top \phi(x_i) \right) \phi(x_i) \right] \right),$$

where l' is the derivative of l wrt to the second argument. Now, after running this for enough iterations; say k , the hope is that the ERM solution has been reached upto to some approximation. This gap is known as optimization error: $R[w^{(k)}] - R[\hat{w}]$. So the total error is generalization error plus this optimization error. Typical optimization analysis provides bounds for $\hat{R}[w^{(k)}] - \hat{R}[\hat{w}]$.

Lecture 17

A Closer Look at Gradient Descent

We wished to study gradient descent in some more detail to know when/how/why it works. We began with an alternate way to understand gradient descent.

The challenging problem of minimizing h can be tackled by locally linearly approximating h , using Taylor expansion and then moving to a minimizer of this linear approximation. This process can be repeated with a hope of solving the original problem. Interestingly, this leads to the very same gradient descent algorithm. More formally, the linear approximation of h at an iterate $w^{(k)}$ is given by the first two Taylor series terms: $a_k(w) \equiv h(w^{(k)}) + \langle \nabla h(w^{(k)}), w - w^{(k)} \rangle$. The idea is to minimize a_k , while regularizing the movement away from $w^{(k)}$: $w^{(k+1)} \equiv \arg \min_{\|w - w^{(k)}\| \leq \delta_k} a_k(x)$, which, by Lagrange duality, is equivalent to:

$$(17.1) \quad w^{(k+1)} \equiv \arg \min_{w \in \mathbb{R}^n} a_k(w) + \frac{1}{2s_k} \|w - w^{(k)}\|_2^2.$$

$$(17.2) \quad = w^{(k)} - s_k \nabla f(w^{(k)})$$

Thus, gradient descent can also be interpreted as successive/iterative local linear approximation algorithm.

The criticality of step-size is clear from this interpretation too: sometimes the linear approximation may be locally a lower bound to h , so minimizing the lower bound may not lead to a decrease of h . If one wishes to guarantee a decrease of h at every iteration¹, then one may have to get an upper bound which is then

¹We are not claiming that is the “best” thing to desire. Algorithms which are long-sighted and forgo a little momentarily increase may actually work the best. But for giving a simplest scenario where convergence can be directly seen, this desire may be appropriate.

minimized instead. In such a case h also is decreased. Such algorithms are known as majorization minimization algorithms. At every iteration a majorant (upper bound) is constructed and it is minimized instead.

Using truncated Taylor's expansion a majorant was immediate: $h(w) \leq h(w^{(k)}) + \langle \nabla h(w^{(k)}), w - w^{(k)} \rangle + \frac{L}{2} \|w - w^{(k)}\|^2$, whenever the maximum eigen value of the Hessian is bounded by L i.e., $\max_{w \in \mathbb{R}^n} x^\top \nabla^2 h(w) x \leq L \|x\|^2$. Infact, this condition can be alternatively expressed² as: $\|\nabla h(w) - \nabla h(u)\| \leq L \|w - u\|$. In other words, ∇h is L -lipschitz continuous. Such functions are called as **Smooth** functions.

It is now clear that miminizing this (convex quadratic) majorant is the same as performing gradient descent with $s_k = \frac{1}{L}$. So gradient descent for smooth functions is infact a majorization minimization algorithm and hence convergence is guaranteed:

$$\begin{aligned} h(w^{(k+1)}) - h(w^{(k)}) &\leq \nabla h(w^{(k)})^\top (w^{(k+1)} - w^{(k)}) + \frac{L}{2} \|w^{(k+1)} - w^{(k)}\|^2 && (\because \text{smoothness}) \\ &= -\frac{1}{2L} \|\nabla h(w^{(k)})\|^2 && (\because (17.2) \text{ with } \frac{1}{L} \text{ step-size}) \end{aligned}$$

Since at every iteration there is strict decrease (as long as gradient is not zero), and the function is lower bounded, gradient descent either converges to the global minimum or to a stationary point. If additionally, the function is convex, this means convergence to a global minimum.

Infact, by summing the above inequality, one gets: $\sum_{i=1}^k \|\nabla h(w^{(i)})\|^2 \leq 2L (h(w^{(1)}) - h(w^{(k+1)}))$, by simple telescoping cancellation. Since $h(\hat{w}) \equiv \min_{w \in \mathbb{R}^n} h(w) \leq h(w^{(k+1)})$, we have: $\min_{i=1, \dots, k} \|\nabla h(w^{(i)})\|^2 \leq \frac{2L\Delta}{k}$, where $\Delta \equiv h(w^{(1)}) - h(\hat{w})$. Hence, the best iterate (atleast one of the iterates) converges to a stationary point at the rate $O(1/k)$.

²Formalized in lemma 3.4 in <http://sbubeck.com/Bubeck15.pdf>.

Lecture 18

SGD: An alternative to ERM

If further assumptions are made regarding h , beyond smoothness, then better convergence properties for gradient descent are observed. For example, if the function is additionally convex, then¹ we have: $h(w^{(k)}) - h(\hat{w}) \leq O\left(\frac{1}{k}\right)$. And, if the function is also strongly convex, then² $h(w^{(k)}) - h(\hat{w}) \leq O\left(\left(1 - \frac{1}{\kappa}\right)^k\right)$, where κ is condition number. There is another interesting property of gradient descent that is exploited in multiple ways in ML and infact, leads to a new alternate to ERM! The property is regarding the robustness of gradient descent to errors/noise in the gradient.

More specifically, instead of the gradient, a proxy, which is correct merely on an average, is employed, then too convergence is observed (on an average). Infact, the same proof as in previous lecture directly holds with expectations written appropriately (detailed proof in next lecture). This is popularly known as stochastic gradient descent. Here, at step k , instead of $\nabla h(w^{(k)})$, a random direction g_k is employed such that $\mathbb{E}[g_k | w^{(k)}] = \nabla h(w^{(k)})$. So, the update equation reads as:

$$(18.1) \quad w^{(k+1)} \equiv w^{(k)} - s_k g_k.$$

SGD is typically used in ML in two ways: (i) directly solve the risk minimization problem (ii) solve the ERM problem quickly. For the first way, the popular choice for g_k is $l' \left(y_k, (w^{(k)})^\top \phi(x_k) \right) \phi(x_k)$ i.e., gradient of loss only over the k^{th} sample:

$$(18.2) \quad w^{(k+1)} \equiv w^{(k)} - s_k l' \left(y_k, (w^{(k)})^\top \phi(x_k) \right) \phi(x_k).$$

¹Proposition 5.5 in Bach [2024].

²Proposition 5.3 in Bach [2024].

This is fine because the unbiasedness condition holds:

$$\mathbb{E}[g_k | w^{(k)}] = \mathbb{E}\left[l' \left(Y_k, \left(w^{(k)} \right)^\top \phi(X_k) \right) \phi(X_k) \middle| w^{(k)} \right] = \mathbb{E}\left[l' \left(Y_k, \left(w^{(k)} \right)^\top \phi(X_k) \right) \phi(X_k) \right],$$

assuming the training samples are mutually independent. Now, exchanging the gradient and the expectation, we get: $\nabla \mathbb{E}[l(Y_k, w^\top \phi(X_k))] \big|_{w=w^{(k)}}$, which is nothing but the gradient of the (true) risk as all training pairs are identically distributed as per the underlying likelihood³. Observe that the algorithm runs only for m iterations as the number of samples is m . The advantage is that we are directly solving the risk minimization problem! This may be surprising because there is no use of law of large numbers anywhere! So, how is the variance of g_k handled? It is magically done using proper step sizes as will be shown in a proof in the next lecture. Also, the computational effort is $O(n)$ per iteration and hence overall $O(mn)$.

SGD can also be used to solve the ERM problem. This is because the ERM objective $\hat{R}$ is nothing but risk (expected loss) under the sample/empirical distribution (uniform over the samples). Hence, to run the SGD for ERM, we simply need to independently uniformly sample the training pairs i.e., sampling uniformly at random with replacement. This would then simulate the scenario where we are sampling from the training set rather than from the underlying distribution. Hence, we would solve the ERM problem rather than the risk minimization problem. So g_k is gradient of the loss computed over a random training sample:

$$(18.3) \quad w^{(k+1)} \equiv w^{(k)} - s_k l' \left(y_{i_k}, \left(w^{(k)} \right)^\top \phi(x_{i_k}) \right) \phi(x_{i_k}),$$

where i_k is the index of the random training sample picked in the k^{th} iteration. Unlike SGD for true risk minimization, this SGD for ERM minimization can run for arbitrary number of iterations. Because we can keep repeating sampling with replacement from the training dataset. The advantage over vanilla gradient descent is that while each iteration of gradient descent costs $O(mn)$, it is $O(n)$ with this SGD.

³See handwritten note below in fig 18.1 for more details.

UNBIASEDNESS CONDITION SGD

$$g_k \equiv \nabla l(y_k, \omega^{(k)T} \phi(x_k))$$

$$E[g_k | \omega^{(k)}] = E[\nabla l(y_k, \omega^{(k)T} \phi(x_k)) | \omega^{(k)}] = E[\nabla l(y_k, \omega^{(k)T} \phi(x_k)) | \omega^{(k)}]_{\omega = \omega^{(k)}}$$

Now, we assume that training samples are mutually independent i.e. $(x_1, y_1), \dots, (x_m, y_m)$ are mutually independent.

$\Rightarrow (x_k, y_k)$ is independent of $\underbrace{(x_1, y_1), \dots, (x_{k-1}, y_{k-1})}_{\text{jointly.}} \forall k$

$\Rightarrow$ For any functions α, β ,
 $\alpha(x_k, y_k)$ is independent of $\beta(x_1, y_1, \dots, x_{k-1}, y_{k-1}) \forall k$

$$\Rightarrow E[\nabla l(y_k, \omega^{(k)T} \phi(x_k)) | \omega^{(k)}] = E[\nabla l(y_k, \omega^{(k)T} \phi(x_k)) | \omega^{(k)}]_{\omega = \omega^{(k)}} \forall k$$

$\therefore$ choose $\alpha(x_k, y_k) \equiv \nabla l(y_k, \omega^{(k)T} \phi(x_k))$
 choose $\omega^{(k)} \beta(x_1, y_1, \dots, x_{k-1}, y_{k-1}) = \omega^{(k)}$

$$\Rightarrow E[g_k | \omega^{(k)}] = E[\nabla l(y_k, \omega^{(k)T} \phi(x_k)) | \omega^{(k)}]_{\omega = \omega^{(k)}} \forall k$$

$$= \nabla E[l(y_k, \omega^{(k)T} \phi(x_k)) | \omega^{(k)}]_{\omega = \omega^{(k)}} \forall k$$

EXCHANGE VALID
 as long as LHS, RHS
 are well-defined

$$= \nabla h(\omega^{(k)}) \forall k$$

$(\because (x_k, y_k) \sim p(x, y) \forall k)$
 identically distributed

Figure 18.1: Unbiasedness condition in SGD

Lecture 19

Convergence of SGD

In order to understand how/why SGD converges and to understand which variant of gradient descent to be employed in which scenario, we need to critically analyze the SGD algorithm. Like in case of vanilla gradient descent, there is extensive literature on this subject and hence we present here only a naive analysis, but note important results, without venturing into their derivations. We infact, repeat the same outline of analysis as presented for gradient descent in the smooth objective case:

$$h(w^{(k+1)}) - h(w^{(k)}) \leq \nabla h(w^{(k)})^\top (w^{k+1} - w^k) + \frac{L}{2} \|w^{k+1} - w^k\|^2 \quad (\because \text{smoothness})$$

$$= -s_k \nabla h(w^{(k)})^\top g_k + \frac{L s_k^2}{2} \|g_k\|^2 \quad (\because (18.1))$$

if $\|g_k\|$ is too large, then no hope anyway as RHS is always positive.

So a common assumption is $\|g_k\| \leq B$ for some $B > 0$.

$$\leq -s_k \nabla h(w^{(k)})^\top g_k + \frac{L B^2 s_k^2}{2}$$

For regularized linear models and SGD (18.2),(18.3), $\|g_k\| = \left| l' \left(y_k, w^{(k)\top} \phi(x_k) \right) \right| \|\phi(x_k)\|$. It is easy to see that this is bounded for the usual square-loss, hinge-loss, logistic

loss as long as data is bounded. Now, taking expectation on both sides:

$$\begin{aligned}
\mathbb{E} [h(w^{(k+1)})] - \mathbb{E} [h(w^{(k)})] &\leq -s_k \mathbb{E} [\nabla h(w^{(k)})^\top g_k] + \frac{LB^2 s_k^2}{2} \\
&= -s_k \mathbb{E} [\mathbb{E} [\nabla h(w^{(k)})^\top g_k | w^{(k)}]] + \frac{LB^2 s_k^2}{2} (\because \text{total expectation rule}) \\
&= -s_k \mathbb{E} [\nabla h(w^{(k)})^\top \mathbb{E} [g_k | w^{(k)}]] + \frac{LB^2 s_k^2}{2} \\
&= -s_k \mathbb{E} [\|\nabla h(w^{(k)})\|^2] + \frac{LB^2 s_k^2}{2} (\because \text{unbiasedness condition})
\end{aligned}$$

Now, telescoping sum as earlier and rearranging leads to: $\sum_{i=1}^k s_i \mathbb{E} [\|\nabla h(w^{(i)})\|^2] \leq \mathbb{E} [h(w^{(1)})] - \mathbb{E} [h(w^{(k+1)})]$, which is further $\leq \mathbb{E} [h(w^{(1)})] - \mathbb{E} [h(\hat{w})] = h(w^{(1)}) - h(\hat{w}) \equiv \Delta_1$, as $\hat{w}$ is the minimizer of h . This gives the inequality for the “best” iterate:

$$\min_{i=1, \dots, k} \mathbb{E} [\|\nabla h(w^{(i)})\|^2] \leq \frac{\Delta_1 + \frac{LB^2}{2} \sum_{i=1}^k s_i^2}{\sum_{i=1}^k s_i}.$$

Now, in general, if we consider step sizes that *diminish* such that $\sum_{i=1}^\infty s_i = \infty$ and $\sum_{i=1}^\infty s_i^2 < \infty$, then we do have asymptotically as $k \rightarrow \infty$, convergence to a stationary point (in expectation)¹. More specifically, if we wish to know rate of convergence at finite k , then let's assume we choose constant step size $s > 0$ for all the k iterations. Then the bound reads:

$$\min_{i=1, \dots, k} \mathbb{E} [\|\nabla h(w^{(i)})\|^2] \leq \frac{\Delta_1}{ks} + \frac{LB^2}{2} s.$$

This says that if we are ready to tolerate an error $\frac{LB^2}{2}s$, which could be low if s is low enough, then the convergence rate is again $O(1/k)$. For example, if we are solving the ERM, and we are ok with a near-optimal solution, then SGD converges as fast as gradient descent but with order of magnitude lesser per iteration cost. Also, a near optimal solution to ERM may just be fine since the true goal is risk minimization for which ERM is a proxy.

Infact, we can choose the “best” step-size by minimizing the upper bound, leading to $s = \sqrt{\frac{2\Delta_1}{B^2 L k}}$, in which case the bound reads as:

$$\min_{i=1, \dots, k} \mathbb{E} [\|\nabla h(w^{(i)})\|^2] \leq B \sqrt{\frac{2\Delta_1 L}{k}}.$$

¹For example, $1, \frac{1}{2}, \dots, \frac{1}{n}, \dots$ sequence as step sizes

The above discussion basically shows that SGD has a slower convergence than vanilla gradient descent. This is essentially because “big” (constant) step-sizes cannot be now taken as it is necessary to take diminishing step sizes in order to handle the “noisy” gradients. Using sophisticated variants of law of large numbers, the bounds in expectation we derived above can be converted to those that hold for any run of SGD (with high probability). Interested students may look at exercise 5.26 for details.

Like in case of gradient descent with further assumptions on the objective better bounds can be derived: for convex+smooth objectives (e.g., unregularized linear models+smooth losses like logistic loss, square loss etc.) the “average iterate” in SGD is guaranteed to converge² to global minimum at the rate $\Delta_k \leq O\left(\sqrt{\frac{1}{k}}\right)$. Additionally, if the objective is strongly convex (e.g., regularized linear models+smooth losses like logistic loss, square loss etc.), it improves to $\Delta_k \leq O\left(\frac{1}{k}\right)$. However, observe that in each case, the rate of convergence is orders of magnitude slower than vanilla gradient descent.

While the discussion seems to speak negatively about SGD in terms of optimization, it is infact a good news in terms of learning. For e.g., this bound for SGD for risk minimization directly gives $R[w^m] - R[w^*] \leq O\left(\sqrt{\frac{1}{m}}\right)$ (in the convex+smooth case). It is amazing that this is completely independent of the number of parameters, n . In comparison, gradient descent can only be applied over the ERM objective, which brings in usual estimation error bounds $O\left(\sqrt{\frac{n}{m}}\right)$.

²See proposition 5.7 in Bach [2024].

Lecture 20

SGD vs SGD+ERM vs GD+ERM

Though in terms of optimization SGD may appear *slow*, in the context of learning, it still wins hands-on over gradient descent in multiple regimes.

Consider the case of smooth+convex objectives e.g., unregularized linear models with smooth losses. The error of SGD for risk minimization is: $R[w^{(m)}] - R[w^*] \leq O\left(\sqrt{\frac{1}{m}}\right)$. The computational cost is $O(n)$ per iteration and hence overall $O(mn)$. On the other hand, GD convergence gives $\hat{R}[w^{(k)}] - \hat{R}[\hat{w}] \leq O\left(\frac{1}{k}\right)$. $R[w^{(k)}] - R[\hat{w}] = \underbrace{R[w^{(k)}] - \hat{R}[w^{(k)}]}_{\text{risk deviation} \leq O\left(\sqrt{\frac{n}{m}}\right)} + \underbrace{\hat{R}[w^{(k)}] - \hat{R}[\hat{w}]}_{O\left(\frac{1}{k}\right)} + \underbrace{\hat{R}[\hat{w}] - R[\hat{w}]}_{\text{risk deviation} \leq O\left(\sqrt{\frac{n}{m}}\right)}$. The

risk deviation terms behave like estimation error¹. Hence, $k = \sqrt{\frac{m}{n}}$ is the optimal number of iterations, after which the optimization error bound reaches $O\left(\sqrt{\frac{n}{m}}\right)$, which is same as the estimation error bound. Hence, $R[w^{(m)}] - R[w^*] = \underbrace{R[w^{(m)}] - R[\hat{w}]}_{\text{opt. err. } O\left(\sqrt{\frac{n}{m}}\right)} + \underbrace{R[\hat{w}] - R[w^*]}_{\text{Est. err. } O\left(\sqrt{\frac{n}{m}}\right)} \leq O\left(\sqrt{\frac{n}{m}}\right)$ for ERM+GD. In order to have a

fair comparison between SGD and ERM+GD, let's run SGD for $\frac{m}{n}$ iterations only. Then, its error is also: $R[w^{(m/n)}] - R[w^*] \leq O\left(\sqrt{\frac{n}{m}}\right)$. However, the computation cost reduces to $O\left(n \frac{m}{n}\right) = O(m)$. However, the computational cost of ERM + GD is $O\left(mn \sqrt{\frac{m}{n}}\right) = O(m^{1.5}n^{0.5})$. If $m \approx n$, then ERM+GD requires computation quadratic in m , whereas SGD requires that linear in m in order to reach the same generalization error. With SGD+ERM, the optimal number of iterations is $\frac{m}{n}$ and hence computational cost is $O\left(n \frac{m}{n}\right) = O(m)$ again. Hence, for unregularized linear models+smooth losses, SGD and ERM+SGD are equally good and they

¹For e.g., see last para on pg 90 in Bach [2024].

(err.)	SGD	ERM+SGD	ERM+GD
Unreg.+conv. $O\left(\sqrt{\frac{n}{m}}\right)$	$O(m)$	$O(m)$	$O(m^2)$
Unreg.+conv.+smooth $O\left(\sqrt{\frac{n}{m}}\right)$	$O(m)$	$O(m)$	$O(m^{1.5}n^{0.5})$
Reg.+conv.+smooth $O\left(\sqrt{\frac{1}{m}}\right)$	$O(n\sqrt{m})$	$O(n\sqrt{m})$	$O(mn \log(m))$

Table 20.1: Comparison of computational cost with SGD vs ERM+SGD vs ERM+GD for reaching same generalization error. Assuming $m > n$ i.e., underparametrized regime.

are an order of magnitude better than ERM+GD in terms of computation (for reaching same generalization).

For strongly convex+smooth objectives like with regularized linear models + smooth losses, the error of SGD improves to: $R[w^{(m)}] - R[w^*] \leq O\left(\frac{1}{m}\right)$. However, the situation with ERM also improves as the estimation error improves to $O\left(\frac{1}{\sqrt{m}}\right)$ (recall (15.5)) and convergence itself improves to $O\left(\left(1 - \frac{1}{\kappa}\right)^k\right)$. Hence the optimal number of GD iterations is $k = O(\log(m))$. For same generalization, the optimal number of iterations for SGD/ERM+SGD is $\sqrt{m}$. Thus to reach generalization error $O\left(\frac{1}{\sqrt{m}}\right)$, GD will require computation of $O(mn \log(m))$, whereas, SGD/SGD+ERM will require $O(n\sqrt{m})$ computations. Hence, even for regularized models, SGD/SGD+ERM is better, however the gap between them and ERM+GD is lesser than in case of unregularized models. Analogous discussion is possible for the case where loss is convex perhaps non-smooth. The final results are summarized in table 20.1.

Note that the table and arguments above are solely based on upper bounds. Hence, in real-world problems the situations may differ a bit². In practice, ERM+SGD is most popularly employed. Infact, most often, so-called mini-batch SGD is employed, which can be understood as some middle ground between ERM+SGD and ERM+GD. In mini-batch SGD, the idea is to uniformly pick few samples (called mini-batch) from the training set at each iteration. g_k is then defined as average gradient of loss over this mini-batch. Ofcourse, unbiasedness condition still holds, justifying this variant. However, this comes at an increased computational cost per iteration. Observe that mini-batch SGD with mini-batch-size 1 is vanilla SGD and mini-batch-size m is vanilla GD. In practice, the picking of mini-batches is done in an organized way:

²Interested students see <https://arxiv.org/pdf/2203.03159>. Here, they do an exact analysis (not on bounds)) on specific problems and observe a similar trend. So the table is perhaps near-accurate in real-world too.

1. Indices of samples in the training dataset are randomly permuted.
2. entire set of size m is partitioned into m/b partitions, each of size b . First b indices form first mini-batch and so on.
3. SGD iterations are done over each mini-batch.
4. After a pass over entire training set, called as an epoch, the above steps are repeated (for a few epochs).

Infact, there are variants of SGD like SAGA Defazio et al. [2014], which improve the quality of g_k without increasing per iteration computation using sophisticated statistical techniques. For (strongly) convex objectives, ERM+SAGA works best in practice too.

Lecture 21

Probabilistic Modeling: Likelihood Estimation

We now wish to study the indirect approach from lecture 6 for solving the supervised learning problem. More specifically, the idea is to model the underlying likelihood and learn it. From this learnt likelihood, the corresponding Bayes optimal can be inferred. Assuming, the learnt likelihood is a close estimate of the underlying, the computed Bayes optimal will also be close to the true one. This strategy is known as (explicit) generative modelling. Observe that here the likelihood function is modelled and not the Bayes optimal function.

More formally,

Definition 21.0.1. Given m samples $z_1, \dots, z_m$ from an unknown likelihood $p^*(z)$, the goal of *likelihood estimation* is to find p^* i.e., solve: $\arg \min_p D(p, p^*)$, where D is a given distance/divergence over likelihoods.

Given a supervised learning problem, if $z_i \equiv (x_i, y_i)$, $p^*(z) \equiv p(x, y)$, then this definition specializes to the generative modelling approach for supervised learning. The problem 21.0.1 itself is a fundamental one with applications beyond supervised learning¹.

Firstly, observe that in this problem, never in the training set i.e., $z_1, \dots, z_m$, the values of the (likelihood) function to be learnt, are seen (even noisy versions). So this is not a problem of function induction, and definitely not a supervised learning problem. Such problems where the outputs are completely missing in the training dataset are called as *Unsupervised learning* problems. In summary, the

¹For e.g., clustering, novelty/anomaly detection, (implicit/modern) generative AI applications can be solved if this fundamental problem is solved.

problem in definition 21.0.1 can be further qualified as unsupervised likelihood estimation.

One can use various divergences, D . A divergence is any function that satisfies: $D(p, q) \geq 0 \forall p, q$ (non-negativity) and $D(p, q) = 0 \iff p = q$ (positive-definiteness). The simplest and popular option is the **KL-divergence**: $KL(p, q) \equiv \int_{\mathcal{Z}} \log \left(\frac{p(z)}{q(z)} \right) dp(z)$. For discrete cases this reads as: $KL(p, q) \equiv \sum_{z \in \mathcal{Z}} \log \left(\frac{p(z)}{q(z)} \right) p(z)$ and for continuous cases: $KL(p, q) \equiv \int_{\mathcal{Z}} \log \left(\frac{p(z)}{q(z)} \right) p(z) dz$. If $\exists z \ni p(z) \neq 0, q(z) = 0$, then KL evaluates to ∞ . If $\exists z \ni q(z) \neq 0, p(z) = 0$, then $0 \log(0) \equiv 0$. So KL divergence gives meaningful comparisons between likelihoods iff their supports² are the same. Observe that $\|\cdot\|_{\mathcal{H}}$ introduced in chapter 9, i.e., $D(p, q) \equiv \|X - Y\|_{\mathcal{H}}, X \sim p, Y \sim q$, cannot be used here as it requires the joint of X, Y to be defined, which is not defined in the context of the likelihood estimation problem.

With KL, the likelihood estimation problem is simplified as follows:

$$\begin{aligned}
 \arg \min_p KL(p^* || p) &= \arg \min_p \int_{\mathcal{Z}} \log \left(\frac{p^*(z)}{p(z)} \right) dp^*(z) \\
 &= \arg \min_p \mathbb{E}_{Z \sim p^*(z)} \left[\log \left(\frac{p^*(Z)}{p(Z)} \right) \right] \\
 &= \arg \min_p \underbrace{\mathbb{E}_{Z \sim p^*(z)} [\log (p^*(Z))]}_{\text{negative entropy of } p^*, \text{ which is a constant}} \underbrace{- \mathbb{E}_{Z \sim p^*(z)} [\log (p(Z))]}_{\text{cross-entropy}} \\
 (21.1) \quad &= \arg \min_p \mathbb{E}_{Z \sim p^*(z)} \left[\underbrace{-\log (p(Z))}_{\text{NLL}} \right]
 \end{aligned}$$

This problem is exactly same as risk minimization (in supervised learning), with an “unsupervised loss”, **negative log-likelihood (NLL)**: $l(p(x)) \equiv -\log(p(x))$. Observe the striking similarity with (4.4). The risk in this case is also known as **cross-entropy** i.e., expected NLL is known as cross-entropy. Now, once again we have a stochastic optimization problem: solve $\arg \min_p \mathbb{E}_{Z \sim p^*(z)} [l(p(Z))]$ given m samples from p^* . Hence all the discussion from lecture 5-20 will analogously hold for this unsupervised learning problem, under this NLL loss. In summary, with KL as the divergence, the unsupervised learning problem of likelihood estimation is exactly same as the problem of supervised learning with an “unsupervised” loss. Please read [section 4.2.2 in Murphy \[2022\]](#) for more details.

²Support of a likelihood is the region where likelihood is positive.

Lecture 22

Maximum Likelihood Estimation MLE

We know that the risk minimization problem (21.1) can be solved in two ways: ERM (then SGD/GD) or directly SGD. SGD on (21.1) requires a definition of gradient in space of likelihoods etc. This line of work is beyond the scope of this course. Interested students may read-up on Wasserstein Gradient flows, usually covered in a deep generative models course¹.

The other alternative, as we know from supervised learning experience, will not work. This is because the ERM problem reads as:

$$\begin{aligned} & \arg \min_p \frac{1}{m} \sum_{i=1}^m [-\log(p(z_i))] \\ &= \arg \max_p \sum_{i=1}^m \log(p(z_i)) \\ &= \arg \max_p \log \left(\prod_{i=1}^m p(z_i) \right) \\ &= \arg \max_p \left(\prod_{i=1}^m p(z_i) \right) \\ &= \arg \max_p \underbrace{p(z_1, \dots, z_m)}_{\equiv p(\mathcal{D})} \quad \because \text{samples as independent} \\ &= \arg \max_p p(\mathcal{D}) \end{aligned}$$

Due to this simplification, the ERM in likelihood estimation problem is known as **Maximum Likelihood Estimation (MLE)** i.e., finding a likelihood function, with

¹Lectures 10-12 in <https://www.overleaf.com/read/kgcgfwvppxcs#32b3f3>

which the likelihood of the training set is the highest.

It is easy to see that the [empirical likelihood](#) $\hat{p}(z) \equiv \frac{1}{m}$, if $z = z_i$ is the solution to the MLE/ERM problem. Indeed, if a portion of likelihood at any z_i is transferred to some other point, then it will decrease the likelihood of the training data. AM-GM inequality will finally show that likelihoods at all training samples must be equal to achieve the maximum. Such a likelihood solution is useless in practice (even though it may converge asymptotically to the true likelihood as m increases due to law of large numbers). This is because any z not in training set will be assigned zero likelihood. So this is simple memorization of training data.

We hence take the same route as in case of supervised learning. We restrict the possible likelihoods so that the ERM/MLE solution may become meaningful. This brings us to the MLE over a model for likelihoods, parametrized by $w \in \mathcal{W}$:

$$(22.1) \quad \arg \max_{w \in \mathcal{W}} p_w(\mathcal{D}).$$

We took two example models to make us familiar with the MLE problem. Consider a Bernoulli model, which is the set of pmfs with Bernoulli distribution parametrized by $\theta \in (0, 1)$, the probability of heads. Consider a training dataset with m_+ heads and m_- tails. Then, MLE reads as:

$$(22.2) \quad \arg \max_{\theta \in (0,1)} \theta^{m_+} (1 - \theta)^{m_-} = \arg \max_{\theta \in (0,1)} (\theta^{m_+} (1 - \theta)^{m_-})^{\frac{1}{m}} = \frac{m_+}{m}.$$

The closed form solution $\frac{m_+}{m}$ is obtained by simply finding a critical point to this objective, which turns out to be a minimizer as the GM function is concave and the feasibility set is open. Please read [section 4.2.3 in Murphy \[2022\]](#) for MLE with Bernoulli model.

The other example is univariate Gaussian model: set of all pdfs with Gaussian distribution parametrized by $\mu \in \mathbb{R}$ and $\sigma^2 > 0$. The MLE problem reads as:

$$\max_{\mu \in \mathbb{R}, \sigma^2 > 0} \frac{1}{m} \sum_{i=1}^m \log \left(\frac{e^{-\frac{(z_i - \mu)^2}{2\sigma^2}}}{\sqrt{2\pi\sigma^2}} \right)$$

This can be solved by finding critical points directly or by solving successive maximizations wrt. μ and then σ^2 . The problem wrt. μ reads as: $\arg \min_{\mu} \sum_{i=1}^m (z_i - \mu)^2$, which is minimized at $\hat{\mu} = \frac{1}{m} \sum_{i=1}^m z_i$. i.e., the MLE estimate of mean of Gaussians is sample mean. The problem wrt. σ^2 reads as:

$$(22.3) \quad \arg \min_{\sigma^2 > 0} \frac{\frac{1}{2m} \sum_{i=1}^m (z_i - \hat{\mu})^2}{\sigma^2} + \frac{1}{2} \log(\sigma^2)$$

$= \arg \min_{\frac{1}{\sigma^2} > 0} \frac{1}{2m} \sum_{i=1}^m (z_i - \hat{\mu})^2 \frac{1}{\sigma^2} - \frac{1}{2} \log(\frac{1}{\sigma^2})$. The later problem is a convex one in the variable $\frac{1}{\sigma^2}$ over an open domain, which can be hence solved by simply finding the critical points. This gives that the MLE estimate of variance of Gaussians is the sample variance: $\frac{1}{m} \sum_{i=1}^m (z_i - \hat{\mu})^2$. Please read [section 4.2.5 in Murphy \[2022\]](#) for MLE with Bernoulli model.

These two example model motivate that ERM/MLE may give meaningful likelihoods with (restrictive) likelihood models like Gaussian etc., though it fails disastrously over an unrestricted set of all possible likelihoods. We study a general/universal modelling technique analogous to the linear models in the next lecture, where MLE is guaranteed to give meaningful likelihood estimates.

With any appropriate model, the restricted Bayes optimal problem (corresponding to (21.1)) reads as:

$$(22.4) \quad \arg \min_{w \in \mathcal{W}} \mathbb{E}_{Z \sim p^*(z)} [-\log(p_w(Z))]$$

Ofcourse this can be solved directly using SGD or via ERM i.e., solving the MLE problem (22.1) using SGD/GD etc.

Lecture 23

Family of Exponential Models

From chapter 12, while dealing with binary classification, we already have the experience of tweaking linear models to work for a problem at hand. So we asked, can we minimally tweak linear models to make them model likelihood functions? For being non-negative, we looked at the form: $e^{w^\top \phi(z)}$ and for normalization, we restricted w such that $Z(w) \equiv \int_{\mathcal{Z}} e^{w^\top \phi(z)} dz < \infty$ (so we are assuming the measure dz is known). Let $\mathcal{W}$ be the set where this normalization constant $Z(w) < \infty$. Infact, we can further generalize this modelling by considering likelihoods of the form:

(23.1)

$$p_w(z) \equiv \frac{h(z)e^{w^\top \phi(z)}}{Z(w)}, Z(w) \equiv \int_{\mathcal{Z}} h(z)e^{w^\top \phi(z)} dz, w \in \mathcal{W} \equiv \{w \mid Z(w) < \infty\}.$$

w are called as the [natural/canonical parameters](#), $\phi(z)$ are known as the [sufficient statistics](#), Z is known as the [partition function](#), and h is known as the base likelihood, often ignored in various presentations. Given a ϕ , the set of all likelihoods with various valid parameters $w \in \mathcal{W}$ form the [exponential model](#) for likelihood. There are many names for this fundamental model as it was discovered by various disciplines: [log-linear models](#), [max-ent models](#) ([maximum-entropy models](#)¹), [energy-based models](#) etc.

We then realized that the univariate Gaussian model belongs to this family of models with $\phi(z) \equiv \begin{bmatrix} z \\ z^2 \end{bmatrix}$. This is immediate by taking $w = \begin{bmatrix} \frac{\mu}{\sigma^2} \\ -\frac{1}{2\sigma^2} \end{bmatrix}$ and $h(z) = 1$. $\mathcal{W} = \mathbb{R} \times \mathbb{R}_{++}$. Infact, most of the named distributions we study in probability courses belong to this family.

The risk (cross-entropy) minimization problem (22.4) with a max-ent model

¹If you are interested to know why this name, then read section 3.4.4 in Murphy [2022].

simplifies as:

$$\begin{aligned}
\arg \min_{w \in \mathcal{W}} \mathbb{E}_{X \sim p^*(z)} [-\log(p_w(X))] &= \arg \min_{w \in \mathcal{W}} \mathbb{E}_{X \sim p^*(z)} \left[-\log \left(\frac{h(X) e^{w^\top \phi(X)}}{Z(w)} \right) \right] \\
&= \arg \min_{w \in \mathcal{W}} \mathbb{E}_{X \sim p^*(z)} \left[-\log \left(\frac{e^{w^\top \phi(X)}}{Z(w)} \right) \right] \\
(23.2) \quad &= \arg \min_{w \in \mathcal{W}} \mathbb{E}_{X \sim p^*(z)} [-w^\top \phi(X)] + \log(Z(w))
\end{aligned}$$

$$(23.3) \quad = \arg \min_{w \in \mathcal{W}} w^\top \mathbb{E}_{X \sim p^*(z)} [-\phi(X)] + \log(Z(w))$$

(23.2) shows that the simplified form of the risk minimization problem in likelihood estimation with KL divergence is infact the same as regularized risk minimization problem (15.3) with the simple loss $l(a) = -a$, regularization-hyperparameter as unity, and regularizer as $\Omega(w) = \log(Z(w))$, which is known as the **cummulant generating function (cgf)**². So analogous estimation error bounds as in chapter 15 can be derived for this problem too, justifying the correctness MLE with models in the exponential family. Ofcourse, modelling error can be controlled by consider higher number of parameters. The corresponding ERM (MLE) problem reads as:

$$(23.4) \quad \arg \min_{w \in \mathcal{W}} w^\top \left(\frac{1}{m} \sum_{i=1}^m -\phi(z_i) \right) + \log(Z(w))$$

We then computed the gradient and Hessian of the cgf: $\frac{\partial \log(Z(w))}{\partial w_i} = \frac{1}{Z(w)} \frac{\partial}{\partial w_i} Z(w) = \frac{1}{Z(w)} \frac{\partial}{\partial w_i} \int_{\mathcal{Z}} h(z) e^{w^\top \phi(z)} dz = \frac{1}{Z(w)} \int_{\mathcal{Z}} h(z) \frac{\partial}{\partial w_i} e^{w^\top \phi(z)} dz = \int_{\mathcal{Z}} \frac{h(z) e^{w^\top \phi(z)}}{Z(w)} \frac{\partial}{\partial w_i} w^\top \phi(z) dz = \int_{\mathcal{Z}} \frac{h(z) e^{w^\top \phi(z)}}{Z(w)} \phi_i(z) dz = \mathbb{E}_{X \sim p_w(z)} [\phi_i(X)]$. Thus, $\frac{\partial^2 \log(Z(w))}{\partial w_j \partial w_i} = \frac{\partial}{\partial w_j} \int_{\mathcal{Z}} \frac{h(z) e^{w^\top \phi(z)}}{Z(w)} \phi_i(z) dz =$. Hence the gradient is $\mathbb{E}_{X \sim p_w(z)} [\phi(X)]$ and Hessian is $\mathbb{E}_{X \sim p_w(z)} [\phi(X) \phi(X)^\top] \succeq 0$. This shows that the problems (23.3) and (23.4) are infact convex and hence SGD for (23.3) and ERM+SGD for (23.4) will converge³ to the respective global minimizers and their performance will be comparable according to table 20.1.

If there are no constraints i.e., $\mathcal{W} = \mathbb{R}^n$, then the SGD update rule for (23.3) reads as:

$$(23.5) \quad w^{(k+1)} \equiv w^{(k)} - s_k \left(-\phi(z_k) + \mathbb{E}_{X \sim p_{w^{(k)}}(z)} [\phi(X)] \right).$$

whereas, the ERM+SGD update rule for (23.4) reads as:

$$(23.6) \quad w^{(k+1)} \equiv w^{(k)} - s_k \left(-\phi(z_{i_k}) + \mathbb{E}_{X \sim p_{w^{(k)}}(z)} [\phi(X)] \right),$$

²The “coefficients” in McLaurin series are so-called cummulants. The first two cummulants are mean and correlation. Hence this is a convex function as correlation is psd.

³Appropriate variants of SGD need to be employed to handle the constraints $w \in \mathcal{W}$. This discussion is beyond the scope of this course.

where i_k is the random index into $1, \dots, m$ picked at k^{th} iteration. Further, just by finding the critical points, it is easy to see that the minimizer w^* for satisfies the moment (expected sufficient statistics) matching constraint:

$$\mathbb{E}_{Z \sim p_{w^*}(z)}[\phi(Z)] = \mathbb{E}_{Z \sim p^*(z)}[\phi(Z)].$$

And, the MLE solution $\hat{w}$ for satisfies the moment matching constraint:

$$\mathbb{E}_{Z \sim p_{\hat{w}}(z)}[\phi(Z)] = \frac{1}{m} \sum_{i=1}^m \phi(z_i).$$

Observe that this set of moment matching conditions in case of univariate Gaussian model were simple linear equations, whose solution can simply be read off by inspection. For general models, these equations may be non-trivial and require a numerical solution like (23.6).

Please read [section 3.4 in Murphy \[2022\]](#) for more details.

Lecture 24

Generative models for Regression and classification

The general steps for building a generative model for supervised learning are:

Modelling: Model the underlying joint likelihood. Identify the parameters etc.

Training: Estimate the parameters using MLE (or equivalent).

Inference: Compute the prediction function:

1. Solve an inference problem to deduce the posterior likelihood corresponding to the learnt likelihood (estimated parameters).
2. Solve (6.1) to find the Bayes optimal for this learnt likelihood. For regression problems, this is same as finding the mean of this posterior, and for classification, this is the mode of this posterior.

We will two specific examples to concretize this generative pipeline:

24.1 Generative Linear Regression

Consider a regression problem: $\mathcal{X} = \mathbb{R}^n, \mathcal{Y} = \mathbb{R}^d$.

Generative linear regression is the name given to the following specific ML pipeline:

Modelling: Let us model the underlying joint using an $n + d$ dimensional Gaussian. Let μ, Σ be the mean, covariance parameters.

Training: From practice problems (or section 4.2.6 in Murphy [2022]), the MLE estimate of mean, covariance are $\hat{\mu}$, the sample mean, and, $\hat{\Sigma}$, the sample covariance respectively.

Posterior: From practice problems (or equation (3.28) in Murphy [2022]), the posterior corresponding to Gaussian with $\hat{\mu}$, $\hat{\Sigma}$ is another Gaussian with mean $\hat{\Sigma}_{21}\hat{\Sigma}_{11}^{-1}(x - \hat{\mu}_1) + \hat{\mu}_2$ and covariance $\hat{\Sigma}_{22} - \hat{\Sigma}_{21}\hat{\Sigma}_{11}^{-1}\hat{\Sigma}_{12}$, where $\hat{\mu} = \begin{bmatrix} \hat{\mu}_1 \\ \hat{\mu}_2 \end{bmatrix}$ and $\hat{\Sigma} = \begin{bmatrix} \hat{\Sigma}_{11} & \hat{\Sigma}_{12} \\ \hat{\Sigma}_{21} & \hat{\Sigma}_{22} \end{bmatrix}$, $\hat{\mu}_1 \in \mathbb{R}^n$, $\hat{\Sigma}_{11} \in \mathbb{R}^{n \times n}$.

If $\hat{\mu} = 0$, this regressor is exactly same as that in linear regression (8.8), hence it is the “generative” version of linear regression¹.

Clearly, this model makes more assumptions than the vanilla linear regression². While this is their down-side, the same is their advantage: because they make more assumptions, they can be of use in applications beyond standard prediction tasks. For example, consider a medical diagnosis application where, typically, one may want to infer the disease incidence rate (output), even if the patient did not undergo all the available lab tests (i.e., only a few input features are available). Such inferences, which are beyond standard predictions, are possible because we have modelled the entire joint! e.g., $p(y|x_o) \propto \int p(y, x_o, x_h) dx_h$, where $x = (x_o, x_h)$ and only x_o are the observed features in x .

24.2 Bayes Classifier

Consider a classification problem: $\mathcal{X} = \mathbb{R}^n$, $|\mathcal{Y}| = c \geq 2$.

Bayes classifier is the name given to the predictive function corresponding to the following specific ML pipeline:

Modelling: Let us model the underlying joint, $p^*(x, y)$, by modelling it's factors: $p^*(x|y) \forall y \in \mathcal{Y}$, known as **class-conditional likelihoods**, and $p^*(y)$, known as the **label/class prior likelihood**. This factorization is convenient for classification problems because X is a continuous r.v. and Y is a discrete r.v. Let us further model the class-conditionals by n -dimensional Gaussians (c of them). And, the prior by multinoulli/categorical model. Multinoulli is nothing but a likelihood of a r.v. taking c values, where the c likelihoods are the parameters. If $c = 2$, it simplifies to the Bernoulli model. Ofcourse, since likelihoods sum to unity, there are only $c - 1$ free parameters. Let us denote the (Gaussian) model for the y^{th}

¹See problem 1 in section 7.1 in practice problems for a more general connection

²There may be other joint likelihoods whose posterior mean is an affine function.

class as $p_{\theta_y}(x|y)$ and the prior by $p_w(y)$. Let $\theta = (\theta_1, \dots, \theta_c)$.

Training: While one advantage of KL as the choice of divergence is that the likelihood estimation problem simplifies to a risk minimization (stochastic optimization) problem, the other advantage is the simplification it provides for models for conditional likelihoods. From first principles:

(24.1)

$$\begin{aligned}
& \min_{\theta, w} KL(p^*(x, y) || p_{\theta, w}(x, y)) \\
&= \min_{\theta, w} KL(p^*(x, y) || p_{\theta_y}(x|y)p_w(y)) \\
&= \min_{\theta, w} \mathbb{E}_{(X, Y) \sim p^*(x, y)} \left[\log \left(\frac{p^*(X, Y)}{p_{\theta_Y}(X|Y)p_w(Y)} \right) \right] \\
&= -\max_{\theta, w} \mathbb{E}_{(X, Y) \sim p^*(x, y)} [\log(p_{\theta_Y}(X|Y)p_w(Y))] \\
&= -\max_{\theta, w} \mathbb{E}_{(X, Y) \sim p^*(x, y)} [\log(p_{\theta_Y}(X|Y))] + \mathbb{E}_{(X, Y) \sim p^*(x, y)} [\log(p_w(Y))] \\
&= -\max_{\theta} \mathbb{E}_{(X, Y) \sim p^*(x, y)} [\log(p_{\theta_Y}(X|Y))] - \max_w \mathbb{E}_{(X, Y) \sim p^*(x, y)} [\log(p_w(Y))] \\
(24.2) \quad &= -\max_{\theta} \mathbb{E}_{(X, Y) \sim p^*(x, y)} [\log(p_{\theta_Y}(X|Y))] - \underbrace{\max_w \mathbb{E}_{Y \sim p^*(y)} [\log(p_w(Y))]}_{\text{Cross-entropy-max/MLE with Categorical model}} \\
&= -\mathbb{E}_{Y \sim p^*(y)} \left[\underbrace{\max_{\theta_Y} \mathbb{E}_{X \sim p^*(x|Y)} [\log(p_{\theta_Y}(X|Y))]}_{\text{Cross-entropy-max/MLE with Gaussian model}} \right] - \max_w \mathbb{E}_{Y \sim p^*(y)} [\log(p_w(Y))]
\end{aligned}$$

This derivation shows that the MLE for joint likelihood conveniently decomposes as c MLEs for each class-conditional likelihoods and an MLE for the label prior. Now training data for each of these MLE problems can be obtained from original training data in a straight-forward manner: for the j^{th} class-conditional, we need samples from $p^*(x|j)$, which are nothing but those inputs in the original training set where the corresponding label is j . For Gaussian class-conditionals this MLE simply leads to: μ_j, Σ_j estimated as the sample mean, sample covariance of these inputs labelled j . Likewise, the labels corresponding to all training samples are indeed samples for $p^*(y)$, which is needed for MLE for the label prior. We know the MLE estimate of w_j turns out to be the fraction of samples in the original training set labelled as j .

Posterior/Inference: Once estimates $\hat{\theta}, \hat{w}$ are obtained from the above, the corresponding posterior is obtained using Bayes rule: $p(y|x) \propto p_{\hat{\theta}_y}(x|y)p_{\hat{w}}(y) \propto$

$e^{-\frac{1}{2}(x-\hat{\mu}_y)^\top \hat{\Sigma}_y^{-1}(x-\hat{\mu}_y) - \frac{1}{2} \log(|\hat{\Sigma}_y|) + \log(\hat{w}_y)}$. The corresponding Bayes optimal is $\hat{f}(x) = \arg \max_{y \in \mathcal{Y}} -\frac{1}{2}(x-\hat{\mu}_y)^\top \hat{\Sigma}_y^{-1}(x-\hat{\mu}_y) - \frac{1}{2} \log(|\hat{\Sigma}_y|) + \log(\hat{w}_y)$. For the special case $c = 2$, this simplifies as: $\hat{f}(x) = \text{sign} \left(-\frac{1}{2}(x-\hat{\mu}_1)^\top \hat{\Sigma}_1^{-1}(x-\hat{\mu}_1) + \frac{1}{2}(x-\hat{\mu}_{-1})^\top \hat{\Sigma}_{-1}^{-1}(x-\hat{\mu}_{-1}) - \frac{1}{2} \log \left(\frac{|\hat{\Sigma}_1|}{|\hat{\Sigma}_{-1}|} \right) \right)$.

Now, in case $c = 2$, this $\hat{f}$ does indeed belong to the class of logistic regression predictive functions whenever ϕ includes quadratic monomials. However, there is no reason to believe³ that the trained parameters of logistic regression over same training data match those for $\hat{f}$. So, unlike the pair generative linear regression and vanilla linear regression, Bayes classifier and logistic regression do not seemingly form a “pair” though the forms of the final predictive functions may be the same. This is infact why we dont call this model as “generative logistic regression”.

24.2.1 Naïve Bayes Classifier

Sometimes, in applications, some more prior/domain knowledge is available: consider medical diagnosis applications where x (input) is symptoms and y is disease. A common example of conditional independence (domain knowledge) is: symptoms like headache and fever are conditionally independent given the underlying disease (like the flu). This means that once you know the person has the flu, knowing they have a headache doesn't make a fever more or less likely than it already was. However, without knowing the disease, the two symptoms are not independent because they are both more likely to occur together if the person has the flu.

Such information can be encoded as $p(x|y) = p(x^1|y) \dots p(x^n|y)$, where $x = (x^1, \dots, x^n)$. Here, x^j is the j^{th} symptom and y is disease. Ofcourse there may be other applications where such a conditional independence (CI) holds. The Bayes classifier model described above with this additional assumption is called the **Naïve Bayes Classifier**⁴. Commonly, each $p(x^j|y)$ is modelled by a univariate Gaussian. Let us denote these parameters as θ_{yj} . Due to this assumption of CI, there is further simplification in MLE: $\max_{\theta_y} \mathbb{E}_{X \sim p^*(x|y)} [\log(p_{\theta_y}(X|y))] =$

³For e.g., logistic regression would have one parameter to represent the constant coefficient in the predictive function. Whereas, in Bayes classifier this constant coefficient is $-\frac{1}{2} \log \left(\frac{|\hat{\Sigma}_1|}{|\hat{\Sigma}_{-1}|} \right) + \log \left(\frac{\hat{w}_1}{\hat{w}_{-1}} \right)$. Given this constant's value (and other coefficients) there is no way to recover the parameters of the Bayes classifier. This again shows that generative models assume more than non-probabilistic models. However, they are more widely applicable for the same reason. For e.g., a Bayes classifier model can also be used to answer the question: given an input x what is the likelihood of label y ? However, this question cannot be answered by any non-probabilistic model for supervised learning, including vanilla logistic regression.

⁴It was called Naïve because initially people applied it just for convenience/computational ease.

$\max_{\theta_y} \mathbb{E}_{X \sim p^*(x|y)} \left[\log \left(\prod_{j=1}^n p_{\theta_{y_j}}(X^j|y) \right) \right] = \max_{\theta_y} \mathbb{E}_{X \sim p^*(x|y)} \left[\sum_{j=1}^n \log \left(p_{\theta_{y_j}}(X^j|y) \right) \right] = \sum_{j=1}^n \max_{\theta_{y_j}} \mathbb{E}_{X^j \sim p^*(x^j|y)} \left[\log \left(p_{\theta_{y_j}}(X^j|y) \right) \right]$. Now, each of this is just an MLE for univariate Gaussian and the training data is simply the j^{th} input feature values of all inputs corresponding having y label in the original training dataset.

Infact, in real-world examples, it may happen that few input features are conditionally independent and a few not. Probabilistic graphical models (CS5560) are a generalization of the probabilistic models we study in this course, where it is easy to encode such domain/prior knowledge.

Lecture 25

Discriminative Modelling

Another approach to probabilistic modelling is so-called discriminative modelling. Here, instead of modelling the joint likelihood, $p^*(x, y)$, the posterior likelihood $p^*(y|x)$ is modelled. The advantage over generative models is ofcourse that the modelling assumptions are weaker. This is also a downside, as applicability decreases. For e.g., missing input features cannot now be handled. But still it is more applicable than non-probabilistic modelling: for e.g., suppose the output is partially observed (say, image with occlusion), then, this can be directly handled by the discriminative models: let y_o, y_h be the observed and un-observed portions of the output. Since $p(y|x) = p(y_o, y_h|x)$ is modelled in discriminative methods, one can obtain $p(y_h|x, y_o)$ and the corresponding Bayes optimal: $\arg \max_{y_h} p(y_h|x, y_o)$. In summary, discriminative models are a middle ground between generative models and non-probabilistic models¹.

From the experience of modelling the class-conditionals in Bayes classifier, modelling posterior likelihoods, $p(y|x)$, is straight forward:

1. First, forget the conditioning and model as if we are modelling the prior, $p(y)$.
2. A general model for it is a model from the exponential family: $p_\theta(y) = \frac{e^{\theta^\top \phi(y)}}{Z(\theta)}$, where ϕ is the sufficient statistics.
3. Next, model $\theta \in \mathbb{R}^d$ as a function of the conditioned variable, which is x for the posterior. We can do this by modelling each component, $\theta_i : \mathcal{X} \mapsto \mathbb{R}$. A general way to model scalar-valued functions is using linear modelling: $\theta_i(x) \equiv w_i^\top \psi_i(x)$, where ψ_i is the feature map.

¹In modern terminology, both generative and discriminative ones are referred to as generative models.

4. Many times, a simplifying assumption is made for the sake of convenience: ψ_i is same for all i . In such a case, denoting W as the matrix with columns as w_i , leads to $\theta(x) = W^\top \psi(x)$. . Calling W as the matrix with columns as w_i , this is same as: $\theta(x) = W^\top \psi(x)$.

5. In summary, the general model for posteriors is: $p_W(y|x) = e^{\psi(x)^\top W \phi(y)} / Z(W^\top \psi(x))$.

Now, our goal is to estimate W such that the corresponding posterior likelihood, $p_W(y|x)$, and true posterior likelihood, $p^*(y|x)$ are close, at any x . Again, if we enforce this at all x it might lead to useless estimation as an x may be extremely low-likely (similar arguments as page 14). Hence, we may formally pose it as the problem where match is in expectation:

$$(25.1) \quad \min_W \mathbb{E}_{X \sim p^*(x)} [KL(p^*(y|X) || p_W(y|X))].$$

Alternatively, consider two joints: $p^*(y|x)p^*(x)$ and $p_W(y|x)p^*(x)$. These two are exactly the same iff posteriors $p^*(y|x)$ and $p_W(y|x)$ are exactly the same. This is because the priors in both are the same. So an alternative objective is $\min_W KL(p^*(y|x)p^*(x) || p_W(y|x)p^*(x))$. But from derivation similar to (24.2), it is clear that this is the same as (25.1):

$$(25.2) \quad \begin{aligned} & \arg \min_W KL(p^*(y|x)p^*(x) || p_W(y|x)p^*(x)) \\ &= \arg \min_W \mathbb{E}_{(X,Y) \sim p^*(x,y)} \left[\log \left(\frac{p^*(Y|X)p^*(X)}{p_W(Y|X)p^*(X)} \right) \right] \\ &= \arg \min_W \underbrace{\mathbb{E}_{(X,Y) \sim p^*(x,y)} \left[\log \left(\frac{p^*(Y|X)}{p_W(Y|X)} \right) \right]}_{\text{same as (25.1) by total expectation rule}} \end{aligned}$$

$$(25.3) \quad = \arg \max_W \underbrace{\mathbb{E}_{(X,Y) \sim p^*(x,y)} [\log(p_W(Y|X))]}_{\text{expected posterior cross-entropy}}$$

Note that, at first glance, it might seem hard to solve (25.1) using SGD/ERM+SGD as the KL term would require multiple samples from $p^*(y|x)$ (for the same x). This is never available whenever X is a continuous rv. But, thanks to total expectation rule (25.2), the final stochastic optimization problem (25.3) can be solved using SGD/ERM+SGD as the samples from $p^*(x, y)$ are only needed, which are available as the training set. If we were not to accept KL as the default divergence, estimating posterior's parameters (discriminative learning) would be a nightmare, as multiple samples from $p^*(y|x)$ (for the same x) may be needed. Finally, the MLE corresponding to (25.3) reads as:

$$(25.4) \quad \arg \max_W \frac{1}{m} \sum_{i=1}^m \log(p_W(y_i|x_i)).$$

Lecture 26

Discriminative models for regression and classification

26.1 Discriminative Linear Regression

Consider regression problems where $\mathcal{X} = \mathbb{R}^n$ and $\mathcal{Y} = \mathbb{R}$.

We call [discriminative linear regression](#) as the following specific ML pipeline:

Modelling: Model posterior using a Gaussian with mean as function of x and constant variance: $p_{w, \sigma^2}(y|x) \sim \mathcal{N}(w^\top \phi(x), \sigma^2)$. This is also same as saying $Y = w^\top \phi(X) + N$, where $N \sim \mathcal{N}(0, \sigma^2)$. This is what you see in most books calling as linear regression, which is different from (9.2). This again is evidence of non-probabilistic modelling (9.2) making weaker assumptions than discriminative models (probabilistic models, in general).

Training: The MLE problem simplifies as:

$$\begin{aligned}
& \max_{w, \sigma^2} \frac{1}{m} \sum_{i=1}^m \log(p_{w, \sigma^2}(y_i | x_i)) \\
&= \max_{w, \sigma^2} \frac{1}{m} \sum_{i=1}^m \log \left(\frac{e^{-\frac{1}{2\sigma^2} (y_i - w^\top \phi(x_i))^2}}{\sqrt{2\pi\sigma^2}} \right) \\
&= \max_{w, \sigma^2} -\frac{1}{m} \sum_{i=1}^m \frac{1}{2\sigma^2} (y_i - w^\top \phi(x_i))^2 - \log(\sqrt{2\pi\sigma^2}) \\
&= \max_{\sigma^2} -\log(\sqrt{2\pi\sigma^2}) - \frac{1}{2\sigma^2 m} \underbrace{\min_w \sum_{i=1}^m (y_i - w^\top \phi(x_i))^2}_{\text{same as (8.3). Denote solution as } \hat{w}} \\
&= \underbrace{\max_{\sigma^2} -\log(\sqrt{2\pi\sigma^2}) - \frac{1}{2\sigma^2 m} \sum_{i=1}^m (y_i - \hat{w}^\top \phi(x_i))^2}_{\text{same as (22.3)}}
\end{aligned}$$

Hence, the estimate of σ^2 is simply linear regression error: $\frac{1}{m} \sum_{i=1}^m (y_i - \hat{w}^\top \phi(x_i))^2$.

Inference: The final regressor is exactly same as that with vanilla linear regression. Hence the name discriminative linear regression.

26.2 Discriminative Logistic Regression

Consider classification problems where $\mathcal{X} = \mathbb{R}^n$ and $|\mathcal{Y}| = c$.

We call **discriminative logistic regression** as the following specific ML pipeline:

Modelling: The posterior is a categorical (Multinoulli) model with natural parameters that are a function of x . Multinoulli likelihood is: $p_\theta(y) = \frac{e^{\theta_y}}{\sum_{i=1}^c e^{\theta_i}}$, when expressed as a member of exponential family. This is also popularly known as the **softmax function**. We assume θ_y is modelled¹ as $\theta_y(x) \equiv w_y^\top \phi(x)$. In summary, the posterior is modelled as: $p_W(y|x) = \frac{e^{w_y^\top \phi(x)}}{\sum_{i=1}^c e^{w_i^\top \phi(x)}}$.

¹Again, this is only for convenience/simplicity. More generally, we can model $\theta_y(x) \equiv w_y^\top \phi_y(x)$.

Learning: The MLE problem in this case reads as:

$$\begin{aligned}
& \arg \max_W \frac{1}{m} \sum_{i=1}^m \log(p_W(y_i|x_i)) \\
&= \arg \max_W \frac{1}{m} \sum_{i=1}^m \log \left(\frac{e^{w_{y_i}^\top \phi(x_i)}}{\sum_{j=1}^c e^{w_j^\top \phi(x_i)}} \right) \\
(26.1) \quad &= \arg \min_W \sum_{i=1}^m \left\{ \underbrace{\log \left(\sum_{j=1}^c e^{w_j^\top \phi(x_i)} \right)}_{\text{LSE function (convex, smooth)}} - w_{y_i}^\top \phi(x_i) \right\}
\end{aligned}$$

This can be solved using SGD.

In the special case $c = 2$ (binary classification), $p_W(1|x) = \frac{e^{w_1^\top \phi(x)}}{e^{w_1^\top \phi(x)} + e^{w_{-1}^\top \phi(x)}} = \frac{1}{1 + e^{(w_{-1} - w_1)^\top \phi(x)}}$, where $w \equiv w_1 - w_{-1}$. Also, $p_W(-1|x) = \frac{1}{1 + e^{w^\top \phi(x)}}$. Hence, $p_w(y|x) = \frac{1}{1 + e^{-y w^\top \phi(x)}}$. Hence, the MLE simplifies as:

$$\begin{aligned}
& \arg \max_w \frac{1}{m} \sum_{i=1}^m \log(p_W(y_i|x_i)) \\
&= \arg \max_w \frac{1}{m} \sum_{i=1}^m \log \left(\frac{1}{1 + e^{-y_i w^\top \phi(x_i)}} \right) \\
&= \arg \min_w \frac{1}{m} \sum_{i=1}^m \log(1 + e^{-y_i w^\top \phi(x_i)})
\end{aligned}$$

This is exactly same as (13.10) for vanilla logistic regression. Hence the name discriminative logistic regression. In modern AI terminology, this is what is referred to as logistic regression.

Inference: The corresponding Bayes optimal is:

$$(26.2) \quad \hat{f}(x) = \arg \max_{y \in \mathcal{Y}} \frac{e^{\hat{w}_y^\top \phi(x)}}{\sum_{i=1}^c e^{\hat{w}_i^\top \phi(x)}},$$

where $\hat{W}$ is the solution to the MLE problem (26.1). In binary classification case, this simplifies and we recover vanilla logistic regression predictor: $\hat{f}(x) = \text{sign}(w^\top \phi(x))$.

Lecture 27

Model Selection

Till now we studied machine learning with a fixed model/hyper-parameters. But we face the question, which is the best model/hyper-parameter? Given the discussion earlier, especially chapter 11, it is clear that one model/hyper-parameter will not work for all applications/tasks. Hence, the natural question to ask is which model/hyper-parameter is best for my problem at hand? This leads to defining the following meta-learning problem: Given samples from the underlying likelihood associated with a learning task, and few models $\mathcal{M}_1, \dots, \mathcal{M}_k$, select the model that has the “highest potential” of resulting in a highly generalizing learner.

The primary issue that needs to be understood is: we wish to evaluate the learning ability/potential of each model in the context of the samples given. This is analogous to the question, given a subject, which student has the highest potential for learning it well? If this has to be answered using the language of supervised-learning/statistics, then there must be past history of how the students learned similar subjects. For e.g., suppose we know that a model learned well in multiple learning stints, then it might learn well ever after (on any new learning stint on the same task). Such a repeated evaluation of a model can be done if multiple training data are available (all iid), for enabling learning/training. Also, multiple test/evaluation data must be available for evaluating the generalization of the learnt/trained model. Moreover, for the training to work in each learning stint, no. training samples must be high, and for estimate of generalization error to be accurate, no. test samples must be high, and the number of learning stints itself must be high to reliably predict future performance.

But, in practice, usually only one dataset is available for learning as well as for model selection. So, using clever ideas from re-sampling, this single dataset is reused to create all these pairs of training and test datasets:

1. The given dataset, $\mathcal{D}$, must be first randomly partitioned into two: $\mathcal{D}_1$ and

$\mathcal{D}_2$. $\mathcal{D}_2$ is reserved for evaluation purposes. It must never be used in anyway in any of the learning or meta-learning stages. In case evaluation is not needed, $\mathcal{D}_2$ can be empty. In the following, $\mathcal{D}_1$ is only used.

2. The indices of samples in $\mathcal{D}_1$ are randomly permuted.
3. $\mathcal{D}_1$ is partitioned into k equal-sized partitions, called [folds](#). If m is the no. samples in $\mathcal{D}_1$, then, m/k is the no. samples in each fold. k can be any number between 2 and m .
4. In the k^{th} learning stint, the samples in the k^{th} fold are earmarked for evaluation and the rest of the data in $k - 1$ folds are used for training. The model that leads to best predictive evaluation score summed over all folds (stints after training) is declared as the “best” model.
5. Observe that across stints, no. samples used for evaluation is m . If $k = m$, then no. samples used in training is $m - 1$ and the number of learning stints is itself m . Assuming m is large, this simulates the situation of large no. stints, each with large training data and test data. This special case is known as [leave-one-out](#) cross-validation.
6. If computation is the bottleneck, $k \ll m$ is employed. This is known as k -fold cross-validation. In such a case, few repeats of this cv are performed to improve estimation.

Once the best model is selected. It can be freshly trained with all the samples in $\mathcal{D}_1$. This is the final trained model. If needed, it's performance can be estimate using $\mathcal{D}_2$. Beware, once $\mathcal{D}_2$ is used/touched, further “ML” is like cheating in exams.

Please read [Section 11.2 in Shalev-Shwartz and Ben-David \[2014\]](#) and [section 7.10 in Hastie et al. \[2001\]](#) for more details.

Lecture 28

Linear Models in Infinite Dimensional Spaces

Till now we have learned about only one modelling strategy: linear modelling, where the idea is to design/choose a feature map $\phi : \mathcal{X} \mapsto \mathbb{R}^n$ and the model for the Bayes optimal is $f_w(x) = w^\top \phi(x)$, or some transformation¹ of it. There are atleast two more modelling techniques that are popular in ML: (i) kernel based models (ii) neural network models. Now, we will study kernel models, which are a generalization of linear models to infinite dimensional spaces.

Observe that given any feature map $\phi : \mathcal{X} \mapsto \mathbb{R}^n$, there will always be target functions f^* which are not of the form $w^\top \phi(x)$. More seriously, f^* may not be even closely approximated. For example, consider ϕ that includes all monomials upto degree d . Then, if f^* is a polynomial of degree $\geq d+1$, then, the model error may be arbitrarily high.

This issue can be resolved only if any target function can be arbitrarily well approximated by functions in the model. Intuitively, think about the case when $\phi(x)$ is the infinite sequence of all degree monomials, or the sequence of basis functions like Fourier basis functions. Then, it seems any target function can be well approximated by linear functions over this $\phi(\mathcal{X})$, as any function can be closely approximated by linear combinations of corresponding basis functions. Hence, model error is (arbitrarily close to) zero.

Another general way of thinking about such a universal representation like basis functions is: recall how words in a language are described in terms of other words. For e.g., consider representing x via a function, $\phi(x) : \mathcal{X} \mapsto \mathbb{R}$, which

¹In case of binary classification, transformation is $\text{sign}(w^\top \phi(x))$ with norm-bound and in case of likelihood modelling it is $e^{w^\top \phi(x)}$, with appropriate normalization.

simply specifies relatedness/similarity with all other points in the input space. i.e., $\phi(x)(x')$ is similarity of x and x' . Thus, by comparing x to every other $x' \in \mathcal{X}$, we describe x .

In summary, generalizing linear models to infinite dimensional spaces seems the key for achieving zero model error. However, one may fear that if model error is (near) zero, then estimation error may shoot up. But, regularized linear models control this via the regularizer. So this may not be an issue.

Another way of understanding this generalization is: (finite dim) linear models have two sources of model error, and hence two ways of controlling the estimation error: (i) feature map choice and (ii) regularizer. Universal feature maps like the above, which are themselves functions (infinite dimensional), alleviate this redundancy, as linear functions over such (infinite dimensional) spaces could be non-restrictive. The catch however is the choice of the regularizer becomes crucial as it dictates the bias in the model.

In summary, let us use feature maps of the form $x \mapsto \phi(x)$, where $\phi(x)$ is itself a function of the form $\phi(x) : \mathcal{X} \mapsto \mathbb{R}$. Now, form a Hilbert (Euclidean) space, $\mathcal{H}$, over $\phi(\mathcal{X})$. This is easy to construct: elements are simply linear combinations of elements in $\phi(\mathcal{X})$ i.e., $\sum_{i=1}^k \phi(z_i)$, $z_i \in \mathcal{X}$. Let us endow this with a valid inner-product $\langle \cdot, \cdot \rangle_{\mathcal{H}}$. Now, linear functions in this space are of the form: $f(x) = \langle w, \phi(x) \rangle_{\mathcal{H}}$, $w \in \mathcal{H}$. The regularized risk minimization problems with this generalization of linear models read as:

$$(28.1) \quad \min_{w \in \mathcal{H}} \frac{1}{2} \|w\|_{\mathcal{H}}^2 + C \mathbb{E} [l(\langle w, \phi(X) \rangle_{\mathcal{H}}, Y)]$$

$$(28.2) \quad \min_{w \in \mathcal{H}} \frac{1}{2} \|w\|_{\mathcal{H}}^2 + \frac{C}{m} \sum_{i=1}^m [l(\langle w, \phi(x_i) \rangle_{\mathcal{H}}, y_i)]$$

Note the striking similarity with (15.3) and (15.4).

Though conceptually very simple, there are multiple practical issues. The first, is that (28.2)'s variable is a vector in an abstract infinite dimensional space of functions. So, optimization does not seem straight-forward. Secondly, typical inner-products over function spaces involve integrals², which will further make this problem cumbersome to deal with. Towards resolving these issues, we present the popular [Reisz representer theorem](#), which is nothing but a straight forward application of the projection theorem (duality): Any $w \in \mathcal{H}$ can be expressed as a sum of a linear combination of the Hilbert embeddings of the training points, $\sum_{i=1}^m \alpha_i \phi(x_i)$, and an orthogonal complement, $w_{\perp}$ i.e., $\langle w_{\perp}, \phi(x_i) \rangle_{\mathcal{H}} = 0 \ \forall \ i = 1, \dots, m$. Now,

²For e.g., $\langle f, g \rangle = \int f(x)g(x) \, dx$.

$\langle w, \phi(x_i) \rangle_{\mathcal{H}} = \langle w_{\perp} + \sum_{j=1}^m \alpha_j \phi(x_j), \phi(x_i) \rangle_{\mathcal{H}} = \langle \sum_{j=1}^m \alpha_j \phi(x_j), \phi(x_i) \rangle_{\mathcal{H}} = \sum_{j=1}^m \alpha_j \langle \phi(x_j), \phi(x_i) \rangle_{\mathcal{H}}$. Also, by pythagorean theorem, $\|w\|_{\mathcal{H}}^2 = \|\sum_{j=1}^m \alpha_j \phi(x_j)\|_{\mathcal{H}}^2 + \|w_{\perp}\|_{\mathcal{H}}^2$. Now, since the optimization problem wrt. the variable $w_{\perp}$ is simply $\min_{w_{\perp}} \|w_{\perp}\|_{\mathcal{H}}^2$, at optimality $w_{\perp} = 0$. Finally, $\|\sum_{j=1}^m \alpha_j \phi(x_j)\|_{\mathcal{H}}^2 = \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j \langle \phi(x_i), \phi(x_j) \rangle_{\mathcal{H}}$. This discussion can be summarized as:

Theorem 28.0.1. *Any optimal solution, $\hat{w}$, of (28.2) will be of the form $\hat{w} = \sum_{i=1}^m \alpha_i \phi(x_i)$. Hence (28.2) is equivalent to:*

$$(28.3) \quad \min_{\alpha \in \mathbb{R}^m} \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j \kappa(x_i, x_j) + C \sum_{i=1}^m l \left(\sum_{j=1}^m \alpha_j \kappa(x_i, x_j), y_i \right),$$

where $\kappa(x_i, x_j) \equiv \langle \phi(x_i), \phi(x_j) \rangle_{\mathcal{H}}$. Also, $\langle \hat{w}, \phi(x) \rangle_{\mathcal{H}} = \sum_{i=1}^m \alpha_i \kappa(x_i, x) \quad \forall x \in \mathcal{X}$.

Refer [section 4.2 in Scholkopf and Smola \[2001\]](#) or [theorem 5.4 in Mohri et al. \[2018\]](#) for more details of the proof.

This interesting theorem is useful in two ways: firstly, it is saying that the infinite-dimensional optimization problem (28.2) is equivalent to the finite dimensional (strongly) convex problem (28.3). It can be efficiently solved using SGD:

$$(28.4) \quad \alpha^{(k+1)} \equiv \alpha^{(k)} - s_k \left\{ C \, l' \left(y_{i_k}, \sum_{j=1}^m \alpha_j^{(k)} \kappa(x_{i_k}, x_j) \right) G_{i_k} + G \alpha^{(k)} \right\},$$

where G is the matrix whose (i, j) entry is $\kappa(x_i, x_j)$, called the gram-matrix. Comparing this with SGD update with (finite dim) linear models, the key difference is that the representation of x_i is G_{i_k} , the i^{th} column/row of the gram matrix, G .

Secondly, as long as the function κ can be computed efficiently, then we are in game. Last, but not the least, our goal is to employ ϕ , equivalently a κ function, such that the set of all linear functions over $\mathcal{H}$ is dense in all (measurable) functions, so that any target function can be arbitrarily well approximated. In subsequent lecture, we will look at such k functions, known as (universal) kernel functions.

Thirdly, the theorem says that the parametric linear model implicit in kernel methods can also be understood as models for $f(x) = \sum_{i=1}^m \alpha_i \kappa(x_i, x)$. Observe that here, the number of parameters, $\alpha \in \mathbb{R}^m$, is not fixed, and are dependent on the training set size. Such models whose form is not fixed and depends on the training set (parameters are not fixed and depend on the training set), are known as [non-parametric models](#). We will discuss more about them in subsequent lectures on nearest neighbour classifier etc. In summary, kernel models can be understood both as parametric models (in primal, $\langle w, \phi(x) \rangle$, form) and as non-parametric models (in dual, $\sum_{i=1}^m \alpha_i \kappa(x_i, x)$, form).

Lecture 29

Kernel Methods

Motivated by the Reisz representer theorem, we define:

Definition 29.0.1. *Given an arbitrary set $\mathcal{X}$, we define a kernel as any function, $\kappa : \mathcal{X} \times \mathcal{X} \mapsto \mathbb{R}$, such that there exists a Hilbert (Euclidean) space, say $\mathcal{H}_\kappa$, and a feature map, $\phi_\kappa : \mathcal{X} \mapsto \mathcal{H}_\kappa$, such that $\kappa(x, z) = \langle \phi_\kappa(x), \phi_\kappa(z) \rangle_{\mathcal{H}_\kappa} \forall x, z \in \mathcal{X}$. Here, $\langle \cdot, \cdot \rangle_{\mathcal{H}_\kappa}$ is the inner-product defining the space $\mathcal{H}_\kappa$.*

This definition is very convenient because: once I have a kernel, whose evaluations are computationally tractable, we can “blindly” employ it as the κ function in the context of the representer theorem. This is because then it is equivalent to considering a linear model in $\mathcal{H}_\kappa$ and running the entire ML pipeline of regularized risk minimization. Neither for solving (28.3), nor for computing the final predictor we need ϕ_κ or compute the inner-product etc. It is enough if the evaluations of the kernel function are tractable. In simple words, kernel renders our theoretical idea of employing infinite dimensional linear models, a practical one. This mathematical trick, popularized as [the kernel trick](#), is infact very general¹ and has applications beyond ML.

¹Infact, foundational rules of probabilistic inference like sum rule, product (chain) rule, Bayes rule etc. can themselves be “kernelized”. See for e.g., section 4 in <https://www.gatsby.ucl.ac.uk/~gretton/papers/SonFukGre13.pdf>. The advantage is that integrals etc. in probabilistic inference reduce to matrix/vector operations, making them computationally efficient on GPUs.

29.1 Kernel Trick

The kernel trick described above can be applied to any problem you encounter involving solely geometric notions such as lengths, distances, angles, projections etc. The motivation could be generalizing linear artifacts in the problem to non-linear ones. Or, the motivation could be generalizing from finite dimensions to infinite dimensions etc. In particular, employing square loss, hinge-loss, logistic loss in (28.3) leads to so-called [kernelized ridge regression](#), [kernelized SVM](#), [kernelized logistic regression](#) respectively. The formulations respectively are:

Kernelized Ridge-regression:

$$(29.1) \quad \min_{\alpha \in \mathbb{R}^m} \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j \kappa(x_i, x_j) + C \sum_{i=1}^m \left(\sum_{j=1}^m \alpha_j \kappa(x_i, x_j) - y_i \right)^2,$$

Consider the matrix of kernel evaluations: i, j entry in G is $\kappa(x_i, x_j)$. This is known as the [gram matrix](#) of the training points with kernel κ . In terms of the gram matrix, this problem reads as:

$$(29.2) \quad \min_{\alpha \in \mathbb{R}^m} \frac{1}{2} \alpha^\top G \alpha + C \|G \alpha - y\|^2$$

The optimality conditions are: $G \hat{\alpha} + C (2G^2 \alpha - 2Gy) = 0$. Now $G \succeq 0$: this is because $\beta^\top G \beta = \sum_{i,j} \beta_i \beta_j \kappa(x_i, x_j) = \sum_{i,j} \beta_i \beta_j \langle \phi_\kappa(x_i), \phi_\kappa(x_j) \rangle_\kappa = \langle \sum_i \beta_i \phi_\kappa(x_i), \sum_i \beta_i \phi_\kappa(x_i) \rangle \geq 0$. This shows gram matrix of any set of points with any kernel is psd. One important practical application of this result is that the problem (28.3) is convex² whenever κ is a valid kernel, and the loss is convex. So SGD (28.4) is guaranteed to converge as per table 20.1 (take $n \approx m$).

Further, if the kernel is such that the gram-matrix is invertible i.e., $G \succ 0$, then this optimality condition simplifies as³: $\hat{\alpha} = 2(I + 2CG)^{-1}y$. Therefore, the kernel ridge regressor is $\hat{f}(x) = 2G_x^\top (I + 2CG)^{-1}y$, where G_x is the column vector with j^{th} entry as $\kappa(x_j, x)$.

Kernelized SVM:

$$(29.3) \quad \min_{\alpha \in \mathbb{R}^m} \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j \kappa(x_i, x_j) + C \sum_{i=1}^m \max \left(0, 1 - y_i \left(\sum_{j=1}^m \alpha_j \kappa(x_i, x_j) \right) \right),$$

The solution $\hat{\alpha}$ can be found by running⁴ GD/SGD⁵. Then, kernelized SVM classifier is: $\hat{f}(x) = \text{sign}(G_x^\top \hat{\alpha})$.

²And infact, strongly convex whenever the $G \succ 0$.

³Note that $I + 2CG$ is invertible since $G \succeq 0$.

⁴Due to some special characteristics of the optimal solution, the most efficient way of solving this problem is coordinate descent.

⁵To scale SGD to massive datasets, one typically approximates using Nystroem approximation.

Likewise, discriminative models like multi-class logistic regression (26.1) can also be kernelized⁶ by replacing $w_y^\top \phi(x)$ by $\sum_{i=1}^m \alpha_{yi} \kappa(x_i, x)$:

$$(29.4) \quad \arg \min_{\alpha_1 \in \mathbb{R}^m, \dots, \alpha_c \in \mathbb{R}^m} \sum_{i=1}^m \left\{ \log \left(\sum_{j=1}^c e^{\sum_{p=1}^m \alpha_{jp} \kappa(x_p, x_i)} \right) - \sum_{p=1}^m \alpha_{y_i p} \kappa(x_p, x_i) \right\}$$

Also, the corresponding kernelized Bayes optimal (26.2) reads as:

$$(29.5) \quad \hat{f}(x) = \arg \max_{y \in \mathcal{Y}} \frac{e^{\sum_{p=1}^m \hat{\alpha}_{yp} \kappa(x_p, x)}}{\sum_{i=1}^c e^{\sum_{p=1}^m \hat{\alpha}_{ip} \kappa(x_p, x)}},$$

where $\hat{\alpha}_1, \dots, \hat{\alpha}_c$ is the solution⁷ to (29.4).

29.2 Kernels

In order to understand kernels more deeply, we gave a lot of examples of kernels over $\mathcal{X} = \mathbb{R}^n$: the first are those that are already inner-products: $\kappa(x, y) = \langle x, y \rangle$. This is a kernel because we can choose $\phi(x) = x$. Next example is $\kappa(x, y) = x_1 y_1$, which is not an inner-product over $\mathbb{R}^n$. However, this is a kernel because with $\phi(x) = x_1$, κ is the dot-product in 1-d (real) space. Generalizing this we give next example: $\kappa_M(x, y) \equiv x^\top M y$, where $M \succeq 0$ is a hyperparameter. This is a kernel because with $\phi(x) = M^{1/2} x$. Then, $\kappa(x, y) = \phi(x)^\top \phi(y)$. With such kernels, the predictive function $f(x) = \sum_{j=1}^m \alpha_j \kappa(x_j, x) = \sum_{j=1}^m \alpha_j x_j^\top M x$, which is linear. Further, $\kappa_{M,c}(x, y) = x^\top M y + c$, $M \succeq 0, c \geq 0$, is also a kernel as $\phi(x) = \begin{bmatrix} M^{1/2} x \\ \sqrt{c} \end{bmatrix}$ is appropriate. This leads to affine functions: $f(x) = \sum_{j=1}^m \alpha_j \kappa(x_j, x) = \sum_{j=1}^m \alpha_j x_j^\top M x + c \sum_{j=1}^m \alpha_j$. So such kernels are known as [linear kernels](#).

Then we gave examples like $\kappa(x, y) = (x^\top y)^2$. This is a kernel because with $\phi(x) = \text{vec}(xx^\top)$, we have $\kappa(x, y) = (x^\top y)^2 = \phi(x)^\top \phi(y)$. Similarly, $\kappa_{M,c}(x, y) = (x^\top M y + c)^2$ is kernel. Because $\phi(x) = \begin{bmatrix} \text{vec}(M^{1/2} x x^\top M^{1/2}) \\ M^{1/2} x \sqrt{c} \end{bmatrix}$ does the magic. Such kernels are known as [quadratic kernels](#).

⁶i.e., representer theorem analogue holds for this problem too. Think why? But, it is not straight-forward to kernelize generative models (23.4). think why? (hint: does representer theorem hold for (23.4)? why not?).

⁷As footnoted earlier, for modern massive datasets, Nystroem approximation is applied and then a usual SGD is run to obtain the ERM solution.

Analogously, one can define **polynomial kernels** as $\kappa_{M,c,d}(x, y) = (x^\top M y + c)^d$. Since it may become cumbersome to verify the definition (i.e., explicitly give a ϕ), we presented four basic theorems (without proof), which we can use to prove a given function is a valid kernel.

Theorem 29.2.1. *Let κ_1, κ_2 be two valid kernels and $c \geq 0$.*

1. *Then $\kappa_1 + \kappa_2$ is a kernel.*
2. *Then $c\kappa_1$ is a kernel.*
3. *Then $\kappa_1\kappa_2$ is a kernel.*
4. *Then $\bar{\kappa}$ defined as $\bar{\kappa}(x, y) = \frac{\kappa(x, y)}{\sqrt{\kappa(x, x)}\sqrt{\kappa(y, y)}}$, is also a kernel. This is called the normalized/cosine kernel associated with κ .*

Using these theorems it is easy to show polynomial kernels are valid kernels. The key advantage with these polynomials kernels is that they render our lecture 11 idea of employing d degree polynomial based linear models a practical one as computing polynomial kernels is tractable. Infact, given some basic kernels, one can create *advanced* kernels by using these theorems.

While this is encouraging, none of these kernels, though valid, seem to model all possible functions (all degree monomials) i.e., none of them actually need infinite dimensional $\phi, \mathcal{H}$. One example that can induce any (analytic) function is the exponential kernel $\kappa(x, y) = e^{x^\top y}$. By it's Taylor expansion it is clear that this involves all monomials (i.e., infinite dimensional ϕ). To rigorously prove it is kernel we use this theorem:

Theorem 29.2.2. *If $\kappa_1, \kappa_2, \dots, \kappa_n, \dots$ is an infinite sequence of (valid) kernels, and $\lim_{n \rightarrow \infty} \kappa_n$ exists, then this limiting function is a valid kernel.*

With this kernel, $f(x) = \sum_{j=1}^m \alpha_j e^{x_j^\top x}$. Hence it can model any (analytic) function. The normalized/cosine kernel corresponding to this is known as the **Gaussian kernel**: $\kappa(x, y) = e^{-\frac{1}{2}\|x-y\|^2} = \frac{e^{x^\top y}}{\sqrt{e^{x^\top x}}\sqrt{e^{y^\top y}}}$.

Like exponential, we can build interesting kernels by using any function whose Taylor expansion has non-negative coefficients (like cosh). Analogously, one can construct such useful kernels using functions whose Fourier coefficients are non-negative i.e., non-negative combinations of Fourier basis functions. One such example kernel is the so-called **Inverse Multi-Quadratic (IMQ)** kernel⁸. Kernels

⁸Interested students may read up on Bochner's theorem (prop 7.4 in Bach [2024]).

like Gaussian, IMQ that can approximate any function (model error near zero) are known as [universal kernels](#)⁹. Finally, sometimes one may want to disprove that a function is a kernel. In such cases, one may use the Aronszajn's theorem, which gives a full characterization of kernels (necessary and sufficient conditions). Interested students may read proposition 7.3 in Bach [2024] for details.

⁹Interested students may read up https://proceedings.neurips.cc/paper_files/paper/2010/file/4e0cb6fb5fb446d1c92ede2ed8780188-Paper.pdf.

Lecture 30

Modularity in Modelling

Though extremely powerful and mathematically very elegant with theoretical guarantees, kernel methods (and hence linear models) are not modular. In the sense that complicated models cannot be build from simpler models and so on. Secondly, the bias they encode is cryptically hidden inside the regularizer, and hence inside the kernel¹. Moreover, the kernel needs to satisfy some technical conditions (e.g., Aronszajn's conditions). These issues render designing kernels a relatively challenging/technical task (a decade 1995 – 2005 was spent on designing kernels for Vision, Text, Speech, Biology applications etc.).

In contrast, so-called neural network models are modular by design. To motivate them, we gave the example of pattern recognition in vision. If we were asked, how we recognized digits, we would probably say, by the way they look. This is what perhaps one would argue a well-trained kernel/linear model is also doing: for example, let's assume $\hat{f}(x) = \arg \max_{y \in \{0,1,\dots,9\}} w_y^\top \phi(x)$, is generalizing well to recognize digits. Then, just because of the form of the inner-product, $\phi^{-1}(w_y)$ would (visually) have to look like the y^{th} digit; otherwise, the inner-product would not be the highest for the right label. So essentially, kernel/linear models learn the way digits look like. The estimated parameters encode the digit shapes.

Now, if the task is changed slightly to recognizing alphabets, the learnt kernel/linear parameters again encode “shapes/patterns” that correspond to alphabets. However, there is nothing common in these models. Now, consider a situation where instead of one flat pattern recognition, we consider models that allow for a hierarchy of pattern recognitions i.e., a sequence of compositions of patterns of patterns. Then, the learnt models for these two taks may have a rough

¹So-called Matern kernels were designed to encode smoothness are a modelling bias. But, in general, given a bias, how to encode it inside a kernel ?

overlap, atleast in first few basic levels of patterns i.e., a few basic patterns are commonly recognized. In simple words, this shows that the ability to recognize basic patterns is common across the two tasks, which is perhaps not far from the truth.

We then wrote more details on models that can recognize a hierarchy of patterns; instead of a flat set of patterns like kernel methods. Consider $w_i^{\{1\}}, i = 1, \dots, n_1$, which are basic (level 1) patterns over pixels. There will be a match for such patterns in the input image, x , whenever the corresponding score $(w_j^{\{1\}})^\top x$, exceeds a threshold (say, 0). Since this 0-1 function is discontinuous, we may use $\max\left(0, (w_j^{\{1\}})^\top x\right)$ (hinge-loss style). The level 1 patterns recognized in x are $x^{\{2\}} \equiv \max\left(0, (W^{\{1\}})^\top x\right)$, where W is the matrix with i^{th} column as $w_i^{\{1\}}$. We can now model patterns over these level 1 patterns to give level 2 patterns: $x^{\{3\}} \equiv \max\left(0, (W^{\{2\}})^\top x^{\{2\}}\right)$ and so on If such models are employed, then one would expect that the last level patterns represent the patterns for the classes (digits/alphabets etc.) themselves, just like in the case of kernel methods. But now the difference is that the intermediate patterns recognized are meaningful and correspond to more finer/basic patterns.

While this example may seem very specific, the key advantage with such models, in general, is that they are modular in nature by their very construction. Hence, complicated tasks can be modelled using models for simpler tasks. For example, suppose “encoder” models that represent inputs from vision, NLP, speech domains in a semantic meaningful way are available. Likewise, say, “decoder” models that produce outputs in these domains are also available. Now, one can use them to build “encoder-decoder” models that convert, say, text to corresponding image and vice-versa. Modularity is the key to build complex systems. This I think is the key to their success in modern AI systems.

In the next lecture we will formally study such compositional models, known as Neural Networks, whose compositional structure can be described using directed acyclic graphs (DAGs). Since complex models are built using a deep hierarchy of simpler and simpler models, these models are dubbed as “deep learning models”.

Lecture 31

Deep Learning Models: Neural Networks

A **Neural network model** is the set of functions induced by a given DAG where nodes are parametrized functions, where a directed edge from node u to v denotes that the function at v uses the output of that at u as one of its inputs. The special nodes that do not have any parents *store* the input x and are known as the **input nodes**. These input nodes are referred to as the **input layer**. The special nodes, which do not have any children are known as **output nodes**. These output nodes are referred to as the **output layer**. The outputs (activations) of the functions in these output nodes denote the evaluation of the induced function for a given input. Note that both the DAG and the parametrized functions need to be fixed for defining the model. Various functions induced by the model are obtained by varying the parameters. See also **section 20.1 in Shalev-Shwartz and Ben-David [2014]**.

For e.g., consider the DAG $x \rightarrow l_U \rightarrow a \rightarrow l_V \rightarrow a \rightarrow l_W$, where $l_W(x) \equiv W^\top x$ is just a linear unit (linear model parameterized by W), and $a(x) \equiv \max(0, x)$ (elementwise max). a is also known as the **ReLU** activation function¹. The nodes representing x are the input nodes. The output of the function l_W (output layer) is the output of the function. It is easy to see that this model is the same as the motivating example from the previous lecture for visual pattern recognition. This is because the function induced is $l_W(a(l_V(a(l_U(x)))))$. Columns of U denote the first level (basic) patterns. $l_U(x)$ hence denote the score for various first level patterns appearing in x . $a(l_U(x))$ are the activations of the patterns. Likewise, V denote patterns over patterns (second level patterns).... and so on. Observe that the DAG in this model is a linear chain. And it consists of only two kinds

¹Other examples of popular activation functions are on pg 249 in Bach [2024].

of functions: l_U , which is just a linear function parametrized by U and a , which is just a particular thresholding/activation function. So this model can be also understood as repeated composition of $a(l)$ with itself, followed by a linear transformation. Such neural network models that are composition of $a(l)$ with itself for d times, followed by an output layer are known as [Multilayer Perceptrons \(MLPs\)](#) or [Feed-forward Neural Nets \(FFNNs\)](#). The outputs of the i^{th} $a(l)$ is known as the i^{th} [layer activation](#). Each $a(l)$ block is referred to as a [layer](#) or [hidden layer](#). The number of intermediate (hidden) layers, without counting the input/output layer, i.e., d , is known as the [depth](#) of an MLP. The maximum number of columns in any parameter matrix in an intermediate layer is known as the [width](#) of the MLP.

As motivated in the last lecture, the key advantage of these network models is modularity: we can pick a network model and compose it with another and so on to build complex models using simple models. For example, consider the task of generating images corresponding to given captions. One can pick an “encoder” network, which itself could be multilayered, and encodes textual captions into semantic representations. Such models are studied in NLP (natural language processing) e.g., transformers. One can then pick a “decoder” network, which itself could be multilayered, and is capable of generating images. Such models are studied in computer-vision e.g., diffusion models. One can then simply create a more complex “encoder-decoder” network model by composing the encoder with the decoder. Such models can generate images from captions (e.g., stable diffusion). More complex the task, the deeper the network model (more compositions). Hence these models also known as [deep learning](#) models.

Though modularity was the key, classical applications of neural networks in late 20th century focused on “shallow” learning i.e., models with very few compositions (layers), as the optimization techniques very restrictive. But in early 2010s, due to the emergence of sophisticated variants of SGD, efficient GPU codes and enormous compute, availability of large data, deep learning (several compositions/layers) was actually possible.

Infact, most of the early analysis of neural network models was in the shallow regime. For example, Hornik et al. [1989] proves that an (infinitely) wide (shallow) network of unit depth can well approximate *any* function i.e, depth-bound networks are universal approximators. In other words, wider the network, lower is the model error, in general. But such results may not be interesting as this is what kernel based models do anyway: they employ an infinite dimensional feature space i.e., a non-hierarchical, flat collection of huge number of patterns (shallow; wide). Recent works have also established more intriguing connections between wide networks and kernel methods Jacot et al. [2018], Chizat et al. [2020].

However, motivated by the success of deep learning models, corresponding

approximation results for width-bound networks have also been studied: For example, it has been shown that width-bound networks are also universal approximators, provided that they satisfy a minimum width requirement: see table 1 in <https://openreview.net/pdf?id=dpDw5U04SU>. In other words, deep learning models approximate functions in a very different way than kernel methods. Interested students may also look into interesting results like <https://proceedings.mlr.press/v49/eldan16.pdf>, which, in informal words, say that depth is exponentially more valuable than width.

Coming to estimation error, bounds similar to those earlier can be derived. For example, theorem 21.5 in Anthony and Bartlett [1999] gives: $O\left(d\sqrt{\frac{n}{m}}\right)$, where d is the depth and n is the number of parameters. Bounds in terms of norm can also be derived: for example, exercise 9.2 in Bach [2024] provides a bound $O\left(\frac{W^d R}{\sqrt{m}}\right)$, where W is bound on norm of the parameter of each linear unit. In summary, if the parameters are not regularized enough, then depth seem to hurt estimation, which brings us back to the usual model vs estimation error trade-off. Later on, when we analyze so-called overparametrized models, we will discover more interesting regimes where this trade-off may be completely absent, further justifying the success of modern deep learning.

While the situation seems familiar in terms of generalization error, it is kind of unfavourable in terms of optimization error. This is because the risk minimization problems (true one as well as empirical one) are those with highly non-convex objectives. Hence, table 20.1 may not hold as is. Infact, it has been shown, e.g., theorem 20.7 in Shalev-Shwartz and Ben-David [2014], that the problem is NP-hard. In practice, still SGD is employed hoping for generalizing solutions. Several heuristics help in better convergence: preconditioning Duchi et al. [2011], momentum Kingma and Ba [2017], batch normalization Ioffe and Szegedy [2015], and layer normalization Ba et al. [2016].

The only theoretically well established case where this non-convex problem can be solved efficiently is that where the width is infinity and depth is one Bach and Chizat [2021], which is then equivalent to kernel methods². A convincing explanation of why/how optimization error may be low with ERM+SGD in deep learning models is a challenging open problem³.

²Many other works that kind of show SGD convergence essentially reduce to (enforce) this case.

³There are some initial attempts like https://proceedings.neurips.cc/paper_files/paper/2018/file/a41b3bb3e6b050b6c9067c67f663b915-Paper.pdf.

Lecture 32

Back propagation

While the gradient computation (for use in SGD) in case of linear and kernel models was straight forward, it is non-trivial in case of neural nets. This is because of the potentially deep compositional nature of the induced function. Theoretically, ofcourse, we can write down the gradient of the risk objective in terms of first order derivatives of the basic units using chain rule. However, some key observations lead to efficient computations. The main reference for this lecture is [section 13.3 in Murphy \[2022\]](#).

We began with the discussion in section 13.3.1. The summary is: consider $f(w) \equiv f_d(f_{d-1}(\dots f_2(f_1(w))))$, Let $w_{i+1} \equiv f_i(w_i)$ and $w_1 \equiv w$. then, the Jacobian $\frac{\partial f(w)}{\partial w} = \frac{\partial f_L(w_L)}{\partial w_L} \frac{\partial f_{L-1}(w_{L-1})}{\partial w_{L-1}} \dots \frac{\partial f_2(w_2)}{\partial w_2} \frac{\partial f_1(w_1)}{\partial w_1}$. Now, such a computation can be done in a forward direction i.e., from right to left or in a backward direction i.e., left to right. range of f_d is lower dimensional than that of the domain of f_1 , then, backward is more efficient as it will require multiplication with (and storage of) a smaller size matrix at every layer than forward and vice-versa. In ML, since f_L is always the loss function, it's output dimensionality is unity and hence invariably, backward computation is more efficient. This application of chain rule in the backward direction is known [back propagation](#) or [backprop](#). The same can be extended to a general DAG as detailed in section 13.3.4.

We then took examples of basic units and derived formulae for the Jacobian. See section 13.3.3.3 for formulae for linear unit and section 13.3.3.2 for the ReLU. We finished the discussion with the example in section 13.3.2.

Lecture 33

Overparametrized Models

Deep learning models are perhaps the most successful models as of today, thanks to their modular nature. Today's LLMs (Large Language Models) and other deep models, are indeed deeply layered and involve billions of parameters. Practitioners have observed various interesting phenomenon in deep learning, which have motivated further studies. Most of these are later understood to be not specific to deep learning models, but are more general ideas for which deep learning was perhaps the play ground.

One such observation is that explained in the seminal work: <https://arxiv.org/pdf/1812.11118>. The summary is that when we have so-called **overparametrized models**: complex models where training error can be lowered to zero e.g., number of parameters in a linear model is less than the number of samples, then, the usual model-error-vs-estimation-error tradeoff does not apply. This is interesting because both model error and estimation error can be reduced simultaneously by employing more complex models.

Firstly, though intriguing, this is not contradicting the classical tradeoff discussed in chapter 11. This is because all the analysis we presented earlier, especially in chapters 11, 20, is for the underparametrized regime i.e., $n < m$.

In order to understand this phenomenon better, we continued our discussion from chapter 10. Recall that the expected estimation error with $\hat{w}_m$ is: $\mathbb{E}[R[\hat{w}^m]] - R[w^*] = \mathbb{E}[(\hat{w}^m - w^*)^\top \mathbb{E}[X X^\top](\hat{w}^m - w^*)]$. In the underparametrized regime, we considered $\hat{w}_m$ as the least squares solution: $\hat{w}_{ls} = (D D^\top)^{-1} D y$. In the overparametrized regime, there will be many (ERM) solutions that interpolate i.e., empirical risk is zero. If we pick some arbitrary ERM solution, then no generalization can be guaranteed (e.g., memorization: recall discussion from chapter 7). However, if we norm-regularize, i.e., pick the ERM with least norm, then such a

min norm interpolant, $\hat{w}_{ln}$, we will show has intriguing generalization properties.

From linear algebra¹ or optimization², we know that $\hat{w}_{ln} = D (D^\top D)^{-1} y$ (assuming $n > m$ and D is full column rank³). Indeed, $\hat{w}_{ln}$ is an ERM solution because $D^\top \hat{w}_{ln} = D^\top D (D^\top D)^{-1} y = y$. Moreover, consider another ERM solution, $\hat{w}$, i.e., $D^\top \hat{w} = y$. Then it is easy to see that $\hat{w} - \hat{w}_{ln} \perp \hat{w}_{ln}$: $(\hat{w} - \hat{w}_{ln})^\top \hat{w}_{ln} = (\hat{w} - \hat{w}_{ln})^\top D (D^\top D)^{-1} y = (D^\top \hat{w} - D^\top \hat{w}_{ln})^\top (D^\top D)^{-1} y = (y - y)^\top (D^\top D)^{-1} y = 0$. Hence, by projection/Pythagoras theorem, we have $\|\hat{w}\|^2 = \|\hat{w} - \hat{w}_{ln}\|^2 + \|\hat{w}_{ln}\|^2 \geq \|\hat{w}_{ln}\|^2$. Hence, $\hat{w}_{ln}$ is indeed the min norm interpolant's parameter.

Now the expected estimation error with the min-norm interpolant is: $\mathbb{E}[R[\hat{w}_{ln}]] - R[w^*] = \mathbb{E}[(\hat{w}_{ln} - w^*)^\top \mathbb{E}[XX^\top](\hat{w}_{ln} - w^*)]$. Along with the assumptions earlier that: N is independent of X , $\mathbb{E}[N] = 0$, and the samples $n_i \sim N$ are mutually independent, we also assume $\mathbb{E}[XX^\top] = I$, just to simplify the presentation:

$$\begin{aligned}
& \mathbb{E}[R[\hat{w}_{ln}]] - R[w^*] \\
&= \mathbb{E}[(\hat{w}_{ln} - w^*)^\top (\hat{w}_{ln} - w^*)] \\
&= \mathbb{E}\left[\left(D (D^\top D)^{-1} y - w^*\right)^\top \left(D (D^\top D)^{-1} y - w^*\right)\right] \\
&= \mathbb{E}\left[\left(D (D^\top D)^{-1} (D^\top w^* + n) - w^*\right)^\top \left(D (D^\top D)^{-1} (D^\top w^* + n) - w^*\right)\right] \\
&= \mathbb{E}\left[n^\top (D^\top D)^{-1} D^\top D (D^\top D)^{-1} n\right] + \mathbb{E}\left[(w^*)^\top \left(D (D^\top D)^{-1} D^\top - I\right)^2 w^*\right] \\
&\quad + 2 \underbrace{\mathbb{E}\left[n^\top (D^\top D)^{-1} D \left(D (D^\top D)^{-1} D^\top - I\right) w^*\right]}_{=\mathbb{E}[n]^\top \mathbb{E}\left[(D^\top D)^{-1} D \left(D (D^\top D)^{-1} D^\top - I\right) w^*\right]=0} \\
(33.1) \quad &= \mathbb{E}\left[n^\top (D^\top D)^{-1} n\right] + \mathbb{E}\left[(w^*)^\top \left(I - D (D^\top D)^{-1} D^\top\right) w^*\right]
\end{aligned}$$

Now, similar to (10.2), we can apply Jensen's inequality to lower bound the vari-

¹(eqn (7.78), section 6C in <https://linear.axler.net/LADR4e.pdf>).

²pg 39-40 in <https://www.cse.iitb.ac.in/~saketh/teaching/cs723Scans3.pdf>.

³With universal kernels, the gram matrix $D^\top D$ is always invertible.

ance:

$$\begin{aligned}
& \mathbb{E} \left[n^\top (D^\top D)^{-1} n \right] \\
&= \mathbb{E} \left[\text{trace} \left(n^\top (D^\top D)^{-1} n \right) \right] \\
&= \mathbb{E} \left[\text{trace} \left((D^\top D)^{-1} n n^\top \right) \right] \\
&= \mathbb{E} \left[\text{trace} \left((D^\top D)^{-1} \mathbb{E} [n n^\top] \right) \right] \quad \because n \perp X \\
&= \mathbb{E} \left[\text{trace} \left((D^\top D)^{-1} \mathbb{E} [N^2] I \right) \right] \quad \because n_i \text{ are iid} \\
&= \mathbb{E} [N^2] \mathbb{E} \left[\text{trace} \left((D^\top D)^{-1} \right) \right] \\
&\geq \mathbb{E} [N^2] \text{trace} \left(\left(\mathbb{E} [D^\top D] \right)^{-1} \right) \quad \because \text{Jensens Inequality} \\
&= \mathbb{E} [N^2] \text{trace} \left(\text{diag} \left(\begin{bmatrix} \mathbb{E} [\|X_1\|^2] \\ \vdots \\ \mathbb{E} [\|X_m\|^2] \end{bmatrix} \right)^{-1} \right) \quad \because \text{training samples are independent} \\
&= \mathbb{E} [N^2] \text{trace} \left(\text{diag} \left(\begin{bmatrix} \sum_{j=1}^n \mathbb{E} [X_{1j}^2] \\ \vdots \\ \sum_{j=1}^n \mathbb{E} [X_{mj}^2] \end{bmatrix} \right)^{-1} \right) \\
&= \mathbb{E} [N^2] \text{trace} \left(\text{diag} \left(\begin{bmatrix} n \\ \vdots \\ n \end{bmatrix} \right)^{-1} \right) \quad \because \mathbb{E} [X_{ij}^2] = 1 \\
&= \mathbb{E} [N^2] \text{trace} \left(\text{diag} \left(\begin{bmatrix} 1/n \\ \vdots \\ 1/n \end{bmatrix} \right) \right) \\
&= \mathbb{E} [N^2] \frac{m}{n}
\end{aligned}$$

Contrast this with the result (10.2). It is just like the roles of n, m are interchanged! Now let's assume that X is a distribution with rotational invariance, like the Standard Normal. i.e., the likelihood of points remains exactly the same even after rotations. So the bias term $\mathbb{E} \left[(w^*)^\top \left(I - D (D^\top D)^{-1} D^\top \right) w^* \right] = \|w^*\|^2 - \mathbb{E} \left[(w^*)^\top D (D^\top D)^{-1} D^\top w^* \right] = \|w^*\|^2 - \|w^*\|^2 \mathbb{E} \left[(e_j)^\top D (D^\top D)^{-1} D^\top e_j \right]$, where e_j is j^{th} standard basis vector. The last equality is true because even if we rotate the axis such that $\frac{w^*}{\|w^*\|} \rightarrow e_j$, the likelihood of D does not change. In

particular, the expectation in the bias term does not change. Again,

$$\begin{aligned}
& \mathbb{E} \left[(e_j)^\top D (D^\top D)^{-1} D^\top e_j \right] \\
&= \frac{1}{n} \sum_{i=1}^n \mathbb{E} \left[(e_i)^\top D (D^\top D)^{-1} D^\top e_i \right] \\
&= \frac{1}{n} \mathbb{E} \left[\mathbf{1}^\top D (D^\top D)^{-1} D^\top \mathbf{1} \right] \\
&= \frac{1}{n} \mathbb{E} \left[\text{trace} \left(D (D^\top D)^{-1} D^\top \right) \right] \\
&= \frac{1}{n} \mathbb{E} \left[\text{trace} \left((D^\top D)^{-1} D^\top D \right) \right] \\
&= \frac{1}{n} \mathbb{E} [\text{trace} (I)] \\
&= \frac{m}{n}
\end{aligned}$$

Therefore, the expected estimation error with min-norm interpolant is atleast: $\|w^*\|^2 + (\mathbb{E}[N^2] - \|w^*\|^2) \frac{m}{n}$. Firstly, this bound shows that overparametrized learning is useful only in the low noise setting i.e., $\mathbb{E}[N^2] < \|w^*\|^2$. Otherwise, the error, $\|\hat{w}_{in} - w^*\|^2$, is atleast $\|w^*\|^2$, which is quite huge (relative error is 100%). In such a low noise setting too, from the lower bound it seems there may be hard problems where overparametrization must be degenerate i.e., $n = m$ (kernel methods?).

Ofcourse, this is just the trend with the lower bound. One must look at upper bounds and/or special cases too in order to conclude anything generic. For example, if we make the simplifying assumption that X is standard Normal even while simplifying the variance term itself, then instead of using Jensens inequality, we can directly compute $\mathbb{E}[N^2] \text{trace} \left(\mathbb{E} \left[(D^\top D)^{-1} \right] \right)$ as, then, $(D^\top D)^{-1}$ is inverse-Wishart distributed. So, $\mathbb{E} \left[(D^\top D)^{-1} \right] = \frac{m}{n-m-1}$ for $n \geq m+2$. Hence, the expected estimation error is exactly equal to:

$$(33.2) \quad \|w^*\|^2 \left(\frac{n-m}{n} \right) + \mathbb{E}[N^2] \frac{m}{n-m-1}$$

Note that this estimation error is minimized when $n = \frac{\|w^*\|}{\|w^*\| - \sqrt{\mathbb{E}[N^2]}}(m+1)$ and⁴ the minimum value is $\|w^*\|^2 - \frac{m}{m+1} \left(\|w^*\| - \sqrt{\mathbb{E}[N^2]} \right)^2$. Further, this is minimized as $m \rightarrow \infty$. However, unlike in the underparametrized case, the estimation error does not go to zero. Asymptotically, as $m \rightarrow \infty$, it reaches $\|w^*\|^2 - \left(\|w^*\| - \sqrt{\mathbb{E}[N^2]} \right)^2$.

⁴Again, note that the minimizing n is meaningful only when noise is low i.e., $\mathbb{E}[N^2] < \|w^*\|^2$.

This is low only if noise is low. This shows a fundamental difference between learning in the underparametrized regime and the overparameterized regime: While in the former, the idea is to underfit data in order to eliminate the adverse effect of noise; in the latter, the noise is “consumed/encoded” inside the parameters, but still meaningful learning happens as one is looking for a min norm solution: of all parameters that “digest” noise, pick the one with least absorption, so that the parameter still generalizes. Though estimation error may not decay to zero, since model error decreases with number of parameters, the generalization error may still be good and better/comparable with that in underparameterized regime.

Another notable difference is that regularization is absolutely essential for estimation in overparametrization⁵. This is because an arbitrary interpolant (zero empirical risk) may incur arbitrarily high estimation error (recall chapter 7). However, though regularization may help, it is not necessary for estimation in the underparametrized regime. Also, unlike in the case of the underparametrized regime, in the overparametrized regime there is no regularization hyperparameter that controls the degree of regularization. So the regularization in overparametrization may not play a critical role in model error.

Visually, estimation error plot looks like fig 33.1. Infact, since model error is expected to decrease as number of parameters increases, the overall generalization error looks two ‘U’s (illustrated⁶ in fig 33.2): one ‘U’ before m and one ‘U’ after m . As mentioned earlier, it is not clear if the sweet spot ‘U’ in underparamterization generalizes better than the sweet spot ‘U’ in the overparametrization and vice-versa. Characterizing this is an interesting open problem. However, since estimation error in overparametrization plateaus asymptotically ($n, m \rightarrow \infty, n/m \rightarrow \text{constant}$), sweet spot 2 may be more “stable” in the sense that generalization error will not steeply increase if we consider unnecessarily complex models.

Interested students may read <https://arxiv.org/pdf/1906.11300> and related works for further details. This lecture is based on [section 12.2 in Bach \[2024\]](#).

⁵Regularization may be implicit/algorithmic: e.g., the optimization algorithm (SGD) may itself provide the regularization Vardi [2023].

⁶The generalization error at sweet spot 2 is lower than that at sweet spot 1 in the illustration. But it may as well happen the other way.

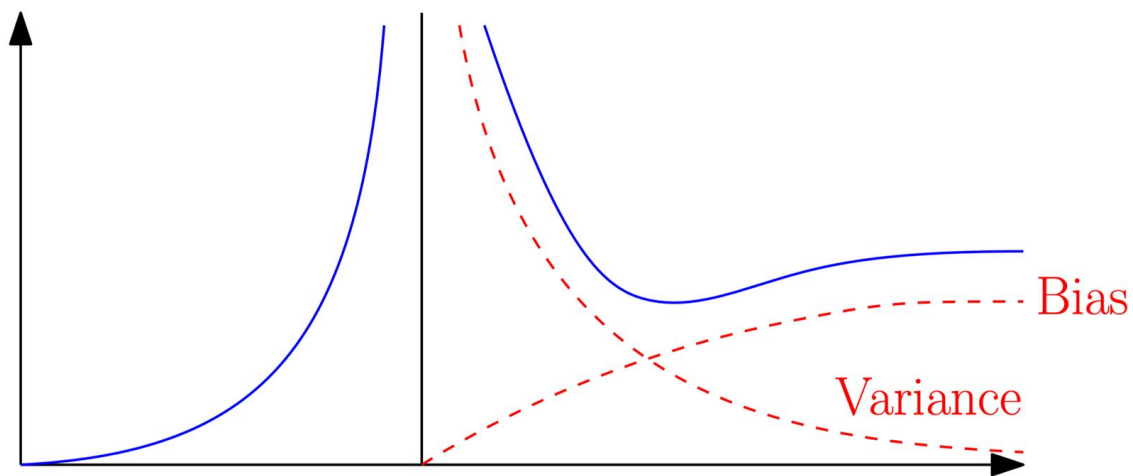


Figure 33.1: Double descent: Estimation error. Reproduced from Bach [2024] book.

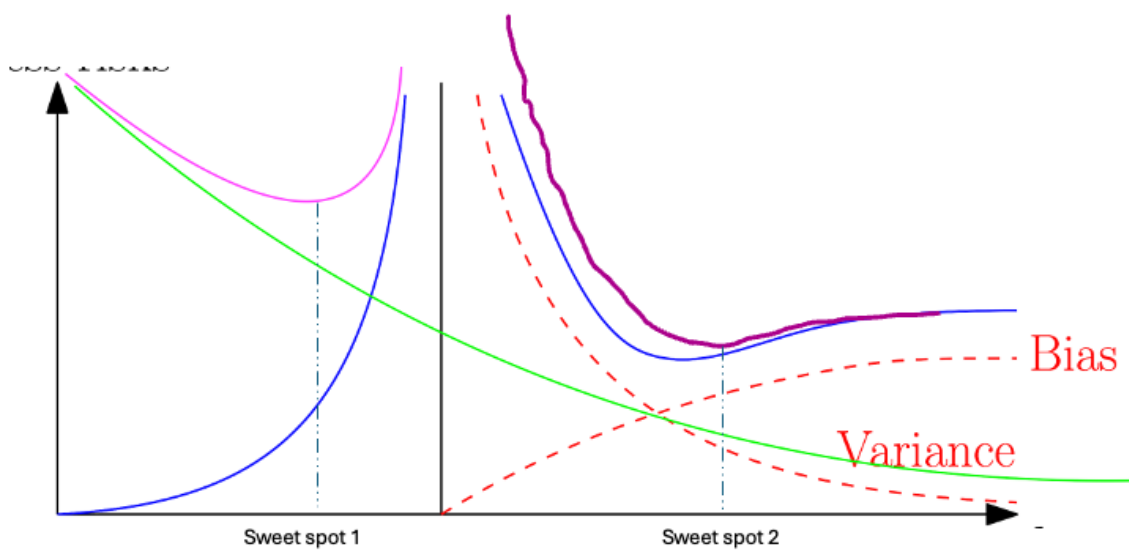


Figure 33.2: Double descent: Generalization error. Blue curve is estimation error. Red dashed is the two terms in estimation error in overparametrized regime. Green curve is model error. Light magenta is generalization in underparametrization and dark magenta is generalization in overparametrization. Edited from Bach [2024] book.

Lecture 34

Non-parametric Models: k -nearest neighbour

Linear, neural network based models, and others based on them are so-called **parametric models**. Here, learning is the same as parameter estimation such that the underlying concept/task is implicitly encoded in the parameters. The estimated parameters can then directly employed during inference, without any reference to the training set. In some sense, the task-relevant information in the training samples is completely encoded into the parameters during training and then information is decoded appropriately during inference solely from the estimated parameters. The training data is not needed for inference.

In contrast, recall kernel based methods: their inference is based on $\hat{f}(x) = \sum_{i=1}^m \alpha_i \kappa(x_i, x) y_i$. Such models where the training data needs to be stored explicitly and is required for any relevant inference, after training are known as **non-parametric** or **exemplar based** models. Because of this explicit memorization of training data:

1. They tend behave like overparametrized models in the sense that their training data loss is zero¹.
2. Like in overparametrized models, regularization is necessary. In other words, assumptions are needed for generalization to new inputs.

¹The exception being kernel based models, which can be operated with heavy regularization so that training loss itself is non-zero, emulating underparametrized models.

34.1 Nearest Neighbour Classification

Perhaps the simplest and a classical example of non-parametric model for classification is the **nearest neighbour** model. Like how the linear model is well-defined given a feature map over the input space, a **Nearest Neighbour (NN)** model is defined given a (valid) distance² function over the input space: $d: \mathcal{X} \times \mathcal{X} \mapsto \mathbb{R}_+$.

Each candidate function/classifier in this model is then described using a finite set, $S \subset \mathcal{X} \times \mathcal{Y}$. Let the i^{th} pair in S be (x'_i, y'_i) . Given the set S , the corresponding function, $f_S: \mathcal{X} \mapsto \mathcal{Y}$, is defined by: $f_S(x) = y'_{i^*(x)}$, where $i^*(x) \in \arg \min_{i=1, \dots, |S|} d(x, x'_i)$. In other words, the function's value at any input seen in S is the corresponding label and at any unseen input is that of the nearest input, $x'_{i^*(x)}$ (ties broken arbitrarily). See figure 16.1 in Murphy [2022] for an illustration of a function in this model in 2-d space. Formally, **1-NN model** is:

$$(34.1) \quad {}^1\mathcal{N}_d = \left\{ f \mid \exists (x'_i, y'_i) \in \mathcal{X} \times \mathcal{Y}, i=1, \dots, s, s \in \mathbb{N}, \exists f(x'_i) = y'_i, \forall i=1, \dots, s, f(x) = y'_{i^*(x)}, i^*(x) \in \arg \min_{i=1, \dots, |S|} d(x, x'_i) \right\}$$

Different sets, S , induce different functions. During training, the set, S , is simply memorized as the training set $S = \mathcal{D}$. Hence, this model is indeed a non-parametric model.

Theorem 19.3 in Shalev-Shwartz and Ben-David [2014] provides a formal justification of why/when this 1-NN classification generalizes well. Note that the theorem is a consequence of the key assumption that the label posterior given an input is same (close) as the label posterior given it's nearest neighbour (asymptotically as $m \rightarrow \infty$).

In other words, the true posterior, $p^*(y/x)$, at an $x \in \mathcal{X}$ and the true posterior at it's nearest neighbour, $p^*(y/x_{i^*(x)})$, will essentially be the same, asymptotically. Hence the label of the nearest neighbour is nothing but a sample from $p^*(y/x)$. In 1-NN, this label is simply copied as the label of x .

In the lecture, we derived **lemma 19.1 in Shalev-Shwartz and Ben-David [2014]** and postponed the other details of theorem 19.3 to advanced courses. Note that the result of this theorem shows that even if $m \rightarrow \infty$, the risk with NN can be twice of the optimal Bayes risk. In other words, NN classifier has non-zero model error. This can be corrected by considering $k > 1$ nearest neighbours. Then, under appropriate assumptions, the labels of these neighbours will again be samples from $p^*(y/x)$. Labeling x using all these k labels (samples) seems an

²Refer https://en.wikipedia.org/wiki/Metric_space for properties of distance like non-negativity, symmetry and triangle inequality. <https://scikit-learn.org/stable/modules/generated/sklearn.metrics.DistanceMetric.html?highlight=dista> is a good collection of popular distance metrics.

improved strategy. This leads to the idea of k -NN classification:

34.2 k -NN Classification

Each candidate function in this model is described using a finite set, $S \subset \mathcal{X} \times \mathcal{Y}$. Let the i^{th} pair in S be (x'_i, y'_i) . Given the set S , the corresponding function, $f_S : \mathcal{X} \mapsto \mathcal{Y}$, is defined by: $f_S(x)$ is the majority label among those of the k nearest neighbours of x . We call this model as the **k -NN classification model** and denote it by ${}^k\mathcal{N}_d$. Again, entire training data is memorized as S . Majority label is appropriate because it is nothing but the mode of the empirical likelihood formed with labels of the k neighbours (which are samples from $p^*(y/x)$ as hinted above). And posterior mode is the Bayes optimal with the 0-1 loss.

Theorem 19.5 in Shalev-Shwartz and Ben-David [2014] shows that as $k \rightarrow \infty, m \rightarrow \infty, \frac{k}{m^{d+1}} \rightarrow 0$, k NN recovers the (true) Bayes optimal! (zero generalization error).

34.3 k -NN Regression

Exactly same as the above except that the predicted label of x is the average of the labels of the k neighbours. Average label is appropriate because it is nothing but the expectation/mean of the empirical likelihood formed with labels of the k neighbours (which are samples from $p^*(y/x)$ as hinted above). And posterior mean is the Bayes optimal with the squared loss.

Now, both the theorems 19.3, 19.5 essentially show that the nearest neighbour rules are cursed with dimensionality. i.e., in high dimensions, there is barely any improved generalization with increased m as error decays as $O(\frac{1}{m^{\frac{1}{n+1}}})$, where m is number of samples and n is the dimensionality of the data. Also, computational cost of NN queries becomes prohibitive in high dimensions³. Hence, typically, NN methods are applied after so-called dimensionality reduction.

³Interested students may refer <https://dl.acm.org/doi/pdf/10.1145/301250.301330> related line of work.

Lecture 35

Non-parametric Models: KDE

While nearest neighbour methods are examples of non-parametric non-probabilistic models, one can have analogous examples of non-parametric probabilistic models. More specifically, we will discuss non-parametric likelihood estimation in this lecture.

If Y is discrete, a non-parametric model for likelihood is familiar to all of us: distribution of empirical frequencies i.e., $\hat{p}(y)$ = fraction of times y is observed in training set. Likewise, non-parametric conditional likelihood models for the discrete case are immediate: using k -nearest neighbour ideas, $\hat{p}(y|x)$ is the fraction of neighbours of x having label as y .

However, if Y is continuous, the same estimate, i.e., the empirical likelihood (uniform over the samples), though a valid non-parametric model, is not useful in ML. The issue with this empirical likelihood is that it assigns zero likelihood to points not appearing in the training set. So it is not a useful model in ML applications.

This issue can be alleviated by employing a smoothened version of the empirical likelihood: instead of an observation y_i leading to a dirac impulse at y_i , let us have a smoothened impulse at y_i , for e.g., Gaussian centered at y_i . While average of the diracs is the empirical likelihood, the average of these Gaussians is the so-called **KDE (Kernel Density Estimate)**: $\hat{p}(y) = \frac{1}{m} \sum_{i=1}^m \kappa(y - y_i)$, where $\kappa(y) = \frac{e^{-\frac{1}{2}\|y\|^2}}{(2\pi)^{n/2}}$. This is visualized in fig 16.9 in Murphy [2022]. Infact, one can take κ as any so-called **density kernel**:

1. $\kappa(y) \geq 0 \ \forall \ y \in \mathcal{Y}$.
2. $\int_{\mathcal{Y}} \kappa(y) = 1$.

$$3. \kappa(y) = \kappa(-y) \forall y \in \mathcal{Y}.$$

As a consequence of symmetry, $\int_{\mathcal{Y}} y \kappa(y) dy = 0$ and $\int_{\mathcal{Y}} y \kappa(y - \mu) dy = \mu$. In fact, the smoothness can be controlled by introducing a hyperparameter: $\kappa_h(y) \equiv \frac{1}{h} \kappa(\frac{y}{h})$. As $h \rightarrow 0$, any kernel will behave like dirac and KDE becomes the empirical likelihood. As h increases, the kernel is smoother.

One can show that as $m \rightarrow \infty$, KDE converges to the underlying likelihood e.g., theorem 2 in <https://proceedings.mlr.press/v70/jiang17b/jiang17b.pdf>. While this convergence is wonderful, the sad news is (again) curse of dimensionality: density estimation error is something like $O(\frac{1}{m^{n+2}})$, where m is number of samples and n is the dimensionality of the data. However, this is not because KDE is inefficient, it is because density estimation itself is a hard problem <https://proceedings.mlr.press/v54/mcdonald17a/mcdonald17a.pdf>. In other words, no estimation can be better than KDE!

35.1 KDE based generative models

Consider the supervised learning setup. If we wish for a generative model using KDE ideas, then we need to design a density kernel over $\mathcal{X} \times \mathcal{Y}$. While this can be done from first principles, one can construct a density kernel over $\mathcal{X} \times \mathcal{Y}$ using a kernel, κ_1 , over $\mathcal{X}$ and a kernel, κ_2 , over $\mathcal{Y}$. It is easy to see that $\kappa(x, y) \equiv \kappa_1(x)\kappa_2(y)$ is a valid density kernel (i.e., κ satisfies the three properties listed earlier). So the KDE for the underlying turns out to be:

$$\hat{p}(x, y) = \frac{1}{m} \sum_{i=1}^m \kappa_1(x - x_i) \kappa_2(y - y_i).$$

From this, the marginal $\hat{p}(x) = \int_{\mathcal{Y}} \frac{1}{m} \sum_{i=1}^m \kappa_1(x - x_i) \kappa_2(y - y_i) dy = \frac{1}{m} \sum_{i=1}^m \kappa_1(x - x_i) \int_{\mathcal{Y}} \kappa_2(y - y_i) dy = \frac{1}{m} \sum_{i=1}^m \kappa_1(x - x_i)$. Likewise, $\hat{p}(y) = \frac{1}{m} \sum_{i=1}^m \kappa_2(y - y_i)$. Interestingly, the KDE of marginals is nothing but the KDE with the respective kernels. More importantly, it is easy to see that $\hat{p}(x, y) \neq \hat{p}(x)\hat{p}(y)$ and hence, the KDE for the joint is non-trivial.

Infact, $\hat{p}(y|x) = \frac{\hat{p}(x, y)}{\hat{p}(x)} = \frac{\frac{1}{m} \sum_{i=1}^m \kappa_1(x - x_i) \kappa_2(y - y_i)}{\frac{1}{m} \sum_{i=1}^m \kappa_1(x - x_i)}$. So the corresponding Bayes optimal: mean of posterior is: $\int_{\mathcal{Y}} y \hat{p}(y|x) dy = \frac{\sum_{i=1}^m \kappa_1(x - x_i) \int_{\mathcal{Y}} y \kappa_2(y - y_i) dy}{\frac{1}{m} \sum_{i=1}^m \kappa_1(x - x_i)} = \sum_{i=1}^m \bar{\kappa}_{\mathcal{D}}(x - x_i) y_i$, where $\bar{\kappa}_{\mathcal{D}}(x - x_i) = \frac{\kappa(x - x_i)}{\sum_{j=1}^m \kappa(x - x_j)}$. This KDE regressor is popularly known as the [Nadaraya-Watson](#) estimator. In other words, the regressor simply interpolates the training labels with weights according to similarity (assuming kernel is Gaus-

sian type) with training inputs. Contrast this with kernel regression ???. While KDE is restricted to interpolations, kernel regression allows linear combinations.

Attention, which drives modern LLMs, could be understood as KDE regression. Interested students may refer section 15.4.2 in Murphy [2022]. This lecture is based on [section 16.3 in Murphy \[2022\]](#).

Lecture 36

Generative AI

The problem of [Generative AI \(GenAI\)](#) is defined as the following:

Definition 36.0.1. *Given m samples from an unknown underlying likelihood, p , generate more samples from p .*

Note that, though this problem is related to that of likelihood estimation, it is not the same as likelihood estimation. The definition does not insist on estimating the underlying likelihood. However, if one indeed estimates the underlying likelihood, then the GenAI problem reduces to that of constructing a sampler for the estimate, i.e., code that can simulate sampling from the estimated likelihood.

Approaches which completely avoid estimating the likelihood or its quintessential equivalent are sometimes known as [implicit generative](#) methods. These are the “direct” approaches and will be studied in advanced courses like CS6090. In this course, we will take the “indirect” approach of likelihood estimation and then sampling from the estimate. This is because, likelihood estimation was already studied earlier in the course. Such methods are sometimes known as [explicit generative](#) methods.

36.1 Energy based Models

Let's recall the general likelihood models, which we called as the models from the exponential family: $p_w(x) = \frac{e^{w^\top \phi(x)}}{Z(w)}$. These likelihood models are essentially normalized linear models. Analogously, one can normalize deep learning models leading to $p_\theta = \frac{e^{-\mathcal{E}_\theta(x)}}{Z(\theta)}$, where $\mathcal{E}_\theta$ is a neural network induced function parametrized by θ . Because likelihood is higher for lower $\mathcal{E}_\theta$, it is usually referred to as energy

function and the corresponding likelihood models are known as [Energy Based Models \(EBMs\)](#). EBMs hence can be understood as reparametrized exponential family models.

Analogous to (23.6), the ERM+SGD update equation reads as:

$$(36.1) \quad \theta^{(k+1)} \equiv \theta^{(k)} - s_k \left(\nabla \mathcal{E}_\theta(x_{i_k}) - \mathbb{E}_{X \sim p_{\theta^{(k)}}} [\nabla \mathcal{E}_\theta(X)] \right)_{\theta=\theta^{(k)}} .$$

From this update equation, it is clear that a sampler is not only desirable for inference in GenAI, but also is required for training to estimate $\mathbb{E}_{X \sim p_{\theta^{(k)}}} [\nabla \mathcal{E}_\theta(X)]$. Moreover, we desire a sampler that works even if the normalization constant is not known. Please refer [section 24.1 and eqns \(24.7\)-\(24.16\) in Murphy \[2023\]](#) for more details on EBMs.

36.2 Langevin Sampling

One key statistic that does not require the knowledge of the normalization constant is the mode: $\arg \max_{x \in \mathcal{X}} p_\theta(x) = \arg \min_{x \in \mathcal{X}} \mathcal{E}_\theta(x)$. We know that SGD updates for this problem will converge to the mode, under appropriate structural conditions on the energy function: $X^{(k+1)} = X^{(k)} - s_k \nabla_x \mathcal{E}_\theta(x)|_{x=X^{(k)}} + s_k N_k$, where $N_k \sim \mathcal{N}(0, I)$. Here, ∇_x denotes the gradient wrt. x variable and not θ . The function $\nabla_x \log(p(x))$ is popularly known as the [Stein score](#)¹ of p .

Our goal however is to not to reach the mode, but to converge to samples from p_θ i.e., one could hope to modify the dynamics of SGD such that after (large) k , we have: $X^{(k)} \sim p_\theta$, irrespective of $X^{(0)}$'s likelihood. It will be convenient to choose $X^{(0)} \sim \mathcal{N}(0, I)$ or some likelihood for which a pseudo-random-number generator already exists, so that when we initialize at a random sample, after k iterations we would have a sample from p_θ .

Due to some complicated analysis of SDEs (Stochastic Differential Equations), which are usually covered in courses like CS6090, the appropriate update rule turns out to be:

$$(36.2) \quad X^{(k+1)} = X^{(k)} - s \nabla_x \mathcal{E}_\theta(x)|_{x=X^{(k)}} + \sqrt{2s} N_k$$

Sampling using this update rule is known as [Langevin sampling](#) Roberts and Tweedie [1996]. Figure 25.5 in Murphy [2023] presents an illustration of this sampling. Even though difficult to derive, it is straight-forward to code/employ this in ML applications. To re-iterate, the procedure for sampling² is:

¹ $\nabla_\theta \log(p_\theta(x))$ is popularly known as the Fisher score of p .

²This is roughly how state-of-the-art so-called diffusion models Ho et al. [2020] work.

1. Initialize at random $x^{(0)}$. (If the likelihood is of images/pictures. Start with image with random pixel values i.e., pure noisy image).
2. At every iteration, sample N_k and perform the update (36.2) to obtain a more purer sample (more denoised image). Performing the update requires computation of the Stein score, which can be done efficiently using backprop as $\mathcal{E}_\theta$ is a neural net induced function.
3. After enough iterations, the final iterate is actually a sample from p_θ and hence a purified sample (fully denoised image).

In order to get a feel of the convergence result justifying Langevin sampling, we took the special case of Gaussian and we repeated the derivation in [section 4 in https://fa.bianp.net/blog/2023/ulaq/](https://fa.bianp.net/blog/2023/ulaq/). If $p_\theta(x) \propto e^{-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)}$ and s is (small enough) step-size, the result of this derivation is:

$$\begin{aligned} \mathbb{E}[X^{(k)}] &= \mu + \underbrace{\left(I - s\Sigma^{-1}\right)^k \left(\mathbb{E}[X^{(0)}] - \mu\right)}_{\text{Error decays exponentially to zero}} \\ \text{cov}(X^{(k)}) &\approx \Sigma + \underbrace{\frac{s}{2}I}_{\text{Constant error linear in } s} + \underbrace{\left(I - s\Sigma^{-1}\right)^{2k} \left(\text{cov}(X^{(0)}) - \Sigma - \frac{s}{2}I\right)}_{\text{Error term that decays exponentially to zero}} \end{aligned}$$

This basically proves that mean of the random process (36.2) converges exponentially fast to the target mean (provided $s < L_{min}$, the smallest eigenvalue of target covariance, Σ). However, the covariance does not converge to the target covariance, but still exponentially rapidly converges to $\Sigma + \frac{s}{2}I$, which is a close approximation. Infact, as $s \rightarrow 0$, we indeed have convergence. This case with infinitesimal steps is also known as [Langevin diffusion](#).

Please refer [section 24.2.1 in Murphy \[2023\]](#) for more details on Langevin sampling. Interested students may read entire section 24.2.

36.3 Conditional Sampling

Given a training set, $\mathcal{D} = \{(x_1, y_1), \dots, (x_m, y_m)\}$, which are iid samples from $p(x, y)$, an interesting GenAI problem is generate samples from $p(y|x)$. For example, given a caption, generate an image illustrating the caption. Then, the same methodology EBM+Langevin described above can be employed:

1. Consider EBM: $p_\theta(y|x) \propto e^{-\mathcal{E}_\theta(x,y)}$.

2. Using $\mathcal{D}$ and MLE, estimate $\hat{\theta}$. (this will involve invoking Langevin steps)
3. Using Stein score $\nabla_y \log (p_{\hat{\theta}} (y|x))$, perform Langevin steps to obtain samples.

Lecture 37

Online Learning

Now we wish to mimic the following kind of learning that happens in humans: “Mother shows baby a pic and asks what is depicted in it. The baby responds. Then, the mother provides the answer. And, this continues by the mother showing another pic, and so on...”.

Though there are multiple aspects that distinguish this setting from the familiar batch learning, the most crucial distinction is that the learner must respond before the supervision is provided (for that input). i.e., based on the supervision provided till now, learner has to respond (at every round). This is the so-called “online” aspect of this learning. A consequence of this is that the “training set error” is a better representation of the learning than in case of batch learning i.e., no. mistakes in a given number of rounds is already a reflection of learning. This is because memorization is infeasible.

Consider a supervised learning problem with a given model (inductive bias) $\mathcal{F}$ parametrized by w , and a loss l . Syntactically speaking, an online supervised learning algorithm, $\mathcal{A}$, is any strategy¹ for picking w_t at round $t \in \mathbb{N}$ under the described setting that y_t , as well as any information regarding future rounds are not available. The online risk, after T rounds, can be defined as: $\frac{1}{T} \sum_{t=1}^T l(f_{w_t}(x_t), y_t)$. Lower this online risk, better is the online algorithm.

Now, in supervised learning the goal is find the true Bayes optimal f^* . However, it is not clear how one would access an online learning algorithm’s potential to reach this goal. This is because, the output of an online algorithm is a sequence of parameters $w_1, \dots, w_T, \dots$. Do we insist that $\{f_{w_T}\} \rightarrow f^*$? This is not meaningful especially when there are multiple Bayes optimal functions. Do we insist $\{R[f_{w_T}]\} \rightarrow R[f^*]$? But what if this happens and yet the online risk is high

¹In contrast, a batch supervised learning algorithm needs to only pick one w based on the given training set.

for any reasonable T ? i.e., the learner “learns” only after $T > T_0$, where T_0 is unreasonably high². This is not useful. So it is clear one needs to involve online risk and one needs to involve the Bayes optimal risk.

Accordingly, we may wish to look at the difference between online risk and the Bayes optimal risk: $\frac{1}{T} \sum_{t=1}^T l(f_{w_t}(x_t), y_t) - R[f^*]$. We may not want to call this generalization error or as *any* error for that matter, as this quantity may be negative at times. It is still meaningful because of the following closer look:

$$\begin{aligned}
 (37.1) \quad & \frac{1}{T} \sum_{t=1}^T l(f_{w_t}(x_t), y_t) - R[f^*] = \\
 & \frac{1}{T} \left(\underbrace{\sum_{t=1}^T l(f_{w_t}(x_t), y_t) - \min_w \sum_{t=1}^T l(f_w(x_t), y_t)}_{\equiv \text{Regret}} \right) \\
 & + \underbrace{\min_w \frac{1}{T} \sum_{t=1}^T l(f_w(x_t), y_t) - \min_w \mathbb{E}[l(f_w(X), Y)]}_{\text{Estimation Error}} \\
 & + \underbrace{\min_w \mathbb{E}[l(f_w(X), Y)] - \min_f \mathbb{E}[l(f(X), Y)]}_{\text{model error}}
 \end{aligned}$$

While model and estimation error and their analysis is familiar, this decomposition gives us a new term: the [regret of an online algorithm](#)³. While regret can be negative, it is usually positive because an online algorithm does not have access to the supervision at any round. In other words, an online algorithm needs to minimize (the online risk) while not having access to this objective itself! However, the terms in this objective, which is a sum, that correspond to previous rounds are still available – enabling hope. Since estimation error and model error are familiar terms. We define the goal of online supervised learning as that of [minimizing/controlling regret](#).

Another way of understanding this is: both batch supervised and online supervised learning are risk minimization problems. However, in batch learning the challenge is that the (true) risk objective, the expected loss is not available; yet iid samples from the underlying likelihood are available — enabling hope. In online learning, the (online) risk objective is not available; yet few terms in the sum are available. In summary, the goal of both learning problems is to

²Like today’s LLMs ;)

³Regret is a better word than error because this can be negative, in which case, we can conclude there is no regret.

“optimize under uncertainty”. They only differ in the way uncertainty is present. While in batch learning, uncertainty is stochastic; In online learning uncertainty is due to the un-availability of the terms corresponding to the future rounds. Also note that, the stochastic assumption that the training data are iid samples from some underlying likelihood does not help simplifying the regret minimization problem! This is because the two terms in regret involve exactly the same set of points (x_i, y_i) . In other words, stochastic assumptions that are fundamental for supervised learning only help control the estimation error. They do not matter for controlling the regret. Because of this fundamental decomposition, people often study the problem of controlling regret as a separate learning problem, popularly dubbed as [Online Learning](#).

Infact, this view of controlling the regret, broadens our scope of mimicing human learning too: consider guessing games that we play: a person will think of something (chooses a $\hat{w}$). She will not reveal $\hat{w}$ but reveals the class $\mathcal{F}$ before beginning the guessing game. Then she asks the learner (opponent) to guess. The learner makes a guess (picks w_1) and then she provides feedback about the guess (information regarding $l(f_{w_1}(x_t), y_t)$). This game continues. The learner is better if the overall feedback is lower/weaker (regret is low).

Formally, we say an online learning algorithm is successful if it's regret grows sublinearly. For e.g., $O(\sqrt{T})$ or $O(\log(T))$ etc. In special case of 0 – 1 loss, this simply means the number of mistakes is $o(T)$, which makes sense. Additionally, along with the usual statistical assumptions, this means after large enough T , we have: $\frac{1}{T} \sum_{t=1}^T l(f_{w_t}(x_t), y_t) - R[f^*] \leq \epsilon$, which basically means the online algorithm's online risk is atmost the Bayes optimal risk. This is then meaningful in the context of supervised learning too.

Because statistical assumptions cannot help controlling regret, it is clear that modelling (inductive bias) is the sole enabler of learning in this setting. Recall the guess game example. There cannot be any guess if the organizer does not announce $\mathcal{F}$. In other words, we though future terms are completely unavailable, if nothing is known about them, then also controlling regret may not be possible. Let's us understand this more deeply with an example.

37.1 Inductive Bias $\implies$ Online Learning

Recall the example from lecture 2: Let $\mathcal{F}$ be the set of all possible 2^{2^n} boolean-valued functions over n -bit inputs. The goal is to find $f^* \in \mathcal{F}$ via online learning:

1. x_1 (a n -boolean input) is shown - then you need to guess $f^*(x_1)$ - then you either make a mistake or not - in either case $f^*(x_1)$ is then revealed.

2. x_2 (another n-boolean input) is shown - then you need to guess $f^*(x_2)$ - then you either make a mistake or not - in either case $f^*(x_2)$ is then revealed.
3. so on ...

In the worst case⁴, at every round the guess could be wrong. But if the learner/guesser is not a *murkha*⁵, i.e., if he eliminates half of the remaining choices of functions that are inconsistent with the feedback (label), then after 2^n rounds the learner will make zero mistakes. But this cannot be called as learning as then the inputs must repeat. At best, this is memorization. But this is fine because there is no inductive bias as $\mathcal{F}$ is all possible functions. So, if nothing is known about the future, then controlling regret is not possible. So controlling regret may only be possible via an inductive bias i.e., assuming something about $\mathcal{F}$.

Now let us just put a restriction i.e., $|\mathcal{F}| < 2^{2^n}$. Then, a non-*murkha*, who simply eliminates remaining choices for the function that are inconsistent with the observation so far⁶, in the worst case, would still end up making mistakes all the time until number of rounds are $|\mathcal{F}|$. So a non-*murkha* cannot be called an efficient learner, essentially because he is merely remembering the past (memorizing).

But now consider a *Kuśāgrabuddhi* (efficient learner), who not only is consistent with the observations in the past⁷, but also guards herself from the uncertainties about the future. In other words, a *Kuśāgrabuddhi* will “plan” for the future. In this setting, planning for the future means: given current input x_t , the choice for f must be such that if $f(x_t) \neq y_t$ (worst case), then the learner still improves a lot (infact, improves the most). More explanation is below.

Given x_t , the set of functions in $\mathcal{F}$ that are consistent with the past i.e., $(x_1, y_1), \dots, (x_{t-1}, y_{t-1})$, can be partitioned into two equivalent classes: one those that evaluate to 0 at x_t and those that evaluate to 1. So there is still a choice beyond what past informs. If the learner chooses any function from the former partition, i.e., predicts 0, then in the worst case, the learner can eliminate the latter partition and vice-versa. Ofcourse, if the learner chooses a function from the bigger partition, then in the worst case the bigger partition is eliminated, leading to maximum improvement. Also, in the best case, the prediction is anyway correct, so it does not count as a mistake. And, even here we will be eliminating the smaller partition. Thus, the regret is better controlled if the bigger partition

⁴Note that we are now worried about worst case rather than average case because statistical assumptions have no place.

⁵non-murkha is formally called as consistent.

⁶One who memorizes past feedback and does nothing else

⁷In batch learning, being consistent with the past was sufficient because statistical assumptions guaranteed that the future is similar (on an average). Ofcourse, the smartness there is in employing the *right* model/regularizer.

is eliminated when a mistake is made. It is easy to see that the regret with this algorithm (regret of *Kuśāgrabuddhi*) is $\begin{cases} t & t \leq \log(|\mathcal{F}|) \\ \log(|\mathcal{F}|) & \text{otherwise} \end{cases}$. This is orders of magnitude better than $|\mathcal{F}|$ and hence qualified to be referred to as efficient learner.

Here is the summary:

1. While statistical assumptions, together with model-restrictions/regularization help control estimation error, statistical assumptions can play no role in controlling regret.
2. While consistency with past evidence (a.k.a. ERM) was alone enough⁸ for controlling estimation error; it is necessary, yet insufficient for online learning. This is because the uncertainty about future is not stochastic. Hence goodness in past may not imply goodness in future.
3. Additionally, guarding against the uncertainties of the future (planning for future?) is necessary for online learning (and may not be necessary for batch learning, thanks to the stochastic assumptions).
4. Finally, this key aspect of guarding against uncertainties may not be possible for some nasty function classes, $\mathcal{F}$. For e.g., if $|\mathcal{F}| < 2^{2^n}$, then possible, else (seems) impossible. We will see in next lecture that the class $\mathcal{F}$ of convex (and Lipschitz) functions is again nice, in the sense that regret can be controlled efficiently.

⁸Atleast in case of many practical models.

Lecture 38

Online Convex Optimization

The regret minimization problem infact has a more general form that enables applications to other fields: economics, operations research, game theory etc. This is known as an [online optimization](#) problem¹:

Definition 38.0.1. *Online optimization* is a formalization for sequential decision-making under uncertainty. In particular, at time step t , the goal is to pick a $w_t \in \mathcal{W}$. After the decision w_t is made, feedback reflecting the quality of this decision is provided. More specifically, some information^a about a loss function $f_t : \mathcal{W} \mapsto \mathbb{R}$ is provided. In this setting, the goal of online optimization is controlling regret i.e., choose $w_t \in \mathcal{W}$ sequentially (as per above protocol) such that:

$$(38.1) \quad \sum_{t=1}^T f_t(w_t) - \min_{w \in \mathcal{W}} \sum_{t=1}^T f_t(w).$$

is controlled. Ideally, growing sublinearly i.e., $o(T)$.

^aZero-order or first order or full information

Note that this is same as online supervised learning from the previous lecture if we consider $f_t(w_t) \rightarrow l(f_{w_t}(x_t), y_t)$.

Online optimization where only values of $f_t(w_t)$ (zero-order information) are provided as feedback are known as [Bandit](#) problems. The problems where only (sub)gradients of $\nabla f_t(w_t)$ are provided are known as [first-order online optimization](#) problems. In full information online optimization, entire function f_t is made

¹An easy starter book on this subject for ML folks is <http://sbubeck.com/BubeckLectureNotes.pdf>.

available.

With this terminology, while ERM in batch supervised learning incurs additional term: the optimization error; online supervised learning instead incurs the regret term. So estimation and modelling is common to both batch and online supervised learning, while the optimization protocol alone changes. Further, problems such as in online supervised learning where f_t are iid (stochastic) are known as **online stochastic optimization** problems. Here, as clarified earlier, regret is not meaningful (i.e., statistical assumptions don't matter) so one considers regret together with estimation error in (37.1) i.e., $\sum_{t=1}^T l(f_{w_t}(x_t), y_t) - T \min_w \mathbb{E}[l(f_w(X), Y)]$. This is itself called as regret in online stochastic optimization problems. And, since f_t are random, w_t are also random, and one is usually interested in expected regret. In summary, **batch supervised learning is nothing but a stochastic optimization problem; whereas online supervised learning is nothing but a full information online stochastic optimization problem** (full information because loss function is known and the training samples are shown sequentially).

As mentioned earlier, online optimization may be possible (i.e., sublinear growth of regret) only for special functions classes. One such special class happens to be the set of all convex functions that are Lipschitz. This actually follows from the usual convergence proofs of SGD.

Consider first order online optimization where functions in $\mathcal{F}$ are convex and Lipschitz. The information available is first order so $\nabla f_t(w_t)$ is only available. Now consider running SGD on the problem $\min_w \sum_{t=1}^T f_t(w)$ in the batch setting. This is a stochastic optimization problem where underlying likelihood is uniform over $f_1, \dots, f_T$. Hence SGD iterate would look like: $w_{t+1} = w_t - s \nabla f_t(w_t)$. We already know this SGD converges in an average sense (batch sense) from 20.1.

Interestingly, this same algorithm, can serve as an online algorithm as for deciding w_{t+1} , we are only using previous gradients: $\nabla f_t(w_t), \dots, \nabla f_1(w_1)$. However, what we have to show that this algorithm achieves sublinearly regret and hence is efficient for online optimization. In online optimization terminology, this same SGD is called as **online gradient descent (OGD)**.

38.1 Regret of Online Gradient Descent (OGD)

We just repeated theorem 21.15 in Shalev-Shwartz and Ben-David [2014] and its proof. Let $w^* \equiv \arg \min_w \sum_{t=1}^T f_t(w)$:

$$\begin{aligned}
& \|w_{t+1} - w^*\|^2 - \|w_t - w^*\|^2 \\
&= \|(w_t - w^*) - s \nabla f_t(w_t)\|^2 - \|w_t - w^*\|^2 \\
&= s^2 \|\nabla f_t(w_t)\|^2 - 2s \nabla f_t(w_t)^\top (w_t - w^*) \\
&\leq s^2 L^2 - 2s \nabla f_t(w_t)^\top (w_t - w^*) \quad \because f_t \text{ is } L\text{-Lipschitz} \\
&\leq s^2 L^2 - 2s (f_t(w_t) - f_t(w^*)) \quad \because f_t \text{ is convex}
\end{aligned}$$

The Lipschitz assumption is not restrictive, as any convex function is locally Lipschitz. This is infact the same assumption as bounded noise in SGD proof from lecture 19. By telescoping sum from this derivation, we obtain: $\sum_{t=1}^T (f_t(w_t) - f_t(w^*)) \leq \frac{1}{2} \left(sTL^2 + \frac{\delta_1^2 - \delta_{T+1}^2}{s} \right)$, where $\delta_t \equiv \|w_t - w^*\|$. Ignoring $-\delta_{T+1}$ in the upper bound, we have regret of OGD upper bounded by $\frac{1}{2} \left(sTL^2 + \frac{\delta_1^2}{s} \right)$ for any $s > 0$. Let us choose the best step-size $s^* = \frac{\delta_1}{L\sqrt{T}}$ that minimizes this upper bound. This minimal upperbound is $\delta_1 L \sqrt{T}$. This shows that OGD achieves sublinear regret.

38.2 Online to Batch conversion (OGD $\rightarrow$ SGD)

If all f_t are the same, say f , then, due to Jensen's inequality: $f(\frac{1}{T} \sum_{t=1}^T w_t) - f(w^*) \leq \frac{1}{T} \sum_{t=1}^T (f(w_t) - f(w^*)) \leq \frac{\delta_1 L \sqrt{T}}{T} = \frac{\delta_1 L}{\sqrt{T}}$. If f is true risk in batch supervised learning, then this provides convergence of SGD, and if f is empirical risk, then this provides convergence of ERM+SGD. This is the proof for the first row in table 20.1.

In other words, batch learning is an “easier” problem than online learning. Whenever online learning is possible over a function-class (model), then batch learning is possible (thanks to online to batch conversion noted above). However, the converse is not true².

²Think about linear classifiers in 1-d for a counterexample.

Lecture 39

Averaging (Ensembling)

One key result in statistics that we used earlier too is the law of large numbers or central limit theorem in the sense that as number of samples increases, the sample mean becomes a better estimate of the true mean. For example, using something like law of large numbers was how we controlled the estimation error.

Now consider the application of the same idea in a different context. Suppose $\hat{f}_1, \dots, \hat{f}_k$ are k different (iid) estimates for the same Bayes optimal. For e.g., they are k predictors obtained by training the same model, with the same hyperparameters, but with k different datasets, all iid from the same underlying likelihood. Now consider a new averaged estimate: $\hat{f} = \frac{1}{k} \sum_{i=1}^k \hat{f}_i$. Is the averaged estimator better than the individual ones?

We have a simple answer in case of squared-loss (can be extended to other losses). Let $\hat{f}$ be trained using a dataset $\mathcal{D}$. Recall from chapter 9 that the generalization error, $R[\hat{f}] - R[f^*]$, with $\hat{f}$ can be simplified in the square loss case¹

¹Recall that Bayes optimal for square loss is $f^*(x) = \mathbb{E}[Y|x]$.

as:

$$\begin{aligned}
&= \mathbb{E} \left[\left(\hat{f}(X) - Y \right)^2 \right] - \mathbb{E} \left[\left(f^*(X) - Y \right)^2 \right] \\
&= \mathbb{E} \left[\left(\hat{f}(X) - f^*(X) + f^*(X) - Y \right)^2 \right] - \mathbb{E} \left[\left(f^*(X) - Y \right)^2 \right] \\
&= \mathbb{E} \left[\left(\hat{f}(X) - f^*(X) \right)^2 \right] + \mathbb{E} \left[\left(f^*(X) - Y \right)^2 \right] + 2\mathbb{E} \left[\left(\hat{f}(X) - f^*(X) \right) \left(f^*(X) - Y \right) \right] - \mathbb{E} \left[\left(f^*(X) - Y \right)^2 \right] \\
&= \mathbb{E} \left[\left(\hat{f}(X) - f^*(X) \right)^2 \right] + 2\mathbb{E} \left[\left(\hat{f}(X) - f^*(X) \right) \left(f^*(X) - Y \right) \right] \\
&= \mathbb{E} \left[\left(\hat{f}(X) - f^*(X) \right)^2 \right] + 2\mathbb{E} \left[\left(\hat{f}(X) - f^*(X) \right) \left(f^*(X) - \mathbb{E}[Y|X] \right) \right] \quad \because \text{Total expectation rule} \\
&= \mathbb{E} \left[\left(\hat{f}(X) - f^*(X) \right)^2 \right].
\end{aligned}$$

Now, $\hat{f}$ is random as $\mathcal{D}$ is random. We denote by $\mathbb{E}_{\mathcal{D}} [\hat{f}(x)]$ the expected value of $\hat{f}(x)$. So the expected generalization error decomposes as:

$$\begin{aligned}
&\mathbb{E}_{\mathcal{D}} \left[\mathbb{E} \left[\left(\hat{f}(X) - f^*(X) \right)^2 \right] \right] \\
&= \mathbb{E}_{\mathcal{D}} \left[\mathbb{E} \left[\left(\hat{f}(X) - \mathbb{E}_{\mathcal{D}} [\hat{f}(X)] + \mathbb{E}_{\mathcal{D}} [\hat{f}(X)] - f^*(X) \right)^2 \right] \right] \\
&= \mathbb{E}_{\mathcal{D}} \left[\mathbb{E} \left[\left(\hat{f}(X) - \mathbb{E}_{\mathcal{D}} [\hat{f}(X)] \right)^2 + \left(\mathbb{E}_{\mathcal{D}} [\hat{f}(X)] - f^*(X) \right)^2 \right] \right] \quad \because \text{cross term is zero after taking expect.} \\
&= \mathbb{E} \left[\underbrace{\text{var}_{\mathcal{D}} (\hat{f}(X))}_{\text{variance of } \hat{f}} \right] + \mathbb{E} \left[\underbrace{\left(\mathbb{E}_{\mathcal{D}} [\hat{f}(X)] - f^*(X) \right)^2}_{\text{bias of } \hat{f}} \right].
\end{aligned}$$

Hence, the expected generalization error² of the estimator decomposes as two terms: expected bias of the estimator and the expected variance of the estimator. Typically, lowering one term will elevate the other and vice-versa. This is also known as the **Bias-Variance Trade-off** in statistics terminology. Typically, in the ML context, in accordance with the discussion of 11, the larger the model (less restrictive), the lower the bias and the higher the variance, and vice-versa. Hence, sometimes the trade-off between model error and estimation error is also referred to as bias-variance trade-off though there is no 1-1 correspondence between the terms.

Now, $\mathbb{E}_{\mathcal{D}} [\hat{f}(X)] = \frac{1}{k} \sum_{i=1}^k \mathbb{E}_{\mathcal{D}} [\hat{f}_i(X)] = \mathbb{E}_{\mathcal{D}} [\hat{f}_1(X)]$ because $\hat{f}_i$ are identically distributed. Hence the bias term with the averaged estimator is exactly same as the bias term with the individual estimators. However, $\text{var}_{\mathcal{D}}(\hat{f}) = \frac{1}{k^2} \sum_{i=1}^k \text{var}_{\mathcal{D}}(\hat{f}_i)$ as

²Statisticians call this itself as estimation error.

$\hat{f}_1, \dots, \hat{f}_k$ are mutually independent. Further, as their are identically distributed, $\text{var}_{\mathcal{D}}(\hat{f}) = \frac{1}{k} \text{var}_{\mathcal{D}}(\hat{f}_1)$. In other words, the variance of the estimator decreases on averaging. Thus, overall, it is clear that the averaged estimator will generalize better than individual estimators.

However, there is a practical challenge in redeeming this statistical advantage. Typically we are only given one training dataset of size m . In light of the above discussion, there are two immediate strategies: Use entire dataset and train to obtain $\hat{f}$. Or partition training set into k subsets and train using each to obtain $\hat{f}_1, \dots, \hat{f}_k$ and then employ averaging idea to obtain $\tilde{f}_k \equiv \frac{1}{k} \sum_{i=1}^k \hat{f}_i$. Let us analyze which option is better by recalling³ the results (10.2), (11.1) that hold in the underparametrized regime, and (33.2) that hold in the overparametrized regime.

Underparametrized Regime: In this regime, the estimator is the least squares one, given by $\hat{f}^{ls}(x) = y^\top D^\top (DD^\top)^{-1} \phi(x) = (D^\top w^* + n)^\top D^\top (DD^\top)^{-1} \phi(x) = w^{*\top} \phi(x) + n^\top D^\top (DD^\top)^{-1} \phi(x)$, where D is the $n \times m$ matrix with i^{th} column as $\phi(x_i)$. Let us assume the same simplifying assumptions as in chapter 10. Since, we assumed $\mathbb{E}[N] = 0$, $f^*(x) = w^{*\top} \phi(x)$ and $\mathbb{E}_{\mathcal{D}}[\hat{f}_i^{ls}(x)] = w^{*\top} \phi(x) + \mathbb{E}_{\mathcal{D}}[n^\top D^\top (DD^\top)^{-1} \phi(x)] = w^{*\top} \phi(x) + \mathbb{E}_{\mathcal{D}}[\mathbb{E}[n]^\top D^\top (DD^\top)^{-1} \phi(x)] = w^{*\top} \phi(x) = f^*(x)$. Third last equality is because N is assumed independent of X and the second last is because $\mathbb{E}[N]$ is assumed 0. In other words, the least square estimator is unbiased (bias is zero). From (10.2), we have expected estimation error, which is in this same as expected generalization error, which again is expected variance of $\hat{f}^{ls}$, is lower bounded by $\frac{n}{m}$ (ignoring any constants). So the variance of $\hat{f}_i^{ls}$ is lower bounded $\frac{n}{m/k} = \frac{kn}{m}$ and is unbiased. Hence, the lower bound on the variance of the corresponding averaged estimator $\tilde{f}_k^{ls}$ reduces to $\frac{1}{k} \text{var}_{\mathcal{D}}(\hat{f}_i^{ls}) = \frac{1}{k} \frac{kn}{m} = \frac{n}{m}$. So the generalization of the averaged predictor is same as that of the predictor trained with entire dataset. In other words, averaging is not beneficial. This is perhaps not surprising because lower bounds are achieved by the “simplest” problems and if already the estimation is most efficient (lower bounds are hit) then averaging cannot improve further.

Now, let us consider the story in terms of the worst case bound (11.1) $\sqrt{\frac{n}{m}}$ i.e., the learning problem is the “hardest”. In such cases, variance of $\hat{f}_i^{ls}$ is upper bounded by $\sqrt{\frac{n}{m/k}} = \sqrt{\frac{nk}{m}}$ and hence that with $\tilde{f}_k^{ls}$ is $\frac{1}{k} \sqrt{\frac{nk}{m}} = \sqrt{\frac{n}{mk}}$. This suggests that averaging will help and maximum statistical benefit of averaging can be realized with maximum k . But since we assumed underparametrization, we need $\frac{m}{k} \geq n$ so the maximum possible k is $\frac{m}{n}$. Substituting this gives variance upper bound for $\tilde{f}_k^{ls}$ as $\sqrt{\frac{n}{m \cdot \frac{m}{n}}} = \frac{n}{m}$. In other words, averaging has the amazing

³Note that since the results assumed $\mathbb{E}[N] = 0$, expected estimation error is same as expected generalization error.

potential to render even the “hardest” problems as the “simplest” ones, improving the generalization by orders of magnitude.

Now, let us consider the Gaussian input special case in section 3.8.1 in Bach [2024]. From equation (3.8) in that book, expected generalization error ($\text{var}_{\mathcal{D}}(\hat{f}^{ls})$) is exactly equal to $\mathbb{E}[N^2] \frac{n}{m-n-1}$, assuming $m > n+1$. Further assuming $m/k > n+1$, $\text{var}_{\mathcal{D}}(\hat{f}_i^{ls})$ is $\mathbb{E}[N^2] \frac{n}{m/k-n-1}$. So, $\text{var}_{\mathcal{D}}(\hat{f}_k^{ls}) = \frac{1}{k} \mathbb{E}[N^2] \frac{n}{m/k-n-1} = \mathbb{E}[N^2] \frac{n}{m-kn-k}$, which is always greater than $\mathbb{E}[N^2] \frac{n}{m-n-1}$ for any $1 < k < \frac{m}{n+1}$. In other words, there may be some special cases where averaging always worsens the generalization.

Overparametrized Regime: In this case, the estimator is the least-norm interpolant $\hat{f}^{ln}(x) = y^\top (D^\top D)^{-1} D\phi(x)$. Let us make the same simplifying assumptions as in chapter 33. This is not an unbiased estimator: $\mathbb{E}_{\mathcal{D}}[\hat{f}^{ln}(x)] = \mathbb{E}_{\mathcal{D}}[y^\top (D^\top D)^{-1} D\phi(x)] = \mathbb{E}_{\mathcal{D}}[(D^\top w^* + n)^\top (D^\top D)^{-1} D\phi(x)] = \mathbb{E}_{\mathcal{D}}[w^{*\top} D (D^\top D)^{-1} D\phi(x)]$. Last equality is true because $\mathbb{E}[N] = 0$ and N is assumed independent of X . From this it is clear that the bias term in the expected generalization error is the second term in (33.1). Hence, the first term must be the variance term. Following the derivation in chapter 33, the first term in (33.2) is the bias term i.e., $\|w^*\|^2 \left(\frac{n-m}{n}\right)$, and the second term is the variance term i.e., $\mathbb{E}[N^2] \frac{m}{n-m-1}$, where $n > m+1$. Now, the bias term with averaging is $\|w^*\|^2 \left(\frac{n-m/k}{n}\right)$, while the variance term becomes $\frac{1}{k} \mathbb{E}[N^2] \frac{m/k}{n-m/k-1}$ for $1 \leq k \leq m$. So, overall, by averaging the bias term increases and the variance term decreases. So there is a non-trivial bias-variance trade-off. Hence, the best k is given by:

$$(39.1) \quad \arg \min_{k=1, \dots, m} \|w^*\|^2 \left(\frac{n-m/k}{n}\right) + \mathbb{E}[N^2] \frac{m}{k(kn-m-k)}$$

If k was real, this reduces to finding roots of $ak^2 + bk + c$, where $a = \|w^*\|^2 mn(n-1)^2$ and $b = -2mn(n-1)(\|w^*\|^2 m + \mathbb{E}[N^2]n)$ and $c = m^2 n (\|w^*\|^2 m + \mathbb{E}[N^2]n)$.

In summary, sometimes (for “easy” problems?) averaging does not help, sometimes (for special cases) averaging worsens the generalization, sometimes (for “hard” problems) it improves generalization. Also, in general, there seems to be a best value of k and needs to be chosen carefully. If chosen arbitrarily, it may worsen generalization. Whenever a careful theoretical analysis (like above) is cumbersome, it is easiest to choose k based on empirical validation i.e. consider k as a hyperparameter and perform cross-validation to choose it (refer fig 10.1 in Bach [2024] for illustration).

Finally, while from the perspective of generalization, k needs to be chosen carefully, from a computational perspective, larger the k the better. This is because all the models can be trained in parallel and each training costs lesser as

m reduces to m/k . Ofcourse, resource restrictions like servers/cores may restrict k to some practical value. A combination of these ideas are fundamental to the topic of Federated Machine Learning.

Bagging is a generalization of the idea in averaging where more sophisticated sampling techniques are used to create the k datasets. Interested students may read section 10.1.2 in Bach [2024] or section 18.3 in Murphy [2022].

Lecture 40

Boosting

If I have to summarize the crux of batch supervised learning, perhaps it would be controlling generalization error. It is challenging because it involves two terms $\text{model error} + \text{estimation error}$ or $\text{bias} + \text{variance}$ etc., which are (usually) opposing. Along this course we understood various ways of controlling generalization error: (i) modelling assumptions (inductive bias) (ii) regularization and its avatars in non-parametric modelling (kernel methods) and overparametrization (deep learning) (iii) ensembling etc.

Overall, it seems our approach in this course till now was always *liberal*¹ in the sense that our quest was for universal models so that the techniques like regularization or ensembling can then be thrown in to control the estimation error. How about the converse *conservative/ahimsic* approach? How about a quest for the minimalist model, that can then be unregularized (boosted) to fit the data at hand? A methodology that formalizes this approach is [Boosting](#)².

The first ingredient in boosting is a minimalist model, [base model](#)³, like, say ReLU activated linear functions over inputs in case of regression, or, in case of classification, so-called [decision stumps](#), which are sign activated 1-d linear functions e.g., $f_{ia}(x) = \text{sign}(x_i - a)$, where x_i is i^{th} feature of x and a is the threshold. The parameters of this model are i, a . Unlike linear, kernel, deep families of models, these examples are just simple models and do not form a family of models that is universal etc.

While this base model, $\mathcal{F}$, may be too simple and may not even achieve

¹because we were afraid of not being *inclusive* and omitting the Bayes optimal

²In simple words, to be practical, liberalism needs brakes while conservativeness needs boosting/acceleration. Hence, till now we needed regularization (brakes), but now we need boosting (unregularization).

³Base learner is a more commonly used terminology.

low enough training set error, the linear span⁴ of $\mathcal{F}$ may be rich. For example, we already know that the linear span of ReLU activated linear functions (i.e., single layer neural nets) can approximate any function. So the next idea in boosting is to consider a sequence of increasingly complex models: linear combinations of one, two, three, ..., elements from $\mathcal{F}$. i.e., $f^1(x) = \beta_1 f_{w_1}(x)$, $f^2(x) = \beta_1 f_{w_1}(x) + \beta_2 f_{w_2}(x)$, ..., $\sum_{t=1}^T \beta_t f_{w_t}(x)$, ..., $f_t \in \mathcal{F} \forall t$. The parameters β_t, w_t need to be learnt. But unlike in case of shallow learning (single layer neural nets), the width (T) is not predecided (as a high enough value) before training is performed. Analogous to online learning, training is done in a sequential manner. So the sequence of learnt functions becomes more and more complex sequentially. One can then stop/resume at/from any stage. This also helps control the estimation error: i.e., while estimating the second-order linear combinations in the second stage in the sequence, the first term in the linear combination is assumed to be fixed. Similarly, while learning β_t, w_t , the previous stage parameters are assumed to be fixed. However, unlike in online learning⁵, all m training points are used for training/learning at all stages, t . The problem solved at t^{th} stage is:

$$(40.1) \quad \min_{\beta_t, w_t} \mathbb{E} \left[l \left(f^{t-1}(X) + \beta_t f_{w_t}(X), Y \right) \right]$$

The expectation could be wrt. the underlying likelihood, in which case, this is risk minimization or it could be wrt. the empirical likelihood, in which case, this is ERM. (40.1) highlights the greedy nature of boosting as an algorithm. In order to understand this problem more closely, let us take two special cases: square-loss and exponential loss.

40.1 Square loss: Matching Pursuit

This part is from [section 10.3.3 in Bach \[2024\]](#). With squared loss and empirical likelihood (40.1) reads as:

$$\begin{aligned} & \arg \min_{\beta_t, w_t} \sum_{i=1}^m \left(f^{t-1}(x_i) + \beta_t f_{w_t}(x_i) - y_i \right)^2 \\ &= \arg \min_{\beta_t, w_t} \beta_t^2 \left(\sum_{i=1}^m f_{w_t}(x_i)^2 \right) - 2\beta_t \sum_{i=1}^m f_{w_t}(x_i) (y_i - f^{t-1}(x_i)) \end{aligned}$$

⁴LIN(S), linear span of S , is the set of all linear combinations of elements from S .

⁵Recall that in online learning w_t uses only supervision from $1, \dots, t-1$ rounds.

The problem wrt β_t is a convex 1-d quadratic, with solution as $\beta_t = \frac{\sum_{i=1}^m f_{w_t}(x_i)(y_i - f^{t-1}(x_i))}{\sum_{i=1}^m f_{w_t}(x_i)^2}$,
 which is nothing but the coefficient of projection of residual $\delta_y^{t-1} \equiv \begin{bmatrix} y_1 - f^{t-1}(x_1) \\ \vdots \\ y_m - f^{t-1}(x_m) \end{bmatrix}$
 onto $\begin{bmatrix} f_{w_t}(x_1) \\ \vdots \\ f_{w_t}(x_m) \end{bmatrix}$. Substituiting this optimal β_t , the problem wrt. w_t is:

$$\begin{aligned} & \arg \max_{w_t} \frac{\sum_{i=1}^m f_{w_t}(x_i)(y_i - f^{t-1}(x_i))}{\sqrt{\sum_{i=1}^m f_{w_t}(x_i)^2}} \\ &= \arg \max_{w_t} \sum_{i=1}^m \bar{f}_{w_t}(x_i) (y_i - f^{t-1}(x_i)), \\ &= \arg \min_{u_t \in \bar{\mathcal{F}}_{\mathcal{D}}} \|u_t - \delta_y^{t-1}\|^2 \end{aligned}$$

where, $\bar{f}(x) = \frac{f(x)}{\sqrt{\sum_{i=1}^m f(x_i)^2}}$ and $\bar{\mathcal{F}}_{\mathcal{D}} \equiv \left\{ u \in \mathbb{R}^m \mid \exists w \ni u_j = \frac{f_w(x_j)}{\sqrt{\sum_{i=1}^m f_w(x_i)^2}} \forall j = 1, \dots, m \right\}$.
 In other words, the problem wrt w_t is same as projecting the residual δ_y^{t-1} onto $\bar{\mathcal{F}}_{\mathcal{D}}$. So implementing boosting with square loss is straight-forward. Also, this gives us the recursion: $\|\delta_y^t\| = \|(I - P_{\bar{\mathcal{F}}_{\mathcal{D}}})(\delta_y^{t-1})\| \leq \|(I - P_{\bar{\mathcal{F}}_{\mathcal{D}}})\| \|\delta_y^{t-1}\|$, where $P_{\bar{\mathcal{F}}_{\mathcal{D}}}$ is the projection operator. Solving this leads to $\|\delta_y^T\| \leq \|(I - P_{\bar{\mathcal{F}}_{\mathcal{D}}})\|^T \|y\|$, showing exponentially fast convergence.

Ofcourse, if $\bar{\mathcal{F}}_{\mathcal{D}}$ is closed convex, then this projection exists uniquely and if it is closed, then solution exists. This projection interpretation makes it a so-called [matching pursuit/dictionary learning](#) problem.

40.2 Exponential loss: AdaBoost

This part is from [section 10.3.4 in Bach \[2024\]](#). With $\mathcal{F}$ as some simple classifier model such as decision stumps, with exponential loss, and taking expectation wrt.

empirical likelihood (40.1) reads as:

$$\begin{aligned}
\hat{R}[f^t] &= \min_{\beta_t, w_t} \sum_{i=1}^m e^{-y_i(f^{t-1}(x_i) + \beta_t f_{w_t}(x_i))} \\
&= \min_{\beta_t, w_t} \sum_{i=1}^m e^{-y_i f^{t-1}(x_i)} e^{-\beta_t y_i f_{w_t}(x_i)} \\
&= \hat{R}[f^{t-1}] \min_{\beta_t, w_t} \sum_{i=1}^m \pi_i e^{-\beta_t y_i f_{w_t}(x_i)}, \quad \pi_i \equiv \frac{e^{-y_i f^{t-1}(x_i)}}{\hat{R}[f^{t-1}]} \\
&= \hat{R}[f^{t-1}] \min_{w_t} \left\{ \underbrace{\min_{\beta_t} e^{\beta_t} \left(\sum_{i: y_i \neq f_{w_t}(x_i)} \pi_i \right) + e^{-\beta_t} \left(1 - \sum_{i: y_i \neq f_{w_t}(x_i)} \pi_i \right)}_{\text{optimal } \beta_t = \frac{1}{2} \log \left(\frac{1 - \sum_{i: y_i \neq f_{w_t}(x_i)} \pi_i}{\sum_{i: y_i \neq f_{w_t}(x_i)} \pi_i} \right). \text{ Optimal objective} = 2 \sqrt{\left(\sum_{i: y_i \neq f_{w_t}(x_i)} \pi_i \right) \left(1 - \sum_{i: y_i \neq f_{w_t}(x_i)} \pi_i \right)}} \right\} \\
&= 2\hat{R}[f^{t-1}] \min_{w_t} \sqrt{\left(\sum_{i: y_i \neq f_{w_t}(x_i)} \pi_i \right) \left(1 - \sum_{i: y_i \neq f_{w_t}(x_i)} \pi_i \right)}
\end{aligned}$$

Now, if $\mathcal{F}$ is such that $\sum_{i: y_i \neq f_{w_t}(x_i)} \pi_i < \frac{1}{2}$ at optimality, then we have $\hat{R}[f^t] < \hat{R}[f^{t-1}]$. In other words, boosting works. More importantly, the only case where it does not work is when $\sum_{i: y_i \neq f_{w_t}(x_i)} \pi_i = \frac{1}{2}$ at optimality. This is because, if $\sum_{i: y_i \neq f_{w_t}(x_i)} \pi_i > \frac{1}{2}$, then f_{w_t} can be chosen as $-f_{w_t}$ (negate the classifier). Ofcourse, we assume $f \in \mathcal{F} \Rightarrow -f \in \mathcal{F}$, which is reasonable.

Moreover, if it can be guaranteed that $\sum_{i: y_i \neq f_{w_t}(x_i)} \pi_i \leq \frac{1}{2} - \epsilon, \epsilon > 0$ (formally known as weak learner), then $\hat{R}[f^T] \leq (1 - 2\epsilon) \hat{R}[f^{T-1}] \leq \dots \leq (1 - 2\epsilon)^T \hat{R}[f^0]$. So again, exponentially fast convergence. Finally, if one stops after round T , $\text{sign}(f^T(x))$ is the predictive classifier.

Now this result is very satisfying: as long as one can guarantee the minimum (weighted) training set error will never be exactly 0.5, exponentially rapid boosting can be performed to decay empirical risk to zero.

This can be extended to any smooth loss as discussed in section 10.3.5 in Bach [2024], specifically [equations \(10.14\), \(10.15\)](#).

Lecture 41

Structured Prediction: Graphical Models

Lecture 42

Models for Sequences: RNN encoder & decoder

Bibliography

- M. Anthony and P.L. Bartlett. *Neural Network Learning: Theoretical Foundations*. Cambridge University Press, 1999. ISBN 9780521573535. URL <https://books.google.co.in/books?id=UH6XRoEQ4h8C>.
- Jimmy Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *ArXiv*, abs/1607.06450, 2016. URL <https://api.semanticscholar.org/CorpusID:8236317>.
- Francis Bach. *Learning Theory from First Principles*. The MIT Press (https://www.di.ens.fr/~fbach/ltfp_book.pdf), 2024. ISBN 9780262049443.
- Francis Bach and Lenaïc Chizat. Gradient descent on infinitely wide neural networks: Global convergence and generalization, 2021. URL <https://arxiv.org/abs/2110.08084>.
- Peter L. Bartlett, Philip M. Long, Gábor Lugosi, and Alexander Tsigler. Benign overfitting in linear regression. *Proceedings of the National Academy of Sciences*, 117(48):30063–30070, April 2020. ISSN 1091-6490. doi: 10.1073/pnas.1907378117. URL <http://dx.doi.org/10.1073/pnas.1907378117>.
- Lenaïc Chizat, Edouard Oyallon, and Francis Bach. On lazy training in differentiable programming, 2020. URL <https://arxiv.org/abs/1812.07956>.
- Aaron Defazio, Francis Bach, and Simon Lacoste-Julien. Saga: a fast incremental gradient method with support for non-strongly convex composite objectives. In *Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 1*, NIPS’14, page 1646–1654, Cambridge, MA, USA, 2014. MIT Press.
- John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(61):2121–2159, 2011. URL <http://jmlr.org/papers/v12/duchi11a.html>.

- V. Feldman, V. Guruswami, P. Raghavendra, and Yi Wu. Agnostic learning of monomials by halfspaces is hard. *IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 385–394, 2009.
- Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer Series in Statistics. Springer New York Inc., New York, NY, USA, 2001.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS '20*, Red Hook, NY, USA, 2020. Curran Associates Inc. ISBN 9781713829546.
- Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989. ISSN 0893-6080. doi: [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8). URL <https://www.sciencedirect.com/science/article/pii/0893608089900208>.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 448–456, Lille, France, 07–09 Jul 2015. PMLR. URL <https://proceedings.mlr.press/v37/ioffe15.html>.
- Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. *CoRR*, abs/1806.07572, 2018. URL <http://arxiv.org/abs/1806.07572>.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. URL <https://arxiv.org/abs/1412.6980>.
- Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of Machine Learning*. The MIT Press (<https://cs.nyu.edu/~mohri/mlbook/>), 2 edition, 2018.
- Kevin P. Murphy. *Probabilistic Machine Learning: An introduction*. MIT Press, 2022. URL probml.ai.
- Kevin P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2023. URL probml.ai.
- Gareth O. Roberts and Richard L. Tweedie. Exponential convergence of langevin distributions and their discrete approximations. *Bernoulli*, 2:341–363, 1996. URL <https://api.semanticscholar.org/CorpusID:18787082>.

Bernhard Scholkopf and Alexander J. Smola. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press, Cambridge, MA, USA, 2001. ISBN 0262194759.

Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press (<https://www.cs.huji.ac.il/~shais/UnderstandingMachineLearning/understanding-machine-learning-theory-algorithms.pdf>), New York, NY, USA, 2014. ISBN 1107057132, 9781107057135.

Gal Vardi. On the implicit bias in deep-learning algorithms. *Commun. ACM*, 66(6):86–93, May 2023. ISSN 0001-0782. doi: 10.1145/3571070. URL <https://doi.org/10.1145/3571070>.